

UNIVERSIDADE FEDERAL
DE ALAGOAS

UNIVERSIDADE FEDERAL DE ALAGOAS
INSTITUTO DE COMPUTAÇÃO
CIÊNCIA DA COMPUTAÇÃO
Tópicos em Engenharia de Software 2026.2

Carla Evellyn Ferreira dos Santos
Sthefany Barboza de Lima

Relatório do Projeto
LPS para Sistemas de Gerenciamento de Eventos Científicos usando Microservices e Componentes de Software [e Agentes de IA]

Repositório no GitHub: https://github.com/sthefanyblima/lps-eventos-cientificos

Maceió - AL
2026

SUMÁRIO

1 ENUNCIADO DO PROJETO 3
2 METODOLOGIA ÁGIL ESCOLHIDA 5
3 TÉCNICAS DE REUSO DE SOFTWARE QUE FORAM UTILIZADAS 6
4 DESENVOLVIMENTO DO PROJETO SEGUNDO A METODOLOGIA ÁGIL E AS TÉCNICAS DE REUSO DE SOFTWARE 7
5 PRODUCT BACKLOG E SPRINT BACKLOG PARA CADA SPRINT 9
5.1 Diagrama de Features 11
5.2 Arquitetura em Camadas e Diagrama de Variabilidade (pontos de variação e variantes) 13
5.3 Diagrama de Componentes de Software e Diagrama de Variabilidade (pontos de variação e variantes) 16
5.4 Tela de Configuração (seletor de features) e construção de um novo Produto 21
6 ARQUITETURA DO SISTEMA BASEADA EM MICROSERVICES 27
6.1 Arquitetura do Sistema usando Microservices (com recomendação pelo perfil do usuário) 27
6.2 Especificação (código documentado) da API Gateway 31
6.3 Especificação (código documentado) de cada Microservice 36
6.3.1 Banco de dados: MySQL, Modelo ER e normalização 40
6.3.2 Service Registry — porta 8761 (núcleo) 44
6.3.3 Users Service — porta 8001 (obrigatório) 48
6.3.4 Events Service — porta 8002 (obrigatório) 51
6.3.5 Registration Service — porta 8003 (obrigatório, VP Pagamento) 54
6.3.6 Recommendation Service — porta 8004 (obrigatório, VP Estratégia) 57
6.3.7 Submissions Service — porta 8005 (opcional, VP Formatos) 59
6.3.8 Reviews Service — porta 8006 (opcional, VP Política de Revisão) 62
6.3.9 Certificates Service — porta 8007 (opcional, VP Tipos de Certificado) 64
6.3.10 Notifications Service — porta 8008 (opcional, VP Canais) 66
6.3.11 AI Agent Service — porta 8009 (opcional, VP Provedor e Memória) 68
6.4 Tela mostrando as portas de todos os microservices em execução 79
6.5 Passo a passo para construir uma Aplicação (Produto) a partir da LPS 80
7 TECNOLOGIAS UTILIZADAS NA IMPLEMENTAÇÃO 83
8 TELAS DO FUNCIONAMENTO DO PRODUTO CONSTRUÍDO 85
9 CÓDIGO NO GITHUB 97
10 REFERÊNCIAS BIBLIOGRÁFICAS 98

1 ENUNCIADO DO PROJETO

Projeto (Modelagem) e Implementação de uma Linha de Produto de Software (LPS) usando Microservices e Componentes de Software [e Agentes de IA].

Obrigatoriamente o projeto deve ser uma LPS que use Microservices e Componentes de Software; opcionalmente pode usar Agentic AI. A linguagem sugerida é Python. O tema escolhido, dentre os exemplos propostos pela disciplina, é:

LPS para Sistemas de Gerenciamento de Eventos Científicos usando Microservices e Componentes de Software e Agentes de IA.

A LPS permite construir diferentes sistemas para eventos científicos a partir de um núcleo comum de artefatos reutilizáveis. Exemplos de produtos: um simpósio nacional com chamada de artigos e revisão duplo-cego; uma escola regional com pôsteres e pagamento por PIX; uma semana acadêmica gratuita, sem submissões. Como pede o enunciado, o sistema faz **recomendação considerando o perfil do usuário (histórico)** por meio de um microservice específico (Recommendation Service). Os microservices usam **banco de dados relacional (MySQL)**, com modelo Entidade-Relacionamento e tabelas normalizadas.

Relação com o artigo do seminário. Tizzei et al. (2017) mostram como técnicas de LPS podem ser integradas a uma arquitetura de microservices para desenvolver um SaaS multi-tenant e configurável (o WISE, da IBM Research), com reuso médio de 62% das linhas de código entre tenants. Este projeto aplica as mesmas técnicas ao domínio de eventos científicos: modelo de features, padrão Strategy para as variantes, parametrização por arquivo de configuração por tenant, separação em camadas dentro de cada serviço, Circuit Breaker e Docker. Cada evento (produto) faz o papel de um *tenant* do WISE.

Relação com o material de apoio da disciplina. A organização interna de cada microservice segue a figura de referência *Microservices e Componentes* (API Gateway/Zuul, Service Registry/Eureka e, em cada microservice, os componentes Controller, Service, Repository, Dto e Model ligados a um banco relacional). O microservice de agente segue a arquitetura de referência para agentes baseados em *foundation models* de Lu et al. (2024). A estrutura do relatório segue o documento *Elementos do Relatório do Projeto* (itens 1 a 10; o item 11 se aplica apenas a Linhas de Produto de Jogos).

PARTE 1 — MODELAGEM

2 METODOLOGIA ÁGIL ESCOLHIDA

Scrum (SCHWABER; SUTHERLAND, 2020):

- **Papéis:** Product Owner — o professor da disciplina, como cliente da LPS; Scrum Master — Sthefany Barboza de Lima; Development Team — Carla Evellyn Ferreira dos Santos e Sthefany Barboza de Lima.
- **Eventos:** quatro Sprints de uma semana. Cada Sprint teve Sprint Planning (definição do Sprint Backlog), Daily assíncrona (registrada nos commits), Sprint Review (demonstração do produto derivado) e Retrospective.
- **Artefatos:** Product Backlog em histórias de usuário com pontos (Fibonacci), Sprint Backlog com tarefas técnicas e, ao final de cada Sprint, um Incremento executável — um produto derivado e testado da LPS.
- **Definição de Pronto (DoD):** a feature está no modelo de features; está implementada como microservice ou componente; tem tabelas no MySQL quando guarda dados; foi testada (`pytest` e teste manual pelo gateway); está documentada (docstrings e Swagger) e versionada no GitHub.

3 TÉCNICAS DE REUSO DE SOFTWARE QUE FORAM UTILIZADAS

Linhas de Produto de Software, Microservices e Componentes de Software, além de **Agentes de IA** como feature opcional.

Quadro 1 – Técnicas de reuso de software aplicadas no projeto

Técnica de reuso	**Como foi aplicada**
Linha de Produto de Software (engenharia de domínio e de aplicação)	Modelo de features FODA com features obrigatórias, opcionais, alternativas (xor), grupos or e restrições *requires*. A derivação automática gera um produto a partir de uma seleção válida de features.
Microservices	Nove microservices de negócio, cada um com **seu próprio banco MySQL** (*database per service*), API REST e implantação independente; mais um **API Gateway** (papel do Zuul) e um **Service Registry** (papel do Eureka). Cada feature de primeiro nível é mapeada para um microservice.
Componentes de Software	(a) Dentro de cada microservice, componentes **controller, service, repository, model e dto**, como na figura de referência da disciplina. (b) Sete pontos de variação implementados como interfaces abstratas, com variantes concretas registradas pelo id da feature (padrão Strategy, como o `NETCDFParser` do WISE).
Parametrização / multi-tenancy	Um arquivo de configuração por produto (`products/<id>.json`), cabeçalho `X-Tenant` e um banco por tenant (`lps_<tenant>_<serviço>`), como no WISE.
Padrões de projeto	Strategy (variantes), Repository e DTO (acesso a dados), API Gateway, Service Registry/Discovery, Circuit Breaker (Nygard) e coreografia por eventos de domínio.
Agentic AI (opcional)	Microservice de agente organizado segundo a arquitetura de referência de Lu et al. (2024). Suas ferramentas são as APIs dos outros microservices do produto; o provedor (LLM ou local) e a memória de longo prazo são features variáveis.

Fonte: elaborado pelas autoras (2026).

4 DESENVOLVIMENTO DO PROJETO SEGUNDO A METODOLOGIA ÁGIL E AS TÉCNICAS DE REUSO DE SOFTWARE

O desenvolvimento seguiu os dois processos clássicos de LPS (POHL et al., 2005), organizados nas Sprints do Scrum:

- **Engenharia de Domínio (Sprints 1–3):** construção do núcleo de ativos reutilizáveis — modelo de features, núcleo comum `common/` (fábrica de serviços, banco de dados, segurança, descoberta), componentes `components/`, microservices `services/`, API Gateway e Service Registry.
- **Engenharia de Aplicação (Sprints 3–4):** derivação de produtos concretos (Tela de Configuração, CLI, launcher, Docker Compose) e validação com três produtos.

Diferente do WISE, que seguiu um processo de reengenharia orientado a features a partir de um sistema legado, aqui a abordagem foi **proativa**: o domínio foi analisado primeiro e as variantes foram planejadas desde o início. As decisões de projeto do artigo foram mantidas:

1. **Análise de domínio com modelo de features.** Técnicas de modelagem de microservices não tratam variabilidade explicitamente; por isso, como a equipe do WISE, o domínio foi modelado com features. Cada produto é a configuração de um tenant (seção 5.1).
2. **Microservices em torno de conceitos de negócio** (*bounded contexts*): usuários, evento, inscrição, recomendação, submissão, revisão, certificado, notificação e agente. Cada serviço esconde seu banco (*information hiding*) e só se comunica por REST.
3. **Modelagem dos dados.** Para cada microservice foi feito o Modelo ER do seu banco e as tabelas foram normalizadas até a 3ª Forma Normal (seção 6.3.1). Não há chave estrangeira física entre bancos de serviços diferentes: referências a dados de outro serviço são ids lógicos, resolvidos por chamada REST.
4. **Variabilidade em dois níveis**, como no artigo: no nível arquitetural, inclusão ou exclusão de microservices (features opcionais) e de operações da API (ex.: `/tracks` só existe com a feature Trilhas); no nível de projeto detalhado, componentes Strategy dentro dos serviços (pagamento, estratégia de recomendação, política de revisão etc.).
5. **Parametrização.** O que varia entre produtos fica no arquivo de configuração do produto, não no código (*twelve-factor app*). Cada requisição carrega o tenant no cabeçalho `X-Tenant` e cada tenant tem seus próprios bancos.

6. **Infraestrutura de microservices.** O API Gateway é o único ponto de entrada e valida o token JWT; os microservices se registram no Service Registry e o gateway descobre os endereços por ele.
7. **Isolamento de falhas.** O Circuit Breaker protege as chamadas entre serviços e as do gateway. Se um serviço opcional não faz parte do produto ou está fora do ar, o chamador degrada de forma graciosa (ex.: sem o Notifications Service, a publicação de eventos é *no-op*).
8. **Automação.** Derivação, validação, execução e testes são feitos por comandos (`python -m spl.cli ...`, `pytest`). Para cada produto é gerado um `docker-compose` com o MySQL, o registry, o gateway e apenas os microservices selecionados.

Medida de reuso (inspirada na avaliação do artigo). Como em Tizzei et al. (2017), foram contadas as linhas de código Python, sem linhas em branco, comentários e docstrings, dos módulos executados por cada produto:

Quadro 2 – Medida de reuso por produto (linhas de código Python executadas)

Produto	Módulos executados	LOC executadas	LOC do núcleo comum aos 3 produtos	% reutilizado do núcleo
SBES 2026	89	2 720	1 566	58%
ERBASE 2026	89	2 720	1 566	58%
Semana de Computação UFAL	53	1 566	1 566	100%

Fonte: elaborado pelas autoras (2026).

Todos os 89 módulos são compartilhados por pelo menos dois produtos. SBES e ERBASE executam os mesmos arquivos, mas com variantes diferentes em cada ponto de variação (classes Strategy distintas no mesmo módulo). O percentual fica próximo do relatado no artigo (62% em média por LOC). A engenharia de domínio (`spl/`: modelo de features, derivação, configurador, launcher) soma mais 391 LOC, reutilizadas por todos os produtos.

5 PRODUCT BACKLOG E SPRINT BACKLOG PARA CADA SPRINT

Product Backlog

Quadro 3 – Product Backlog

ID	História de usuário	Feature(s)	Prioridade	Pontos	Sprint
PB01	Como engenheira de domínio, quero um modelo de features validável para representar a variabilidade da LPS.	todas	Alta	8	1
PB02	Como desenvolvedora, quero um núcleo comum (fábrica de serviço, configuração, acesso a banco por serviço/tenant, cliente HTTP com circuit breaker) para não repetir código.	infraestrutura	Alta	5	1
PB03	Como projetista de dados, quero o Modelo ER e as tabelas normalizadas de cada microservice no MySQL.	todas	Alta	5	1
PB04	Como participante, quero me cadastrar e entrar com e-mail e senha, e manter meus interesses.	Gestão de Usuários	Alta	5	1
PB05	Como participante, quero ver e buscar a programação e filtrar por trilha.	Gestão do Evento, Trilhas	Alta	3	1
PB06	Como cliente do sistema, quero um API Gateway como ponto único de entrada, com autenticação JWT, e um Service Registry para descoberta dos serviços.	—	Alta	8	1
PB07	Como participante, quero me inscrever pagando conforme a política do evento (gratuito, PIX ou cartão).	Inscrição, Pagamento	Alta	5	2
PB08	Como participante, quero montar minha agenda de sessões.	Inscrição	Alta	3	2
PB09	Como participante, quero recomendações de sessões baseadas no meu perfil e histórico.	Recomendação	Alta	8	2
PB10	Como autor, quero submeter trabalhos nos formatos aceitos pelo evento.	Submissão	Média	5	2
PB11	Como revisor, quero revisar trabalhos segundo a política do evento (simples-cego, duplo-cego ou aberta).	Revisão	Média	5	3
PB12	Como participante ou autor, quero certificados de participação e de apresentação com código de verificação.	Certificados	Média	3	3
PB13	Como usuário, quero ser notificado por e-mail ou no aplicativo.	Notificações	Baixa	3	3

ID	História de usuário	Feature(s)	Prioridade	Pontos	Sprint
PB14	Como engenheira de aplicação, quero uma Tela de Configuração (seletor de features) que gere e execute um novo produto.	derivação	Alta	8	3
PB15	Como participante, quero um assistente com agente de IA que consulte a programação, recomende e adicione sessões à agenda, com guardrails e memória.	Assistente IA	Baixa	8	4
PB16	Como operadora, quero ver as portas e a saúde de todos os microservices.	observabilidade	Média	2	4
PB17	Como engenheira de aplicação, quero um docker-compose por produto (com MySQL) e documentação de execução no GitHub.	implantação	Média	3	4
PB18	Como professor, quero testes e três produtos de exemplo que demonstrem a variabilidade.	—	Média	3	4

Fonte: elaborado pelas autoras (2026).

Sprint Backlog

Sprint 1 — Fundação do domínio (meta: modelo de features, bancos e serviços de base funcionando atrás do gateway) · 34 pontos

Quadro 4 – Sprint Backlog — Sprint 1

Tarefa	História	Responsável	Status
Escrever `feature_model.json` (FODA) e as regras de validação (`spl/feature_model.py`)	PB01	Sthefany	Pronto
Implementar `common/` (config, `database.py`, `create_service`, client + CircuitBreaker)	PB02	Carla	Pronto
Modelo ER, normalização e criação das tabelas MySQL via SQLAlchemy	PB03	Carla	Pronto
Users Service (cadastro, login com senha e JWT, interesses, histórico)	PB04	Sthefany	Pronto
Events Service (trilhas, tags, sessões, busca, `/tracks` parametrizado)	PB05	Carla	Pronto
API Gateway (JWT, proxy, `X-Tenant`, 404 para feature ausente) e Service Registry	PB06	Sthefany	Pronto

Fonte: elaborado pelas autoras (2026).

Sprint 2 — Inscrição, recomendação e submissão · 21 pontos

Quadro 5 – Sprint Backlog — Sprint 2

Tarefa	História	Responsável	Status
Componente `PaymentMethod` (Gratuito, Pix, CartaoCredito)	PB07	Carla	Pronto
Registration Service (inscrição, pagamentos, confirmação PIX, agenda)	PB07, PB08	Carla	Pronto
Componente `RecommendationStrategy` (Conteúdo, Popularidade, Híbrida)	PB09	Sthefany	Pronto
Recommendation Service (perfil = interesses + histórico + agenda; histórico de recomendações no banco)	PB09	Sthefany	Pronto
Submissions Service (formatos = grupo or; coautores e palavras-chave)	PB10	Carla	Pronto

Fonte: elaborado pelas autoras (2026).

Sprint 3 — Features opcionais e derivação · 19 pontos

Quadro 6 – Sprint Backlog — Sprint 3

Tarefa	História	Responsável	Status
Componente `ReviewPolicy` e Reviews Service (decisão por média, anonimização)	PB11	Sthefany	Pronto
Componente `CertificateIssuer` e Certificates Service (HTML + verificação)	PB12	Carla	Pronto
Componente `NotificationChannel`, Notifications Service e publicação de eventos	PB13	Carla	Pronto
Derivação (`spl/derivation.py`), CLI e Tela de Configuração web	PB14	Sthefany	Pronto

Fonte: elaborado pelas autoras (2026).

Sprint 4 — Agentic AI, operação e entrega · 16 pontos

Quadro 7 – Sprint Backlog — Sprint 4

Tarefa	História	Responsável	Status
AI Agent Service segundo Lu et al. (2024): interação, memória, planejamento, execução, guardrails, caixa-preta; componente `AgentProvider` (LLM e Local)	PB15	Sthefany	Pronto
Página `/ports`, `/api/ports` e painel do registry com heartbeat	PB16	Carla	Pronto
`docker-compose` por produto (MySQL + registry + serviços), `Dockerfile`, README	PB17	Carla	Pronto
`pytest` (13 testes) e produtos SBES, ERBASE e Semana UFAL	PB18	Sthefany	Pronto

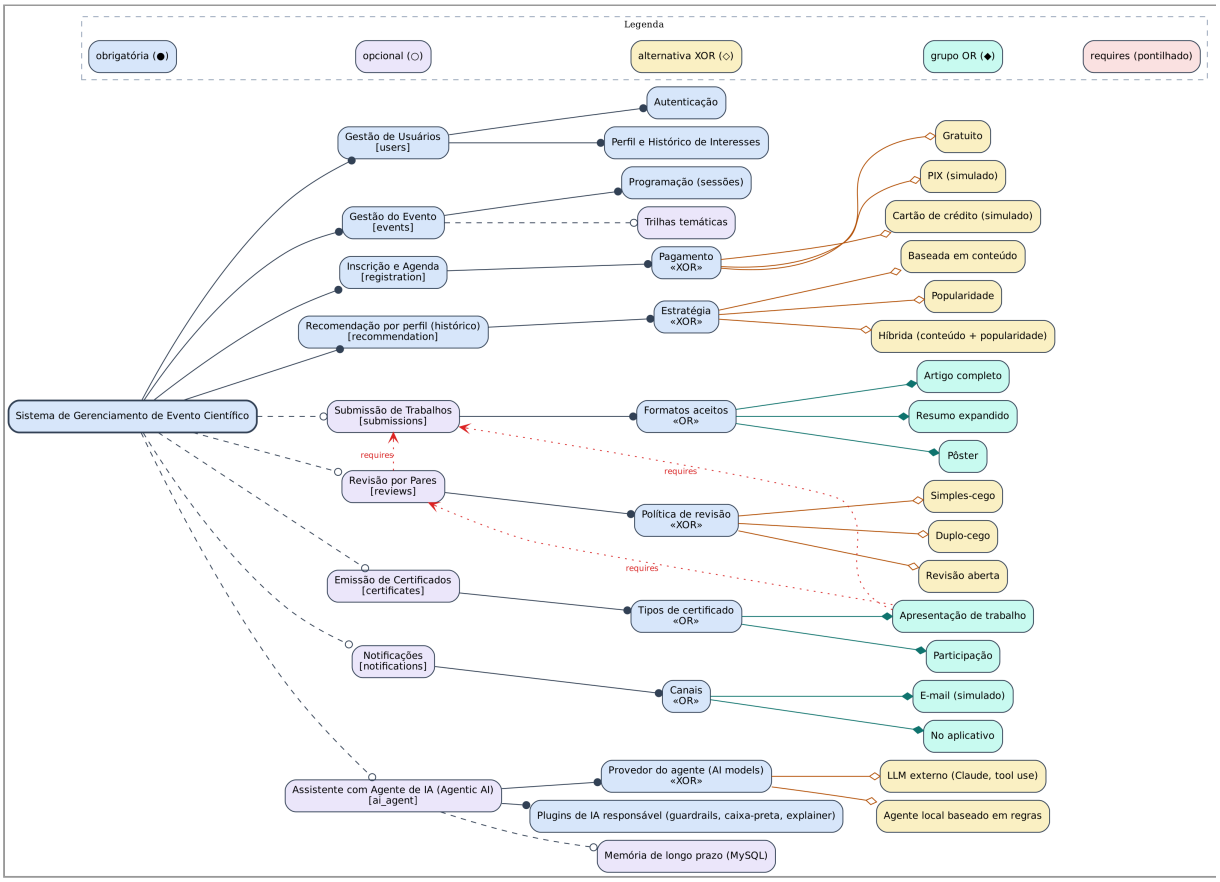
Fonte: elaborado pelas autoras (2026).

5.1 Diagrama de Features

Gerado automaticamente a partir de `feature_model.json`, em notação FODA: ● obrigatória; ○ opcional; ◇ alternativa (grupo XOR, exatamente uma); ◆ grupo OR

(uma ou mais); linhas pontilhadas vermelhas: restrições *requires*. Entre colchetes está o microservice que implementa cada feature de primeiro nível.

Figura 1 – Diagrama de Features da LPS



Fonte: elaborado pelas autoras (2026).

Restrições cruzadas: **Revisão por Pares requer Submissão de Trabalhos**; **Certificado de Apresentação requer Submissão e Revisão** (só um trabalho aceito gera certificado).

Configuração dos produtos (tenants), equivalente à Tabela 1 do artigo (as features obrigatórias foram omitidas):

Quadro 8 – Configuração dos produtos (tenants) derivados da LPS

Feature variável	**SBES 2026**	**ERBASE 2026**	**Semana UFAL**
Trilhas temáticas	X		
Pagamento: Gratuito / PIX / Cartão	Cartão	PIX	Gratuito
Recomendação: Conteúdo / Popularidade / Híbrida	Híbrida	Conteúdo	Popularidade
Submissão (Artigo · Resumo · Pôster)	Artigo, Resumo	Resumo, Pôster	
Revisão (Simples · Duplo · Aberta)	Duplo-cego	Simples-cego	

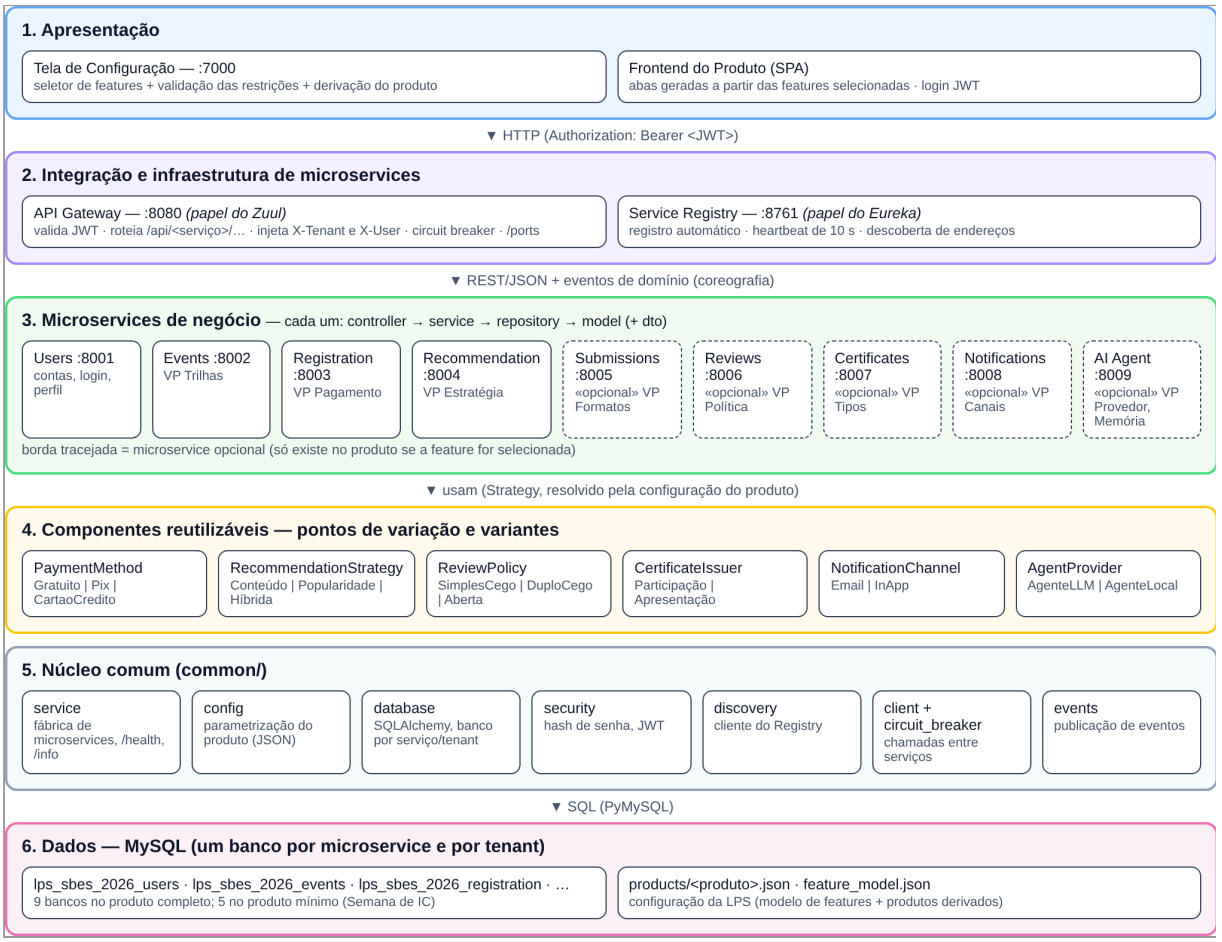
Feature variável	SBES 2026	ERBASE 2026	Semana UFAL
Certificados (Participação · Apresentação)	Ambos	Participação	Participação
Notificações (E-mail · In-app)	Ambos	In-app	
Assistente IA — provedor (LLM · Local)	LLM	Local	
Assistente IA — memória de longo prazo	X		
Microservices no produto	9 + gateway + registry	9 + gateway + registry	5 + gateway + registry
Bancos MySQL (um por microservice)	9	9	5

Fonte: elaborado pelas autoras (2026).

5.2 Arquitetura em Camadas e Diagrama de Variabilidade (pontos de variação e variantes)

A arquitetura tem seis camadas. Na camada 2 estão os dois elementos de infraestrutura da figura de referência da disciplina (API Gateway no papel do Zuul e Service Registry no papel do Eureka). Os microservices tracejados são opcionais e «VP» marca o ponto de variação que cada serviço contém.

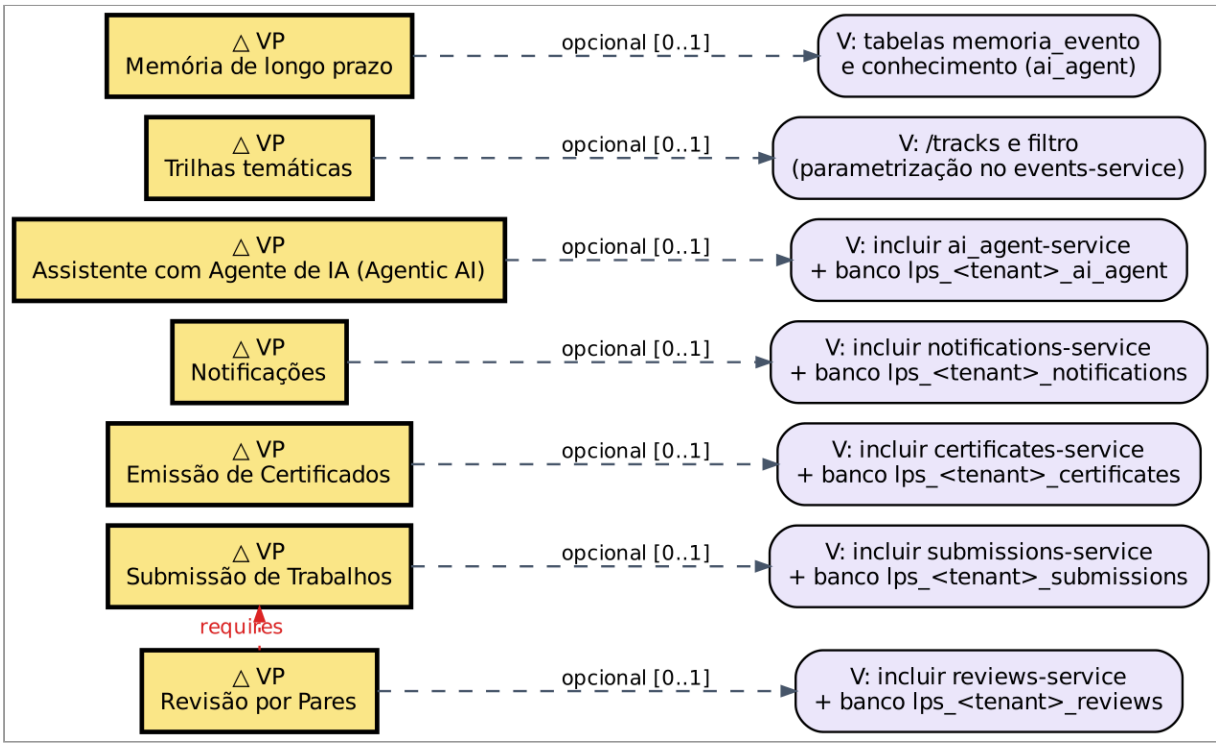
Figura 2 – Arquitetura em Camadas da LPS



Fonte: elaborado pelas autoras (2026).

Diagrama de Variabilidade no nível arquitetural (notação OVM; POHL et al., 2005): cada feature opcional de primeiro nível é um ponto de variação (VP) cuja variante é incluir o microservice — e o seu banco — no produto. Trilhas é resolvida por parametrização dentro do Events Service.

Figura 3 – Diagrama de Variabilidade — nível arquitetural



Fonte: elaborado pelas autoras (2026).

Quadro 9 – Pontos de variação no nível arquitetural

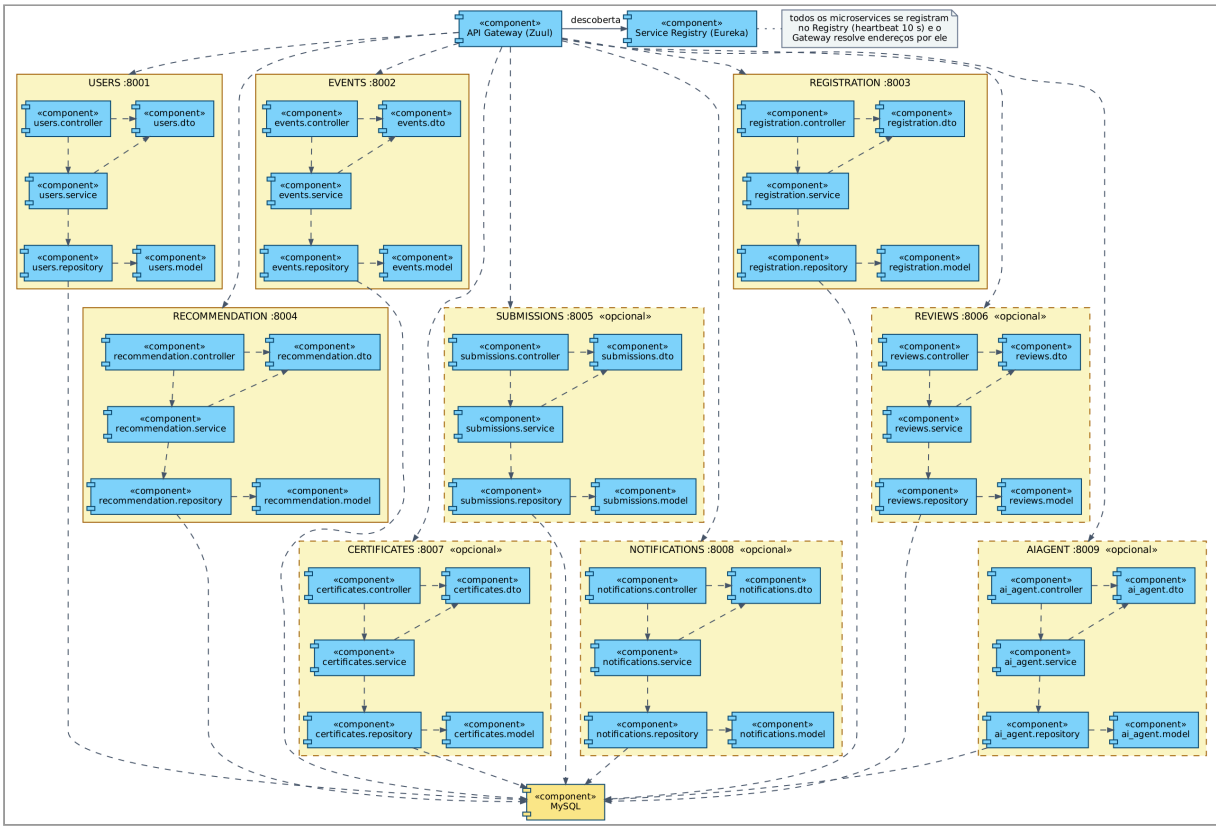
Ponto de variação (arquitetura)	**Variantes**	**Mecanismo de resolução**
Submissão de Trabalhos	incluir ou não `submissions` (+ banco `lps_<t>_submissions`)	derivação: serviço entra em `products/<id>.json` e no compose
Revisão por Pares	incluir ou não `reviews` (*requires* Submissão)	derivação
Emissão de Certificados	incluir ou não `certificates`	derivação
Notificações	incluir ou não `notifications`	derivação; publicação vira *no-op* sem ele
Assistente IA	incluir ou não `ai_agent`	derivação; aba e ferramentas aparecem só se presentes
Memória de longo prazo (Assistente IA)	usar ou não as tabelas `memoria_evento` e `conhecimento`	parametrização (`has_feature("MemoriaLongoPrazo")`)
Trilhas temáticas	expor `/tracks` e o campo trilha	parametrização (`has_feature("Trilhas")`)

Fonte: elaborado pelas autoras (2026).

5.3 Diagrama de Componentes de Software e Diagrama de Variabilidade (pontos de variação e variantes)

O diagrama de componentes segue o padrão da figura *Microservices e Componentes* da disciplina: o **API Gateway (Zuul)** recebe as requisições e consulta o **Service Registry (Eureka)**; cada microservice é formado pelos componentes **controller** (API REST), **dto** (objetos de transferência validados), **service** (regras de negócio), **repository** (acesso a dados) e **model** (entidades ORM); os repositories persistem no **MySQL**. Microservices tracejados são opcionais.

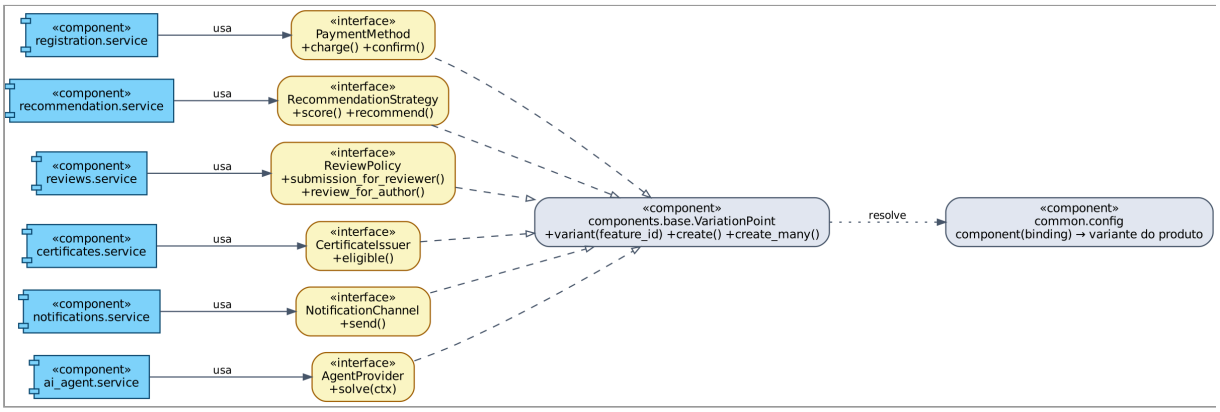
Figura 4 – Diagrama de Componentes de Software (padrão Controller–Service–Repository–Model–Dto)



Fonte: elaborado pelas autoras (2026).

Os services usam as interfaces dos **componentes reutilizáveis** de `components/`, que são os pontos de variação da LPS:

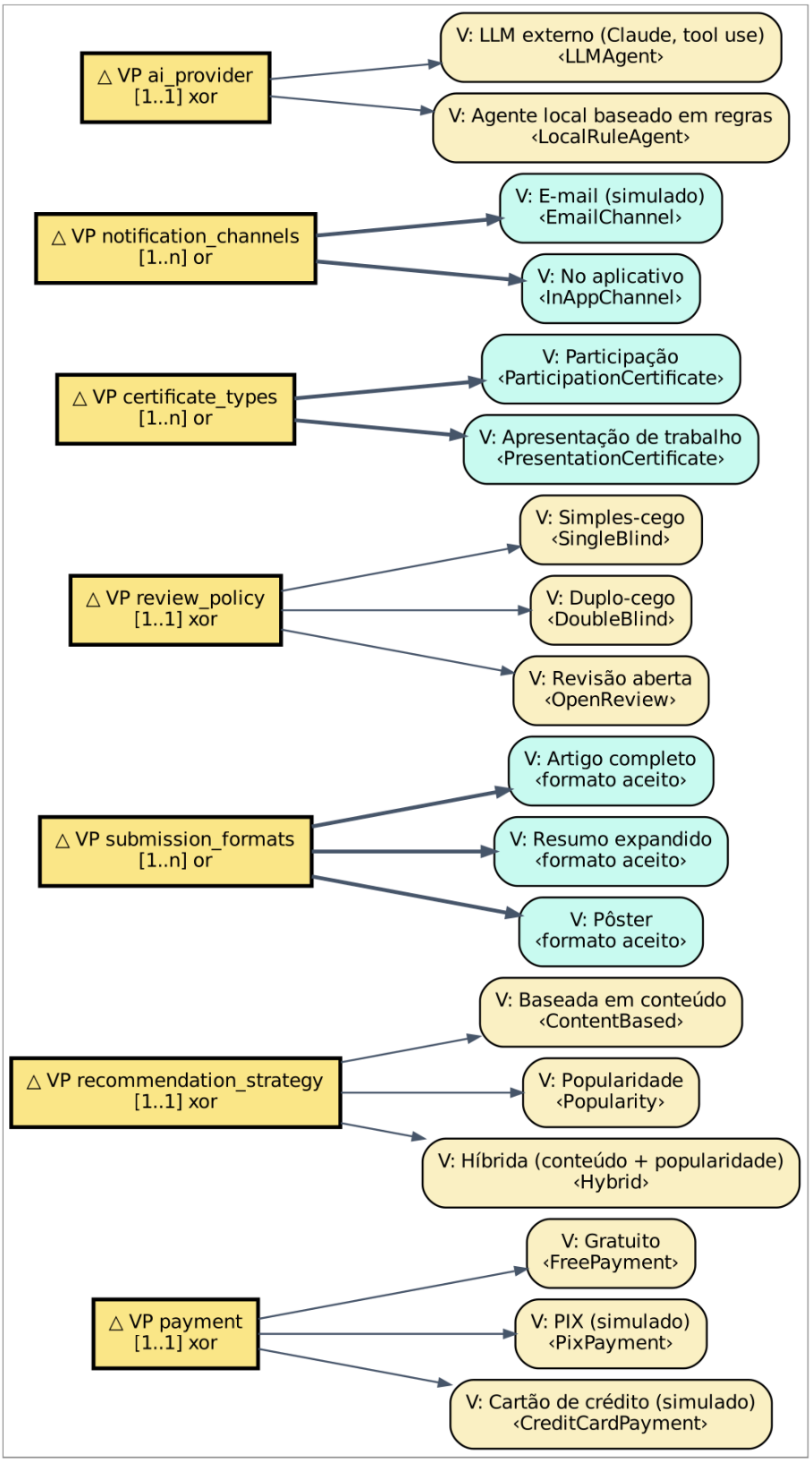
Figura 5 – Componentes reutilizáveis usados pelos services



Fonte: elaborado pelas autoras (2026).

Diagrama de Variabilidade no nível de componentes: cada ponto de variação é uma interface abstrata (Strategy) e cada variante é uma classe concreta registrada com o id da feature. VPs xor têm cardinalidade [1..1] (uma instância) e VPs or têm [1..n] (todas as selecionadas são instanciadas).

Figura 6 – Diagrama de Variabilidade — componentes



Fonte: elaborado pelas autoras (2026).

Quadro 10 – Pontos de variação no nível de componentes

Ponto de variação (binding)	**Interface**	**Variantes (classe)**	**Grupo**	**Microservice**
`payment`	`PaymentMethod`	`FreePayment`, `PixPayment`, `CreditCardPayment`	xor	registration
`recommendation_strategy`	`RecommendationStrategy`	`ContentBased`, `Popularity`, `Hybrid`	xor	recommendation
`review_policy`	`ReviewPolicy`	`SingleBlind`, `DoubleBlind`, `OpenReview`	xor	reviews
`certificate_types`	`CertificateIssuer`	`ParticipationCertificate`, `PresentationCertificate`	or	certificates
`notification_channels`	`NotificationChannel`	`EmailChannel`, `InAppChannel`	or	notifications
`ai_provider`	`AgentProvider`	`LLMAgent`, `LocalRuleAgent`	xor	ai_agent
`submission_formats`	parâmetro	`ArtigoCompleto`, `ResumoExpandido`, `Poster`	or	submissions

Fonte: elaborado pelas autoras (2026).

O mecanismo genérico de ponto de variação (`components/base.py`) liga o modelo de features ao código:

Código 1 – Arquivo `components/base.py`

```
"""Infraestrutura de componentes com pontos de variação.

Cada ponto de variação é uma **interface abstrata** (Strategy). Cada variante é
um **componente** concreto registrado com o *id da feature* do modelo. Em tempo
de inicialização, o microservice pede ao registro a variante resolvida na
configuração do produto — o cliente fica acoplado só à abstração, como o
``NETCDFLoader`` → ``NETCDFParser`` do WISE.
"""
from __future__ import annotations

from typing import Generic, TypeVar

T = TypeVar("T")


class VariationPoint(Generic[T]):
    def __init__(self, binding: str, interface: type):
        self.binding = binding
        self.interface = interface
        self._variants: dict[str, type] = {}

    def variant(self, feature_id: str):
        """Decorador: registra uma classe como variante do ponto de variação."""
        def deco(cls):
            if not issubclass(cls, self.interface):
                raise TypeError(f"{cls.__name__} não implementa {self.interface.__name__}")
            cls.feature_id = feature_id
            self._variants[feature_id] = cls
            return cls
        return deco

    def create(self, feature_id: str, **kwargs) -> T:
        try:
            return self._variants[feature_id](**kwargs)
```

```
        except KeyError:
            raise ValueError(f"Variante '{feature_id}' não existe em '{self.binding}'. "
                             f"Disponíveis: {list(self._variants)}") from None

    def create_many(self, feature_ids: list[str], **kwargs) -> list[T]:
        return [self.create(f, **kwargs) for f in feature_ids]

    @property
    def variants(self) -> list[str]:
        return list(self._variants)
```

Fonte: código do projeto (2026).

Exemplo de componente com suas variantes (`components/payment.py`; todos os pagamentos são simulados):

Código 2 – Arquivo `components/payment.py`

```
"""Ponto de variação **Pagamento** (xor): Gratuito | Pix | CartaoCredito.

Todos os pagamentos são SIMULADOS (projeto acadêmico): nenhum dado real é
processado nem enviado a terceiros.
"""
from __future__ import annotations

import hashlib
import uuid
from abc import ABC, abstractmethod

from .base import VariationPoint


class PaymentMethod(ABC):
    label = "?"

    @abstractmethod
    def charge(self, amount: float, payer: dict, data: dict) -> dict:
        """Retorna {status: approved|pending|refused, amount, details}."""

    def confirm(self, payment: dict) -> dict:
        return {**payment, "status": "approved"}


payment_vp: VariationPoint[PaymentMethod] = VariationPoint("payment", PaymentMethod)


@payment_vp.variant("Gratuito")
class FreePayment(PaymentMethod):
    label = "Gratuito"

    def charge(self, amount, payer, data):
        return {"status": "approved", "amount": 0.0, "details": "Evento gratuito —
inscrição confirmada"}


@payment_vp.variant("Pix")
class PixPayment(PaymentMethod):
    label = "PIX (simulado)"

    def charge(self, amount, payer, data):
        txid = uuid.uuid4().hex[:20].upper()
        code = f"SIMULADO|PIX|{txid}|{amount:.2f}"
        return {"status": "pending", "amount": amount,
                "details": "Use o código copia-e-cola e depois confirme o pagamento",
                "pix_code": code, "txid": txid}
```

```
@payment_vp.variant("CartaoCredito")
class CreditCardPayment(PaymentMethod):
    label = "Cartão de crédito (simulado)"
    TEST_PREFIX = "4111"   # só aceita números de teste

    @staticmethod
    def luhn(number: str) -> bool:
        digits = [int(d) for d in number if d.isdigit()][::-1]
        total = sum(d if i % 2 == 0 else (d * 2 - 9 if d * 2 > 9 else d * 2) for i, d in
enumerate(digits))
        return len(digits) >= 13 and total % 10 == 0

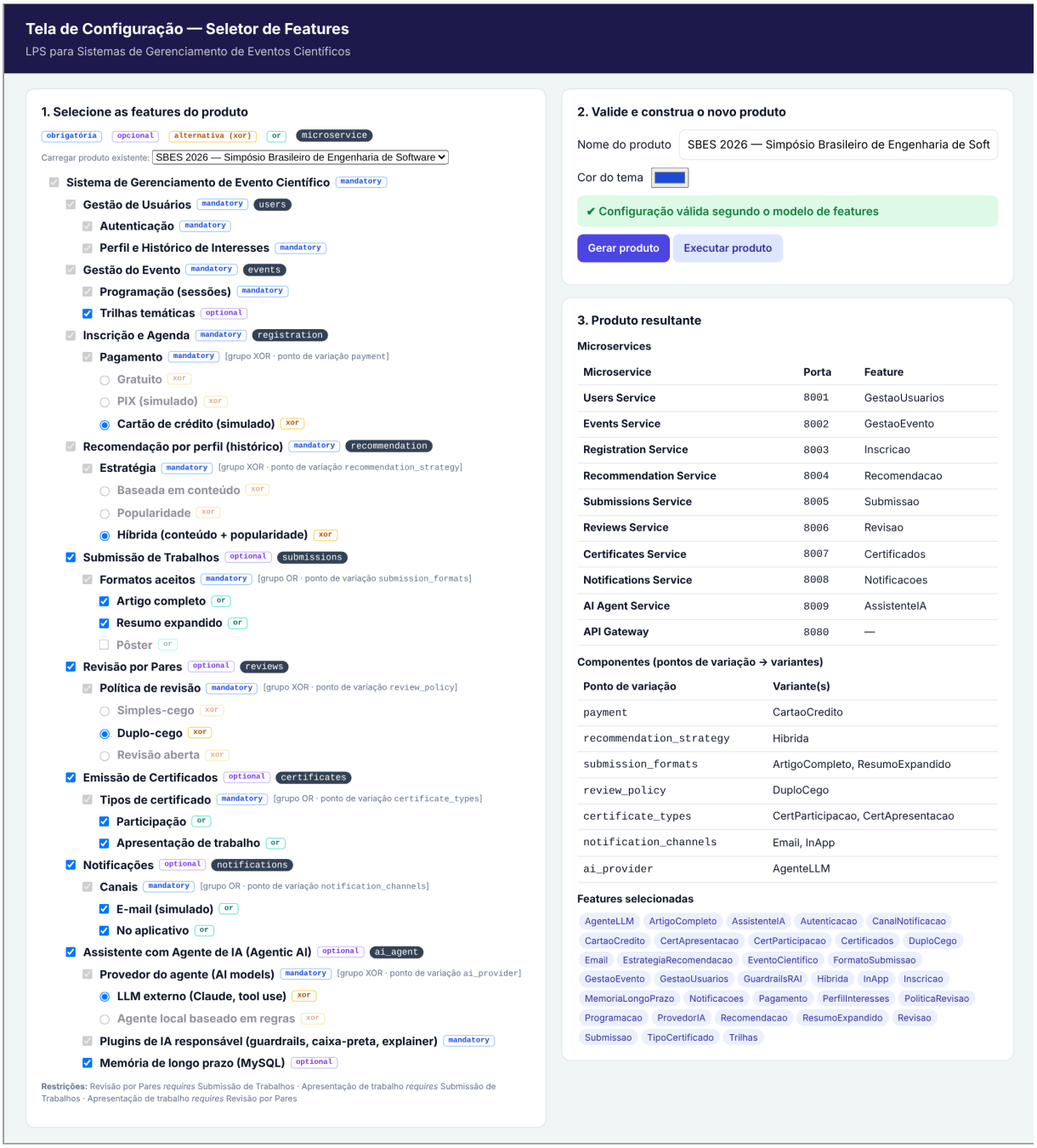
    def charge(self, amount, payer, data):
        number = str(data.get("card_number", "")).replace(" ", "")
        if not number.startswith(self.TEST_PREFIX) or not self.luhn(number):
            return {"status": "refused", "amount": amount,
                    "details": "Use o cartão de teste 4111 1111 1111 1111"}
        token = hashlib.sha256(number.encode()).hexdigest()[:12]
        return {"status": "approved", "amount": amount, "details": f"Aprovado (token
{token}, final {number[-4:]})"}
```

Fonte: código do projeto (2026).

5.4 Tela de Configuração (seletor de features) e construção de um novo Produto

A Tela de Configuração (`python -m spl.cli configurator`, em `http://127.0.0.1:7000`) mostra a árvore de features com caixas de seleção (opcionais e or) e botões de rádio (xor). Ela propaga as features obrigatórias, valida a configuração em tempo real contra o modelo (grupos e restrições) e mostra o produto resultante: microservices, portas, bancos MySQL e variantes de cada componente. **Gerar produto** escreve `products/<id>.json` e `products/<id>.compose.yml`; **Executar produto** sobe o registry, os microservices e o gateway.

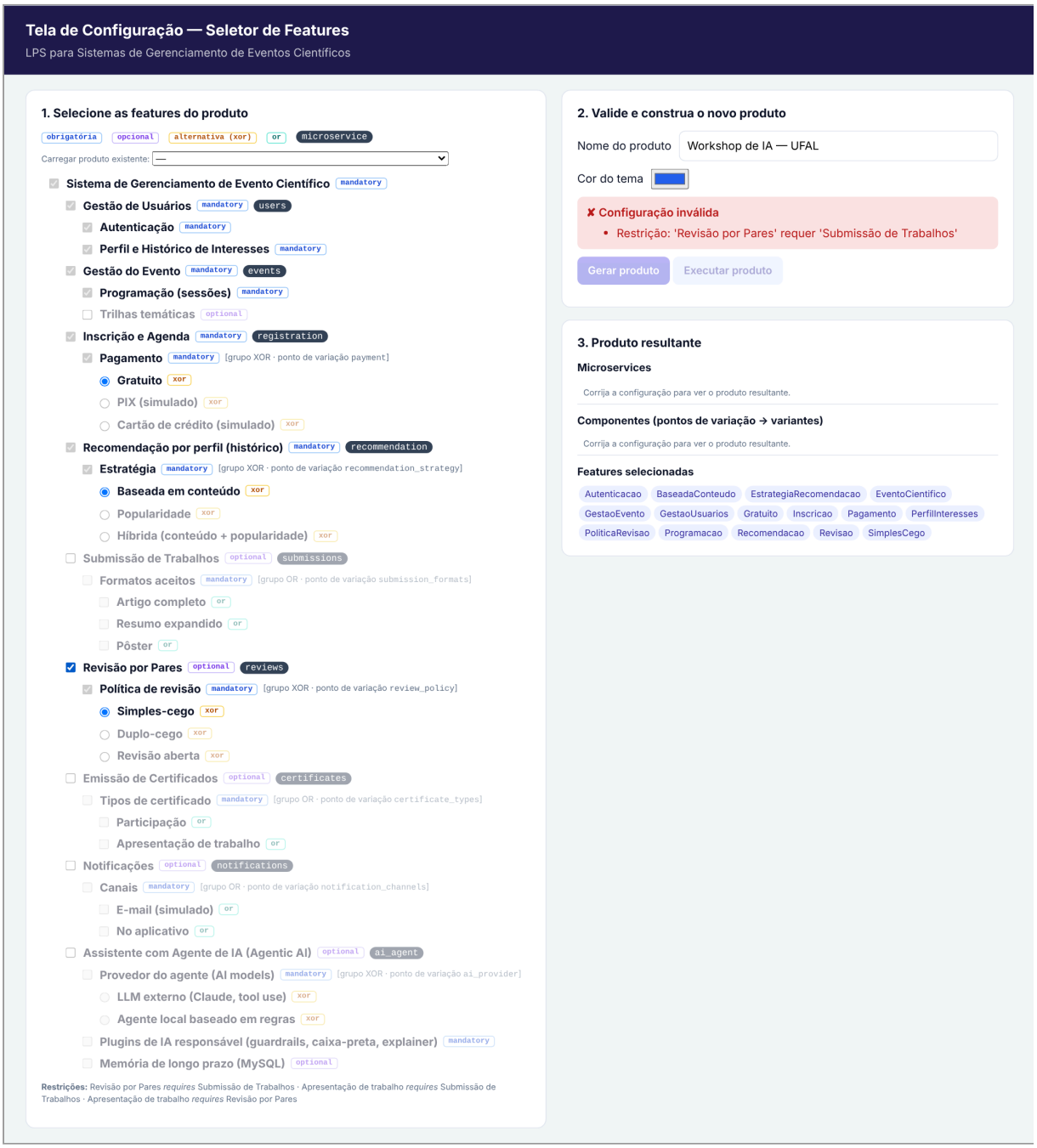
Figura 7 – Tela de Configuração com o produto SBES 2026 carregado



Fonte: elaborado pelas autoras (2026).

A validação impede produtos inválidos. Ao marcar Revisão por Pares sem Submissão, a restrição *requires* é violada e o botão “Gerar produto” fica desabilitado:

Figura 8 – Configuração inválida detectada pela Tela de Configuração



Fonte: elaborado pelas autoras (2026).

Construção de um novo produto (Semana de Computação UFAL 2026). Foram selecionados apenas Gratuito, Popularidade e Certificados de Participação. Pela linha de comando, o mesmo produto é construído assim:

```
$ python -m spl.cli derive "Semana de Computação UFAL 2026" --id semana-ic-ufal \
      Gratuito Popularidade Certificados CertParticipacao
Produto 'Semana de Computação UFAL 2026' derivado:
  products/semana-ic-ufal.json
  products/semana-ic-ufal.compose.yml
Microservices: users, events, registration, recommendation, certificates
```

```
$ python -m spl.cli run semana-ic-ufal
```

Código 3 – terminal — produto Semana UFAL em execução

```
Produto: Semana de Computação UFAL 2026  (tenant: semana-ic-ufal)
MICROSERVICE    PORTA   STATUS
registry        8761    UP
users           8001    UP
events          8002    UP
registration    8003    UP
recommendation  8004    UP
certificates    8007    UP
gateway         8080    UP

Banco de dados: 127.0.0.1:3306/{db}
Interface:  http://127.0.0.1:8080/
Registry:   http://127.0.0.1:8761/
Portas:     http://127.0.0.1:8080/ports
Swagger:    http://127.0.0.1:<porta>/docs

Ctrl+C para encerrar.
```

Fonte: elaborado pelas autoras (2026).

Arquivo de configuração gerado (parametrização do produto/tenant):

Código 4 – Arquivo `products/semana-ic-ufal.json`

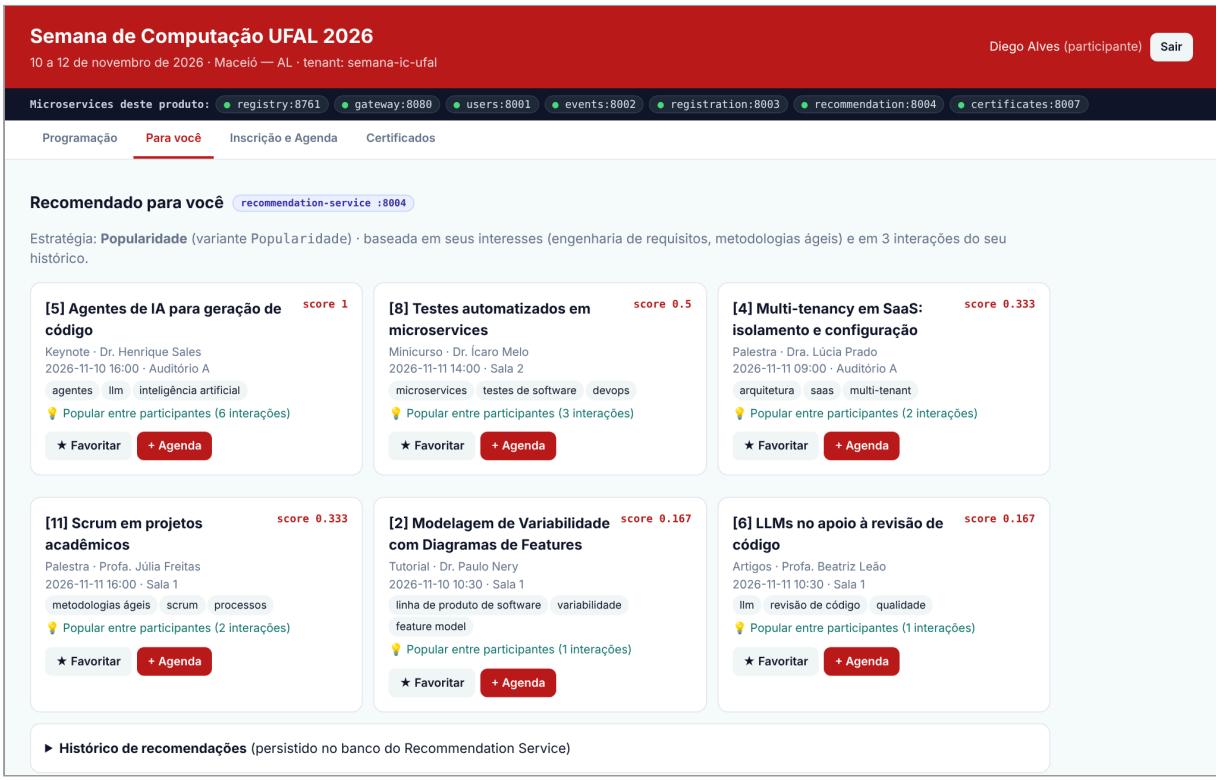
```
{
  "id": "semana-ic-ufal",
  "name": "Semana de Computação UFAL 2026",
  "theme": "#b91c1c",
  "features": [
    "Autenticacao",
    "CertParticipacao",
    "Certificados",
    "EstrategiaRecomendacao",
    "EventoCientifico",
    "GestaoEvento",
    "GestaoUsuarios",
    "Gratuito",
    "Inscricao",
    "Pagamento",
    "PerfilInteresses",
    "Popularidade",
    "Programacao",
    "Recomendacao",
    "TipoCertificado"
  ],
  "services": {
    "users": {
      "port": 8001,
      "url": "http://127.0.0.1:8001",
      "title": "Users Service",
      "feature": "GestaoUsuarios"
    },
    "events": {
      "port": 8002,
      "url": "http://127.0.0.1:8002",
      "title": "Events Service",
      "feature": "GestaoEvento"
    },
    "registration": {
      "port": 8003,
      "url": "http://127.0.0.1:8003",
```

```
      "title": "Registration Service",
      "feature": "Inscricao"
    },
    "recommendation": {
      "port": 8004,
      "url": "http://127.0.0.1:8004",
      "title": "Recommendation Service",
      "feature": "Recomendacao"
    },
    "certificates": {
      "port": 8007,
      "url": "http://127.0.0.1:8007",
      "title": "Certificates Service",
      "feature": "Certificados"
    }
  },
  "gateway": {
    "port": 8080,
    "url": "http://127.0.0.1:8080"
  },
  "registry": {
    "port": 8761,
    "url": "http://127.0.0.1:8761"
  },
  "components": {
    "payment": "Gratuito",
    "recommendation_strategy": "Popularidade",
    "certificate_types": [
      "CertParticipacao"
    ]
  }
}
```

Fonte: código do projeto (2026).

No produto em execução aparecem apenas as abas das features selecionadas, e a recomendação usa a variante Popularidade:

Figura 9 – Novo produto (Semana UFAL): menos abas e estratégia Popularidade



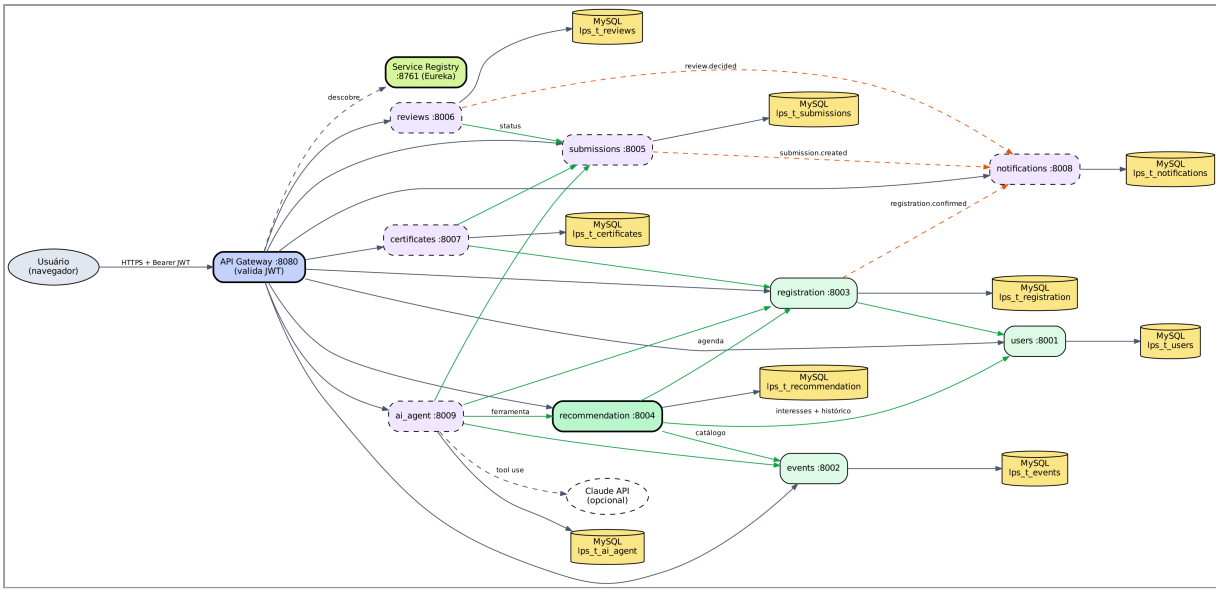
Fonte: elaborado pelas autoras (2026).

6 ARQUITETURA DO SISTEMA BASEADA EM MICROSERVICES

6.1 Arquitetura do Sistema usando Microservices (com recomendação pelo perfil do usuário)

O usuário acessa apenas o **API Gateway** (:8080). O gateway valida o token JWT, descobre o endereço do microservice no **Service Registry** (:8761) e encaminha a requisição com os cabeçalhos `X-Tenant`, `X-User-Id` e `X-User-Role`. Cada microservice tem **o seu próprio banco MySQL** (`lps_<tenant>_<serviço>`; na figura, `t` = tenant). Na figura: verde — serviços obrigatórios; lilás tracejado — serviços opcionais; cilindros — bancos MySQL; setas verdes — chamadas REST entre serviços; setas laranja — eventos de domínio (coreografia).

Figura 10 – Arquitetura do sistema baseada em Microservices



Fonte: elaborado pelas autoras (2026).

Quadro 11 – Microservices, portas, features e bancos de dados

Microservice	Porta	Feature	Banco MySQL	Responsabilidade
Service Registry	8761	núcleo	— (em memória, como o Eureka)	registro, heartbeat e descoberta
API Gateway	8080	núcleo	—	entrada única, JWT, roteamento, circuit breaker
Users	8001	Gestão de Usuários	`lps_<t>_users`	contas, login (JWT), interesses, histórico
Events	8002	Gestão do Evento	`lps_<t>_events`	trilhas, tags, sessões, programação
Registration	8003	Inscrição	`lps_<t>_registration`	inscrição, pagamento, agenda

Microservice	Porta	Feature	Banco MySQL	Responsabilidade
Recommendation	8004	Recomendação	`lps_<t>_recommendation`	recomendação por perfil + histórico
Submissions	8005	Submissão (opcional)	`lps_<t>_submissions`	trabalhos, coautores, palavras-chave
Reviews	8006	Revisão (opcional)	`lps_<t>_reviews`	pareceres, política de revisão
Certificates	8007	Certificados (opcional)	`lps_<t>_certificates`	emissão e verificação
Notifications	8008	Notificações (opcional)	`lps_<t>_notifications`	eventos recebidos, notificações, e-mails
AI Agent	8009	Assistente IA (opcional)	`lps_<t>_ai_agent`	agente, memória, caixa-preta

Fonte: elaborado pelas autoras (2026).

Microservice de recomendação (Recommendation Service, :8004). É obrigatório em todos os produtos, como exige o enunciado. Ele monta o **perfil do usuário** consultando três serviços:

- **Users:** interesses declarados e **histórico de interações** (`view`, `favorite`, `agenda`);
- **Registration:** sessões que já estão na agenda;
- **Events:** catálogo de sessões.

A estratégia é um componente variável:

- **Baseada em conteúdo:** monta um vetor de termos do usuário. Os interesses têm peso 4 e as sessões do histórico entram com peso por ação (ver = 1, favoritar = 2, agenda = 3). A similaridade de cosseno é calculada contra os termos de cada sessão (título, tags com peso 3, trilha).
- **Popularidade:** soma ponderada das interações de todos os usuários do tenant.
- **Híbrida:** combina as duas, com 0,7 para conteúdo e 0,3 para popularidade.

Sessões que já estão na agenda são excluídas, e cada recomendação vem com uma justificativa (ex.: “Relacionada ao seu perfil: arquitetura, saas”). O histórico é alimentado automaticamente: adicionar à agenda, favoritar e visualizar geram registros no Users Service. Cada lista gerada é **gravada na tabela `recomendacao`**, o que permite consultar o histórico de recomendações do usuário (`GET /api/recommendation/recommendations/{uid}/history`).

Código 5 – Arquivo `services/recommendation/service.py`

```
"""Service — recomendação baseada no perfil do usuário (interesses + histórico).

Monta o perfil consultando Users (interesses, histórico), Registration (agenda)
```

```
e Events (catálogo); a estratégia é um componente variável (Strategy) e cada
recomendação gerada é persistida no banco do serviço.
"""
from __future__ import annotations

from common.client import call
from common.config import component
from common.database import session_scope
from components.recommendation import rec_vp

from .model import Base
from .repository import RecommendationRepository

STRATEGY = rec_vp.create(component("recommendation_strategy", "BaseadaConteudo"))


class RecommendationService:
    def __init__(self, tenant: str):
        self.tenant = tenant

    def _tx(self):
        return session_scope("recommendation", self.tenant, Base)

    def profile(self, uid: int) -> dict | None:
        user = call("users", "GET", f"/users/{uid}", self.tenant)
        if not user:
            return None
        return {
            "user": user,
            "interests": user.get("interests", []),
            "history": call("users", "GET", f"/users/{uid}/history", self.tenant, default=
[]),
            "agenda": call("registration", "GET", f"/agenda/{uid}/ids", self.tenant,
default=[]),
        }

    def recommend(self, uid: int, k: int = 5) -> dict | None:
        prof = self.profile(uid)
        if prof is None:
            return None
        sessions = call("events", "GET", "/sessions", self.tenant, default=[])
        global_history = call("users", "GET", "/history", self.tenant, default=[])
        items = STRATEGY.recommend(prof, sessions, global_history, k)
        with self._tx() as s:
            RecommendationRepository(s).save_batch(uid, STRATEGY.feature_id, items)
        return {"user_id": uid, "strategy": STRATEGY.label, "variant": STRATEGY.feature_id,
                "based_on": {"interests": prof["interests"], "history_size":
len(prof["history"])}, "items": items}

    def history(self, uid: int) -> list[dict]:
        with self._tx() as s:
            return [{"id": r.id, "session_id": r.sessao_id, "session_title":
r.titulo_sessao, "score": round(r.score, 3),
                     "strategy": r.estrategia, "reason": r.motivo,
                     "generated_at": r.gerado_em.isoformat(sep=" ", timespec="seconds") if
r.gerado_em else ""}
                    for r in RecommendationRepository(s).history(uid)]
```

Fonte: código do projeto (2026).

Código 6 – Arquivo `components/recommendation.py`

```
"""Ponto de variação **Estratégia de Recomendação** (xor):
BaseadaConteudo | Popularidade | Hibrida.

Entrada comum a todas as estratégias:
* ``profile``  -> interesses declarados + histórico do usuário (sessões vistas,
  favoritadas, adicionadas à agenda);
* ``sessions`` -> catálogo da programação;
```

```
* ``global_history`` -> interações de todos os usuários (para popularidade).
"""
from __future__ import annotations

import math
import re
import unicodedata
from abc import ABC, abstractmethod
from collections import Counter

from .base import VariationPoint

ACTION_WEIGHT = {"view": 1.0, "favorite": 2.0, "agenda": 3.0}
STOP = {"de", "da", "do", "das", "dos", "e", "em", "para", "com", "a", "o", "as", "os",
"um", "uma", "no", "na", "por", "the", "of", "and", "in"}


def tokens(text: str) -> list[str]:
    text = unicodedata.normalize("NFKD", text.lower())
    text = "".join(c for c in text if not unicodedata.combining(c))
    return [t for t in re.findall(r"[a-z0-9]+", text) if len(t) > 2 and t not in STOP]


def session_terms(s: dict) -> Counter:
    c = Counter(tokens(s.get("title", "")))
    for tag in s.get("tags", []):
        c.update({t: 3 for t in tokens(tag)})          # tags pesam mais
    c.update({t: 1 for t in tokens(s.get("track", "") or "")})
    return c


def cosine(a: Counter, b: Counter) -> float:
    num = sum(a[k] * b[k] for k in set(a) & set(b))
    den = math.sqrt(sum(v * v for v in a.values())) * math.sqrt(sum(v * v for v in
b.values()))
    return num / den if den else 0.0


class RecommendationStrategy(ABC):
    label = "?"

    @abstractmethod
    def score(self, profile: dict, sessions: list[dict], global_history: list[dict]) ->
dict[int, tuple[float, str]]:
        """Retorna {session_id: (score, justificativa)}."""

    def recommend(self, profile, sessions, global_history, k=5) -> list[dict]:
        seen = {h["session_id"] for h in profile.get("history", []) if h["action"] ==
"agenda"}
        seen |= set(profile.get("agenda", []))
        scores = self.score(profile, sessions, global_history)
        ranked = sorted((s for s in sessions if s["id"] not in seen),
                        key=lambda s: scores.get(s["id"], (0, ""))[0], reverse=True)
        out = []
        for s in ranked[:k]:
            sc, why = scores.get(s["id"], (0.0, ""))
            out.append({**s, "score": round(sc, 3), "reason": why})
        return out


rec_vp: VariationPoint[RecommendationStrategy] = VariationPoint("recommendation_strategy",
RecommendationStrategy)


@rec_vp.variant("BaseadaConteudo")
class ContentBased(RecommendationStrategy):
    label = "Baseada em conteúdo"

    def user_vector(self, profile, sessions) -> Counter:
        by_id = {s["id"]: s for s in sessions}
```

```
        vec = Counter()
        for interest in profile.get("interests", []):
            vec.update({t: 4 for t in tokens(interest)})
        for h in profile.get("history", []):
            s = by_id.get(h["session_id"])
            if s:
                w = ACTION_WEIGHT.get(h["action"], 1.0)
                for t, v in session_terms(s).items():
                    vec[t] += v * w
        return vec

    def score(self, profile, sessions, global_history):
        uv = self.user_vector(profile, sessions)
        out = {}
        for s in sessions:
            sv = session_terms(s)
            common = sorted(set(uv) & set(sv), key=lambda t: -uv[t] * sv[t])[:3]
            why = ("Relacionada ao seu perfil: " + ", ".join(common)) if common else
"Explorar novos temas"
            out[s["id"]] = (cosine(uv, sv), why)
        return out


@rec_vp.variant("Popularidade")
class Popularity(RecommendationStrategy):
    label = "Popularidade"

    def score(self, profile, sessions, global_history):
        pop = Counter()
        for h in global_history:
            pop[h["session_id"]] += ACTION_WEIGHT.get(h["action"], 1.0)
        top = max(pop.values(), default=1)
        return {s["id"]: (pop[s["id"]] / top, f"Popular entre participantes
({int(pop[s['id']])} interações)")
                for s in sessions}


@rec_vp.variant("Hibrida")
class Hybrid(RecommendationStrategy):
    label = "Híbrida"
    alpha = 0.7  # peso do conteúdo

    def score(self, profile, sessions, global_history):
        c = ContentBased().score(profile, sessions, global_history)
        p = Popularity().score(profile, sessions, global_history)
        return {sid: (self.alpha * c[sid][0] + (1 - self.alpha) * p[sid][0],
                      c[sid][1] + f" · {p[sid][1].lower()}") for sid in c}
```

Fonte: código do projeto (2026).

6.2 Especificação (código documentado) da API Gateway

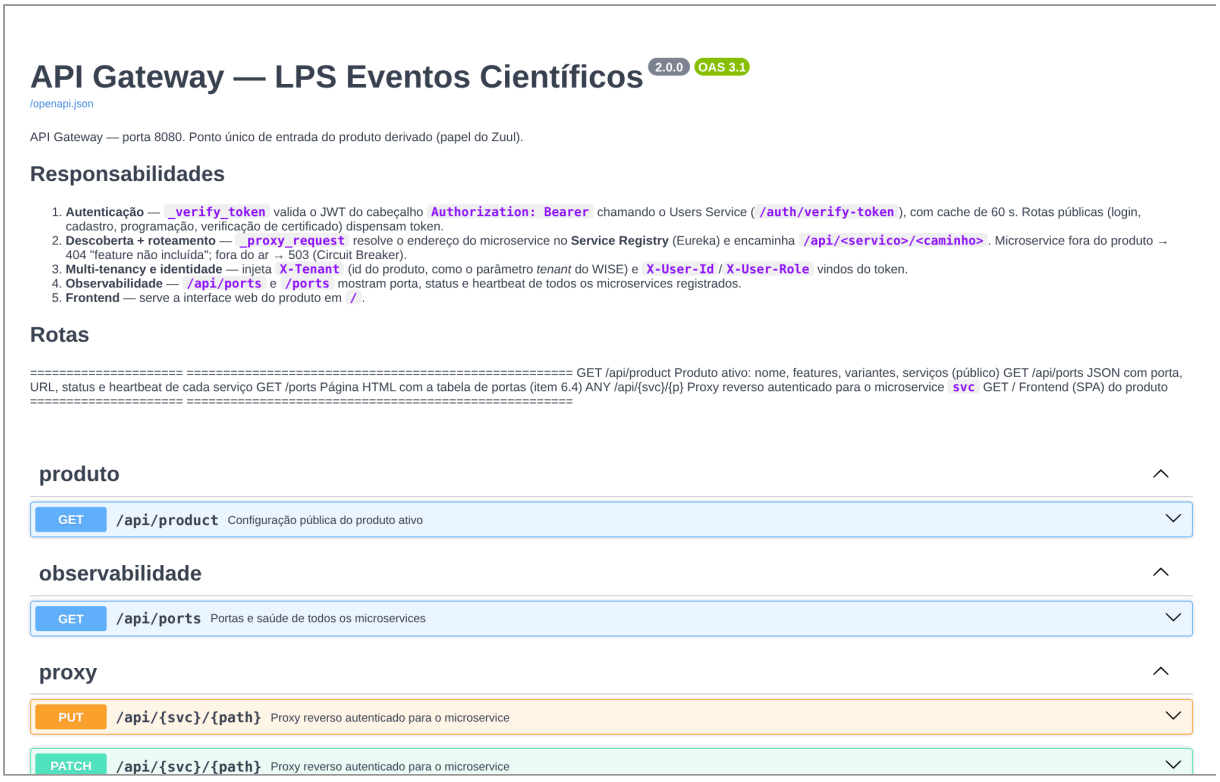
O API Gateway faz o papel do **Zuul** na figura de referência. Como no relatório-modelo da disciplina, dois métodos são centrais: **`_verify_token`**, que valida o JWT recebido no cabeçalho `Authorization: Bearer` chamando o Users Service (com cache de 60 s), e **`_proxy_request`**, que usa o resultado dessa verificação para liberar ou bloquear a requisição, resolve o endereço do microservice no Service Registry e encaminha a chamada. Rotas públicas (login, cadastro, programação, verificação de certificado, políticas do evento) dispensam token; as demais retornam **401** sem token. Microservice fora do produto retorna **404 “feature não selecionada”**; microservice fora do ar retorna **503** (Circuit Breaker).

Quadro 12 – Rotas do API Gateway

Método	Rota	Descrição
GET	/api/product	Configuração pública do produto ativo (features, variantes, serviços)
GET	/api/ports	Porta, URL, status e heartbeat de todos os microservices (JSON)
GET	/ports	Página HTML com a tabela de portas (item 6.4)
GET POST PUT PATCH DELETE	/api/{svc}/{path}	Proxy reverso autenticado para o microservice svc
GET	/	Frontend (SPA) do produto
GET	/docs	Especificação OpenAPI (Swagger)

Fonte: elaborado pelas autoras (2026).

Figura 11 – Swagger do API Gateway (http://127.0.0.1:8080/docs)



Fonte: elaborado pelas autoras (2026).

Código 7 – Arquivo gateway/main.py

```
"""API Gateway — porta 8080. Ponto único de entrada do produto derivado (papel do Zuul).

Responsabilidades
-----------------
1. **Autenticação** — ``_verify_token`` valida o JWT do cabeçalho
   ``Authorization: Bearer`` chamando o Users Service (``/auth/verify-token``),
   com cache de 60 s. Rotas públicas (login, cadastro, programação, verificação
   de certificado) dispensam token.
2. **Descoberta + roteamento** — ``_proxy_request`` resolve o endereço do
   microservice no **Service Registry** (Eureka) e encaminha
   ``/api/<servico>/<caminho>``. Microservice fora do produto → 404 "feature não
   incluída"; fora do ar → 503 (Circuit Breaker).
```

```
3. **Multi-tenancy e identidade** — injeta ``X-Tenant`` (id do produto, como o
   parâmetro *tenant* do WISE) e ``X-User-Id``/``X-User-Role`` vindos do token.
4. **Observabilidade** — ``/api/ports`` e ``/ports`` mostram porta, status e
   heartbeat de todos os microservices registrados.
5. **Frontend** — serve a interface web do produto em ``/``.

Rotas
-----
====================  ====================================================
GET  /api/product      Produto ativo: nome, features, variantes, serviços (público)
GET  /api/ports        JSON com porta, URL, status e heartbeat de cada serviço
GET  /ports            Página HTML com a tabela de portas (item 6.4)
ANY  /api/{svc}/{p}    Proxy reverso autenticado para o microservice ``svc``
GET  /                 Frontend (SPA) do produto
====================  ====================================================
"""
from __future__ import annotations

import re
import time

import httpx
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import HTMLResponse, Response
from fastapi.staticfiles import StaticFiles

from common.circuit_breaker import CircuitBreaker
from common.config import ROOT, product, registry_url, service_url

app = FastAPI(title="API Gateway — LPS Eventos Científicos", description=__doc__,
version="2.0.0")
_client = httpx.AsyncClient(timeout=60)
_breakers: dict[str, CircuitBreaker] = {}
_token_cache: dict[str, tuple[float, dict]] = {}
HOP_HEADERS = {"host", "content-length", "connection", "transfer-encoding", "content-
encoding",
               "x-user-id", "x-user-role", "x-tenant"}

# (método, regex do caminho "<svc>/<path>") que não exigem token
PUBLIC_ROUTES = [
    ("POST", r"^users/auth/(login|register|verify-token)$"),
    ("GET", r"^users/auth/demo-accounts$"),
    ("GET", r"^events/(event|sessions(/\d+)?|tracks)$"),
    ("GET", r"^certificates/(verify|view)/\w+$"),
    ("GET", r"^registration/payment-method$"),
    ("GET", r"^submissions/formats$"),
    ("GET", r"^reviews/policy$"),
    ("GET", r"^ai_agent/architecture$"),
]


def _is_public(method: str, svc: str, path: str) -> bool:
    target = f"{svc}/{path}".rstrip("/")
    return any(method == m and re.match(rx, target) for m, rx in PUBLIC_ROUTES)


async def _verify_token(request: Request) -> dict | None:
    """Valida o JWT no Users Service; devolve as claims ou ``None``."""
    auth = request.headers.get("Authorization", "")
    token = auth.split(" ")[-1] if auth.lower().startswith("bearer ") else None
    if not token:
        return None
    hit = _token_cache.get(token)
    if hit and time.monotonic() - hit[0] < 60:
        return hit[1]
    base = service_url("users")
    try:
        r = await _client.post(f"{base}/auth/verify-token", json={"token": token},
                               headers={"X-Tenant": product()["id"]}, timeout=3)
    except httpx.HTTPError:
```

```
        return None
    if r.status_code != 200:
        return None
    claims = r.json()
    _token_cache[token] = (time.monotonic(), claims)
    return claims


@app.get("/api/product", tags=["produto"], summary="Configuração pública do produto ativo")
def product_info():
    p = product()
    return {k: p[k] for k in ("id", "name", "theme", "features", "components", "services",
"gateway")}


async def _registered() -> dict[str, dict]:
    try:
        r = await _client.get(f"{registry_url()}/registry/services", timeout=2)
        return {i["name"]: i for i in r.json()}
    except (httpx.HTTPError, ValueError):
        return {}


@app.get("/api/ports", tags=["observabilidade"], summary="Portas e saúde de todos os
microservices")
async def ports():
    p = product()
    reg = await _registered()
    items = [
        {"service": "registry", "title": "Service Registry (Eureka)", "feature": "núcleo",
"port": 8761,
         "url": registry_url(), "status": "UP" if reg or await _ping(registry_url() +
"/health") else "DOWN",
         "heartbeat": "—"},
        {"service": "gateway", "title": "API Gateway", "feature": "núcleo", "port":
p["gateway"]["port"],
         "url": p["gateway"]["url"], "status": "UP", "heartbeat": "—"},
    ]
    for name, info in p["services"].items():
        r = reg.get(name)
        items.append({"service": name, "title": info["title"], "feature": info["feature"],
"port": info["port"],
                      "url": r["url"] if r else info["url"], "status": r["status"] if r
else "NÃO REGISTRADO",
                      "heartbeat": f"{r['seconds_since_heartbeat']} s" if r else "—"})
    return {"product": p["name"], "database": "MySQL (um banco por serviço e por tenant)",
"services": items}


async def _ping(url: str) -> bool:
    try:
        return (await _client.get(url, timeout=1)).status_code == 200
    except httpx.HTTPError:
        return False


@app.get("/ports", response_class=HTMLResponse, include_in_schema=False)
async def ports_page():
    data = await ports()
    trs = "".join(
        f"<tr><td><b>{s['title']}</b><br><code>{s['service']}</code></td><td>{s['feature']}
</td>"
        f"<td class=port>{s['port']}</td><td><code>{s['url']}</code></td>"
        f"<td><span class='st {'up' if s['status'] == 'UP' else 'down'}'>{s['status']}
</span></td>"
        f"<td>{s['heartbeat']}</td></tr>" for s in data["services"])
    return f"""<!doctype html><html lang=pt-br><meta charset=utf-8><title>Portas dos
microservices</title>
<style>body{{font-family:system-ui,sans-
serif;margin:32px;background:#f8fafc;color:#0f172a}}
```

```
table{{border-collapse:collapse;width:100%;background:#fff;box-shadow:0 1px 3px #0002}}
th,td{{padding:10px 14px;border-bottom:1px solid #e2e8f0;text-
align:left}}th{{background:#0f172a;color:#fff}}
.port{{font:700 18px ui-monospace,monospace}}.st{{padding:3px 10px;border-radius:99px;font-
weight:600}}
.up{{background:#dcfce7;color:#166534}}.down{{background:#fee2e2;color:#991b1b}}</style>
<h1>Microservices em execução</h1><p>Produto: <b>{data['product']}</b> ·
{len(data['services'])} processos ·
banco: {data['database']} · endereços resolvidos pelo Service Registry</p>
<table><tr><th>Microservice</th><th>Feature</th><th>Porta</th><th>URL</th><th>Status</th>
<th>Último heartbeat</th></tr>{trs}</table>
<p><a href="/">← voltar ao produto</a> · <a href="http://127.0.0.1:8761/">painel do
registry</a> · <a href="/docs">Swagger do gateway</a></p></html>"""


@app.api_route("/api/{svc}/{path:path}", methods=["GET", "POST", "PUT", "PATCH", "DELETE"],
tags=["proxy"],
               summary="Proxy reverso autenticado para o microservice")
async def proxy(svc: str, path: str, request: Request):
    return await _proxy_request(svc, path, request)


async def _proxy_request(svc: str, path: str, request: Request) -> Response:
    """Autentica (se a rota não for pública), descobre o serviço e encaminha a
requisição."""
    if svc not in product()["services"]:
        raise HTTPException(404, f"Microservice '{svc}' não faz parte do produto (feature
não selecionada)")
    headers = {k: v for k, v in request.headers.items() if k.lower() not in HOP_HEADERS}
    headers["X-Tenant"] = product()["id"]
    headers["X-Forwarded-From-Gateway"] = "true"
    claims = await _verify_token(request)
    if claims:
        headers["X-User-Id"], headers["X-User-Role"] = str(claims["user_id"]),
claims["role"]
    elif not _is_public(request.method, svc, path):
        raise HTTPException(401, "Autenticação necessária (Authorization: Bearer <token>)")
    elif svc != "users" or not path.startswith("auth/"):
        headers["X-User-Id"], headers["X-User-Role"] = "0", "anonimo"   # público:
identidade anônima

    base = service_url(svc)
    breaker = _breakers.setdefault(svc, CircuitBreaker(svc))
    if breaker.state == "OPEN":
        raise HTTPException(503, f"Circuito para '{svc}' aberto — serviço indisponível")
    try:
        r = await _client.request(request.method, f"{base}/{path}",
params=request.query_params,
                                  content=await request.body(), headers=headers)
        if r.status_code >= 500:
            breaker.failures += 1
            if breaker.failures >= breaker.threshold:
                breaker.opened_at = time.monotonic()
        else:
            breaker.failures, breaker.opened_at = 0, None
    except httpx.HTTPError:
        breaker.failures += 1
        if breaker.failures >= breaker.threshold:
            breaker.opened_at = time.monotonic()
        raise HTTPException(503, f"Microservice '{svc}' indisponível")
    out_headers = {k: v for k, v in r.headers.items() if k.lower() not in HOP_HEADERS}
    out_headers["X-Served-By"] = f"{svc}:{product()['services'][svc]['port']}"
    return Response(r.content, status_code=r.status_code, headers=out_headers)


app.mount("/", StaticFiles(directory=ROOT / "frontend", html=True), name="frontend")
```

Fonte: código do projeto (2026).

A segurança usada pelo gateway e pelo Users Service está em `common/security.py` (senha com PBKDF2-SHA256 e token JWT HS256 com validade de 8 h):

Código 8 – Arquivo `common/security.py`

```
"""Segurança: hash de senha (PBKDF2-SHA256) e tokens JWT (HS256).

O Users Service emite o token no login; o API Gateway o valida antes de
encaminhar requisições protegidas e repassa a identidade aos microservices
nos cabeçalhos ``X-User-Id`` e ``X-User-Role``.
"""
from __future__ import annotations

import base64
import hashlib
import hmac
import os
import time

import jwt

SECRET = os.environ.get("JWT_SECRET", "lps-eventos-dev-secret-troque-em-producao")
ALGORITHM = "HS256"
TOKEN_TTL = 8 * 3600
ITERATIONS = 120_000


def hash_password(password: str, salt: bytes | None = None) -> str:
    salt = salt or os.urandom(16)
    digest = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, ITERATIONS)
    return
f"pbkdf2_sha256${ITERATIONS}${base64.b64encode(salt).decode()}${base64.b64encode(digest).de
code()}"


def verify_password(password: str, stored: str) -> bool:
    try:
        _, iters, salt_b64, digest_b64 = stored.split("$")
        digest = hashlib.pbkdf2_hmac("sha256", password.encode(),
base64.b64decode(salt_b64), int(iters))
        return hmac.compare_digest(digest, base64.b64decode(digest_b64))
    except ValueError:
        return False


def create_token(user_id: int, role: str, name: str, tenant: str) -> str:
    now = int(time.time())
    payload = {"sub": str(user_id), "role": role, "name": name, "tenant": tenant, "iat":
now, "exp": now + TOKEN_TTL}
    return jwt.encode(payload, SECRET, algorithm=ALGORITHM)


def decode_token(token: str) -> dict:
    """Retorna as claims ou lança ``jwt.InvalidTokenError``."""
    return jwt.decode(token, SECRET, algorithms=[ALGORITHM])
```

Fonte: código do projeto (2026).

6.3 Especificação (código documentado) de cada Microservice

Todos os microservices têm a mesma organização interna, igual à da figura *Microservices e Componentes* da disciplina:

Quadro 13 – Componentes internos de cada microservice

Componente (arquivo)	**Papel**
`controller.py`	API REST (FastAPI `APIRouter`): rotas, validação de entrada, autorização (`Caller`)
`dto.py`	Objetos de transferência (Pydantic) usados na entrada e na saída das rotas
`service.py`	Regras de negócio; usa os componentes variáveis (Strategy) e chama outros serviços
`repository.py`	Consultas e gravações no banco (SQLAlchemy `Session`)
`model.py`	Entidades ORM = tabelas MySQL do banco do serviço
`main.py`	Cria o app com `create_service` (registro no Registry, `/health`, `/info`)

Fonte: elaborado pelas autoras (2026).

Todos expõem `GET /health` e `GET /info` (variantes ativas e estado dos circuit breakers) e uma página Swagger em `http://127.0.0.1:<porta>/docs`. A fábrica comum de microservices e o acesso ao banco ficam no núcleo da LPS:

Código 9 – Arquivo `common/service.py`

```
"""Fábrica de microservices — ativo reutilizável do núcleo da LPS.

Todo microservice criado por ``create_service`` ganha, sem escrever código:

* ``GET /health`` e ``GET /info`` (observabilidade, variantes ativas);
* registro automático no **Service Registry** + heartbeat (descoberta de serviço);
* as dependências ``tenant_header`` (multi-tenancy pelo cabeçalho ``X-Tenant``,
  como no WISE) e ``current_user`` (identidade repassada pelo API Gateway).

Cada serviço segue a mesma organização interna (figura de referência da disciplina):
``controller.py`` (API REST) → ``service.py`` (regras de negócio) →
``repository.py`` (acesso a dados) → ``model.py`` (entidades ORM), com ``dto.py``
(objetos de transferência validados pelo Pydantic).
"""
from __future__ import annotations

import logging
import os
from contextlib import asynccontextmanager
from dataclasses import dataclass

from fastapi import APIRouter, FastAPI, Header, HTTPException

from .client import breaker_states
from .config import component, product, tenant_id
from .discovery import register

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(name)s] %(message)s")


def _public_url(name: str, port: int) -> str:
    if os.environ.get("SERVICE_HOST_MODE") == "docker":
        return f"http://{name.replace('_', '-')}:{port}"
    return f"http://127.0.0.1:{port}"


def create_service(name: str, title: str, description: str, router: APIRouter,
                   bindings: list[str] | None = None) -> FastAPI:
    svc = product()["services"].get(name, {})
```

```
    port = svc.get("port", 0)

    @asynccontextmanager
    async def lifespan(_app):
        register(name, _public_url(name, port), port, svc.get("feature"), title)
        yield

    app = FastAPI(title=title, description=description, version="2.0.0", lifespan=lifespan)

    @app.get("/health", tags=["observabilidade"], summary="Verifica se o microservice está
ativo")
    def health():
        return {"service": name, "status": "UP", "product": product()["id"]}

    @app.get("/info", tags=["observabilidade"], summary="Feature e variantes resolvidas
neste produto")
    def info():
        return {
            "service": name,
            "title": title,
            "product": product()["name"],
            "variants": {b: component(b) for b in (bindings or [])},
            "circuit_breakers": breaker_states(),
        }

    app.include_router(router)
    return app


def tenant_header(x_tenant: str | None = Header(default=None, alias="X-Tenant")) -> str:
    """Dependência FastAPI: tenant da requisição (padrão = produto ativo)."""
    return x_tenant or tenant_id()


@dataclass
class Caller:
    """Quem fez a requisição. ``internal`` = chamada direta entre microservices."""
    user_id: int | None
    role: str | None

    @property
    def internal(self) -> bool:
        return self.user_id is None

    def ensure_self_or(self, user_id: int, *roles: str) -> None:
        """Usuário só acessa os próprios dados, salvo papéis autorizados."""
        if self.internal or self.user_id == user_id or self.role in roles:
            return
        raise HTTPException(403, "Acesso negado a dados de outro usuário")

    def ensure_role(self, *roles: str) -> None:
        if self.internal or self.role in roles:
            return
        raise HTTPException(403, f"Requer papel: {', '.join(roles)}")


def current_user(x_user_id: str | None = Header(default=None, alias="X-User-Id"),
                 x_user_role: str | None = Header(default=None, alias="X-User-Role")) ->
Caller:
    return Caller(int(x_user_id) if x_user_id else None, x_user_role)
```

Fonte: código do projeto (2026).

Código 10 – Arquivo `common/database.py`

```
"""Acesso a banco de dados relacional — um banco por microservice e por tenant.

Padrão *Database per Service*: cada microservice tem o seu próprio banco
MySQL e nenhum serviço lê o banco de outro (a comunicação é sempre via API).
```

```
Multi-tenancy no estilo do WISE (Tizzei et al., 2017): um banco por tenant,
chamado ``lps_<tenant>_<servico>`` (ex.: ``lps_sbes_2026_users``).

O SGBD vem da variável ``DATABASE_URL`` (template com ``{db}``)::

    mysql+pymysql://lps:lps123@127.0.0.1:3306/{db}     # MySQL / MariaDB (padrão do
projeto)
    sqlite:///data/{db}.db                              # alternativa sem servidor (testes)

As tabelas são criadas pelo ORM (SQLAlchemy) na primeira conexão, e os dados
de demonstração são inseridos se o banco estiver vazio.
"""
from __future__ import annotations

import os
import re
import threading
from contextlib import contextmanager
from typing import Callable

from sqlalchemy import create_engine, text
from sqlalchemy.engine import Engine, make_url
from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker

from .config import ROOT

DEFAULT_URL = "mysql+pymysql://lps:lps123@127.0.0.1:3306/{db}"
_engines: dict[str, sessionmaker] = {}
_lock = threading.Lock()


def database_name(service: str, tenant: str) -> str:
    return re.sub(r"[^a-z0-9_]", "_", f"lps_{tenant}_{service}".lower())


def database_url(service: str, tenant: str) -> str:
    template = os.environ.get("DATABASE_URL", DEFAULT_URL)
    url = template.format(db=database_name(service, tenant))
    if url.startswith("sqlite:///") and not url.startswith("sqlite:////"):
        path = ROOT / url.removeprefix("sqlite:///")
        path.parent.mkdir(parents=True, exist_ok=True)
        url = f"sqlite:///{path}"
    return url


def _ensure_database(url: str) -> None:
    """No MySQL, cria o banco do serviço/tenant se ainda não existir."""
    u = make_url(url)
    if not u.drivername.startswith("mysql"):
        return
    server = create_engine(u.set(database=""), pool_pre_ping=True)
    with server.connect() as conn:
        conn.execute(text(f"CREATE DATABASE IF NOT EXISTS `{u.database}` "
                          "CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci"))
    server.dispose()


def session_factory(service: str, tenant: str, base: type[DeclarativeBase],
                    seed: Callable[[Session, str], None] | None = None) -> sessionmaker:
    key = f"{service}:{tenant}"
    with _lock:
        if key in _engines:
            return _engines[key]
        url = database_url(service, tenant)
        _ensure_database(url)
        engine: Engine = create_engine(url, pool_pre_ping=True, pool_recycle=1800)
        base.metadata.create_all(engine)
        factory = sessionmaker(bind=engine, expire_on_commit=False)
        if seed:
            with factory() as s:
```

```
                seed(s, tenant)
                s.commit()
        _engines[key] = factory
        return factory


@contextmanager
def session_scope(service: str, tenant: str, base, seed=None):
    """Unidade de trabalho: abre sessão, faz commit no fim ou rollback em erro."""
    s: Session = session_factory(service, tenant, base, seed)()
    try:
        yield s
        s.commit()
    except Exception:
        s.rollback()
        raise
    finally:
        s.close()
```

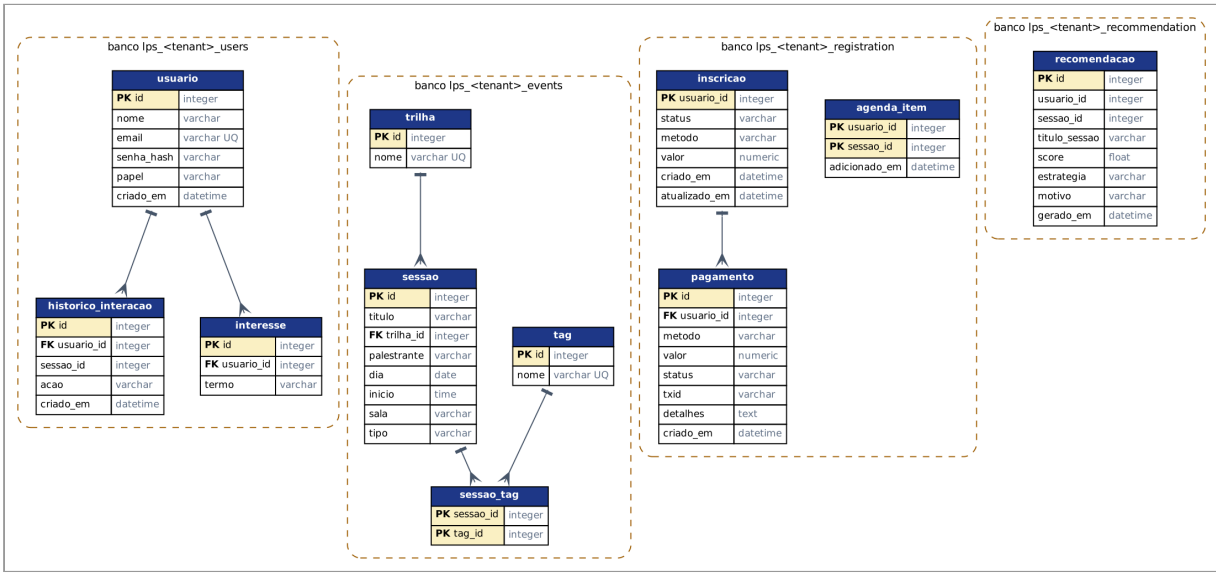
Fonte: código do projeto (2026).

6.3.1 Banco de dados: MySQL, Modelo ER e normalização

SGBD. MySQL 8 (no Docker Compose) ou MariaDB 10.11 (compatível, usado nos testes locais), acessado pelo driver PyMySQL através do ORM SQLAlchemy 2. Segue o padrão *Database per Service*: **cada microservice tem o seu banco** e nenhum serviço lê o banco de outro. Para multi-tenancy, como no WISE, há **um banco por tenant**: `lps_<tenant>_<serviço>`. Os bancos e tabelas são criados automaticamente na primeira conexão; o script completo está em `docs/schema_mysql.sql`.

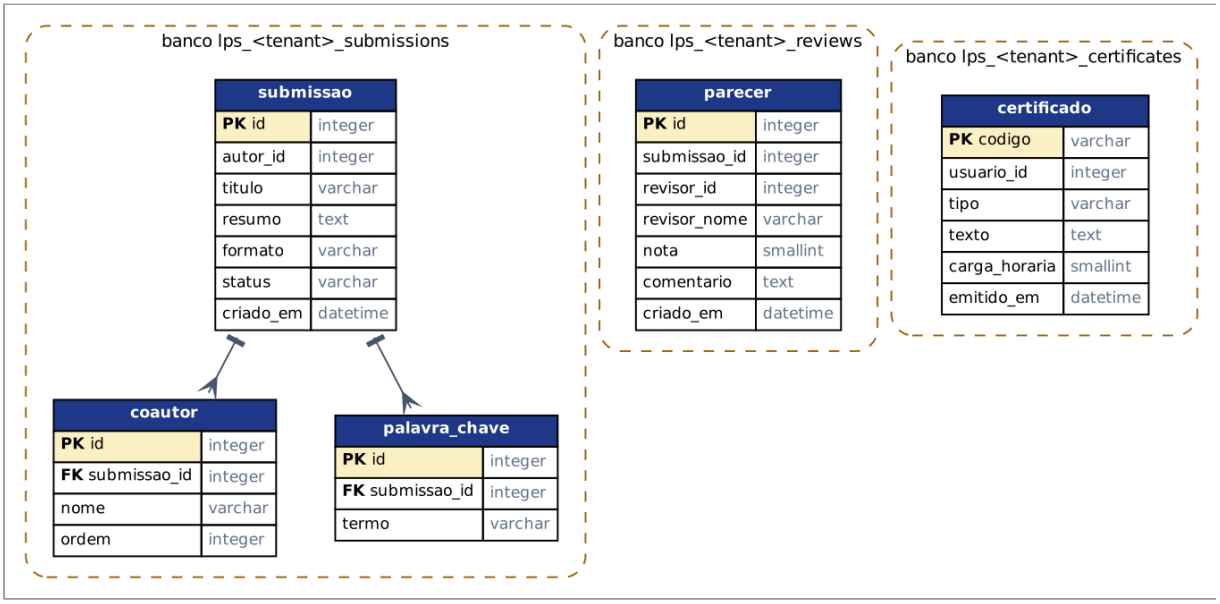
Modelo Entidade-Relacionamento. Cada quadro tracejado é o banco de um microservice. PK = chave primária, FK = chave estrangeira, UQ = valor único; as linhas ligam as tabelas relacionadas (1:N, “pé de galinha” no lado N).

Figura 12 – Modelo ER — bancos dos microservices obrigatórios (users, events, registration, recommendation)



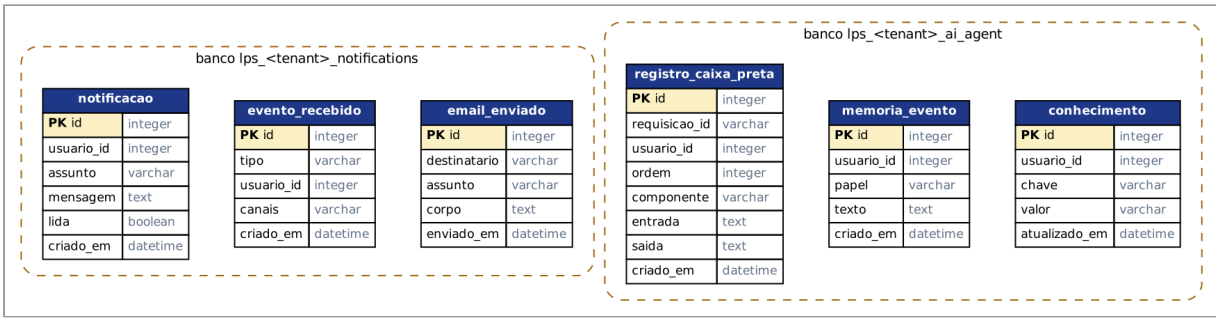
Fonte: elaborado pelas autoras (2026).

Figura 13 – Modelo ER — bancos de submissions, reviews e certificates (opcionais)



Fonte: elaborado pelas autoras (2026).

Figura 14 – Modelo ER — bancos de notifications e ai_agent (opcionais)



Fonte: elaborado pelas autoras (2026).

Normalização. As tabelas foram projetadas e verificadas até a **3ª Forma Normal**:

Quadro 14 – Normalização das tabelas (1FN, 2FN e 3FN)

Forma normal	Regra	Como foi aplicada (exemplos)
1FN	Atributos atômicos, sem grupos repetidos	Interesses do usuário não são uma lista em texto dentro de `usuario`: viram a tabela `interesse (usuario_id, termo)`. O mesmo vale para `coautor` e `palavra_chave` (em `submissao`) e para as tags das sessões (`tag` + `sessao_tag`).
2FN	1FN + nenhum atributo depende de parte de uma chave composta	`agenda_item` tem chave composta (`usuario_id`, `sessao_id`) e só o atributo `adicionado_em`, que depende da chave inteira. `sessao_tag (sessao_id, tag_id)` não tem atributos não-chave. Os dados da sessão (título, sala) não são copiados para a agenda.
3FN	2FN + nenhum atributo não-chave depende de outro atributo não-chave	O nome da trilha não fica repetido em cada `sessao`: `sessao.trilha_id → trilha(id, nome)`. Os pagamentos ficam em `pagamento` (1:N com `inscricao`), e não como colunas repetidas na inscrição. `notificacao` e `email_enviado` são tabelas separadas de `evento_recebido`.

Fonte: elaborado pelas autoras (2026).

Referências a dados de outro microservice (ex.: `historico_interacao.sessao_id`, `parecer.submissao_id`, `certificado.usuario_id`) são **ids lógicos**: não existe chave estrangeira física entre bancos diferentes, para manter os serviços independentes. A integridade é garantida pelas chamadas REST.

Evidências no MySQL (produto SBES 2026 em execução). Bancos criados, um por microservice:

Código 11 – mysql> SHOW DATABASES LIKE ‘lps_sbes%’;

```
+------------------------------+
| Database (lps\_sbes%)        |
+------------------------------+
| lps_sbes_2026_ai_agent       |
| lps_sbes_2026_certificates   |
| lps_sbes_2026_events         |
| lps_sbes_2026_notifications  |
| lps_sbes_2026_recommendation |
| lps_sbes_2026_registration   |
| lps_sbes_2026_reviews        |
| lps_sbes_2026_submissions    |
| lps_sbes_2026_users          |
+------------------------------+
```

Fonte: elaborado pelas autoras (2026).

Tabelas de cada banco:

Código 12 – mysql> SHOW TABLES (por banco)

```
== lps_sbes_2026_users
historico_interacao interesse usuario
```

```
== lps_sbes_2026_events
trilha tag sessao sessao_tag
== lps_sbes_2026_registration
agenda_item inscricao pagamento
== lps_sbes_2026_recommendation
recomendacao
== lps_sbes_2026_submissions
submissao palavra_chave coautor
== lps_sbes_2026_reviews
parecer
== lps_sbes_2026_certificates
certificado
== lps_sbes_2026_notifications
evento_recebido notificacao email_enviado
== lps_sbes_2026_ai_agent
memoria_evento conhecimento registro_caixa_preta
```

Fonte: elaborado pelas autoras (2026).

Consulta com junção no banco do Users Service (usuário 1:N interesses):

Código 13 – mysql> SELECT u.id, u.nome, u.papel, GROUP_CONCAT(i.termo) … FROM usuario u JOIN interesse i …

```
+----+--------------+--------------+-------------------------------------------------------
---+
| id | nome         | papel        | interesses
|
+----+--------------+--------------+-------------------------------------------------------
---+
|  1 | Ana Souza    | participante | linha de produto de software, microservices,
arquitetura |
|  2 | Bruno Lima   | autor        | inteligência artificial, agentes, llm
|
|  3 | Carla Mendes | revisor      | testes de software, qualidade, devops
|
+----+--------------+--------------+-------------------------------------------------------
---+
```

Fonte: elaborado pelas autoras (2026).

DDL gerada para o banco do Events Service (chaves, índices e FKs):

Código 14 – mysql> SHOW CREATE TABLE … (lps_sbes_2026_events)

```
CREATE TABLE `sessao` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `titulo` varchar(200) NOT NULL,
  `trilha_id` int(11) DEFAULT NULL,
  `palestrante` varchar(120) NOT NULL,
  `dia` date NOT NULL,
  `inicio` time NOT NULL,
  `sala` varchar(60) NOT NULL,
  `tipo` varchar(30) NOT NULL,
  PRIMARY KEY (`id`),
  KEY `ix_sessao_dia` (`dia`),
  KEY `ix_sessao_trilha_id` (`trilha_id`),
  CONSTRAINT `sessao_ibfk_1` FOREIGN KEY (`trilha_id`) REFERENCES `trilha` (`id`) ON DELETE
SET NULL
) ENGINE=InnoDB AUTO_INCREMENT=15 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
CREATE TABLE `sessao_tag` (
  `sessao_id` int(11) NOT NULL,
  `tag_id` int(11) NOT NULL,
  PRIMARY KEY (`sessao_id`,`tag_id`),
  KEY `tag_id` (`tag_id`),
  CONSTRAINT `sessao_tag_ibfk_1` FOREIGN KEY (`sessao_id`) REFERENCES `sessao` (`id`) ON
```

```
DELETE CASCADE,
  CONSTRAINT `sessao_tag_ibfk_2` FOREIGN KEY (`tag_id`) REFERENCES `tag` (`id`) ON DELETE
CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
CREATE TABLE `tag` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `nome` varchar(80) NOT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `nome` (`nome`)
) ENGINE=InnoDB AUTO_INCREMENT=31 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
CREATE TABLE `trilha` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `nome` varchar(80) NOT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `nome` (`nome`)
) ENGINE=InnoDB AUTO_INCREMENT=6 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Fonte: elaborado pelas autoras (2026).

6.3.2 Service Registry — porta 8761 (núcleo)

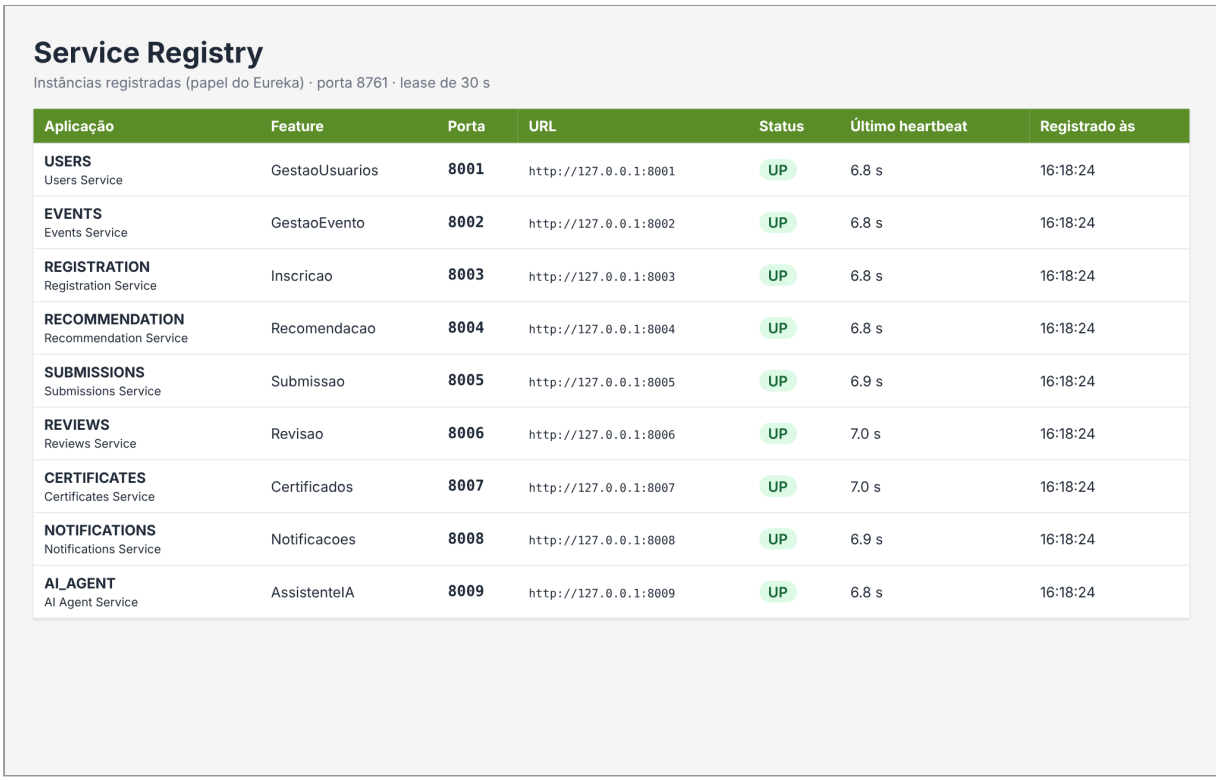
Faz o papel do **Eureka** na figura de referência. Cada microservice se registra ao iniciar (`create_service → discovery.register`) e envia heartbeat a cada 10 s; sem heartbeat por 30 s a instância fica DOWN. O gateway e os serviços resolvem endereços aqui (`discovery.lookup`, com cache de 10 s).

Quadro 15 – Rotas do Service Registry

Método	Rota	Descrição
POST	/registry/register	Registra (ou re-registra) uma instância
PUT	/registry/heartbeat/{name}	Renova a instância
DELETE	/registry/services/{name}	Remove a instância
GET	/registry/services	Lista as instâncias com status
GET	/registry/services/{name}	Resolve o endereço de um serviço
GET	/	Painel HTML

Fonte: elaborado pelas autoras (2026).

Figura 15 – Painel do Service Registry (http://127.0.0.1:8761/)



Service Registry

Instâncias registradas (papel do Eureka) · porta 8761 · lease de 30 s

Aplicação	Feature	Porta	URL	Status	Último heartbeat	Registrado às
USERS Users Service	GestaoUsuarios	**8001**	http://127.0.0.1:8001	UP	6.8 s	16:18:24
EVENTS Events Service	GestaoEvento	**8002**	http://127.0.0.1:8002	UP	6.8 s	16:18:24
REGISTRATION Registration Service	Inscricao	**8003**	http://127.0.0.1:8003	UP	6.8 s	16:18:24
RECOMMENDATION Recommendation Service	Recomendacao	**8004**	http://127.0.0.1:8004	UP	6.8 s	16:18:24
SUBMISSIONS Submissions Service	Submissao	**8005**	http://127.0.0.1:8005	UP	6.9 s	16:18:24
REVIEWS Reviews Service	Revisao	**8006**	http://127.0.0.1:8006	UP	7.0 s	16:18:24
CERTIFICATES Certificates Service	Certificados	**8007**	http://127.0.0.1:8007	UP	7.0 s	16:18:24
NOTIFICATIONS Notifications Service	Notificacoes	**8008**	http://127.0.0.1:8008	UP	6.9 s	16:18:24
AI_AGENT AI Agent Service	AssistenteIA	**8009**	http://127.0.0.1:8009	UP	6.8 s	16:18:24

Fonte: elaborado pelas autoras (2026).

Código 15 – Arquivo `services/registry/main.py`

```
"""Service Registry — porta 8761 (papel do **Eureka** na figura de referência).

Infraestrutura do núcleo da LPS (presente em todo produto). Cada microservice
se registra ao iniciar e envia heartbeat a cada 10 s; o API Gateway e os
próprios serviços descobrem endereços aqui, em vez de usar portas fixas no código.
O registro é mantido em memória, como no Eureka.

Rotas
-----
=====================================  ==========================================
POST /registry/register                registra (ou re-registra) uma instância
PUT  /registry/heartbeat/{name}        renova a instância (sem heartbeat por 30 s → DOWN)
DELETE /registry/services/{name}       remove a instância
GET  /registry/services                lista todas as instâncias com status
GET  /registry/services/{name}         resolve o endereço de um serviço
GET  /                                 painel HTML com os serviços registrados
=====================================  ==========================================
"""
from __future__ import annotations

import time
from datetime import datetime

from fastapi import FastAPI, HTTPException
from fastapi.responses import HTMLResponse
from pydantic import BaseModel

app = FastAPI(title="Service Registry (Eureka-like)", description=__doc__, version="2.0.0")
LEASE = 30.0
_instances: dict[str, dict] = {}


class InstanceIn(BaseModel):
    name: str
```

```
    url: str
    port: int
    feature: str | None = None
    title: str = ""


def _view(i: dict) -> dict:
    age = time.time() - i["last_heartbeat"]
    return {**i, "status": "UP" if age <= LEASE else "DOWN", "seconds_since_heartbeat":
round(age, 1),
            "registered_at":
datetime.fromtimestamp(i["registered_at"]).strftime("%H:%M:%S")}


@app.post("/registry/register", tags=["registro"], summary="Registra uma instância de
microservice")
def register(body: InstanceIn):
    now = time.time()
    _instances[body.name] = {**body.model_dump(), "registered_at": now, "last_heartbeat":
now}
    return _view(_instances[body.name])


@app.put("/registry/heartbeat/{name}", tags=["registro"], summary="Renova o lease da
instância")
def heartbeat(name: str):
    if name not in _instances:
        raise HTTPException(404, "Instância não registrada")
    _instances[name]["last_heartbeat"] = time.time()
    return {"name": name, "status": "UP"}


@app.delete("/registry/services/{name}", tags=["registro"], summary="Cancela o registro")
def deregister(name: str):
    _instances.pop(name, None)
    return {"removed": name}


@app.get("/registry/services", tags=["descoberta"], summary="Lista as instâncias
registradas")
def services():
    return sorted((_view(i) for i in _instances.values()), key=lambda i: i["port"])


@app.get("/registry/services/{name}", tags=["descoberta"], summary="Resolve o endereço de
um serviço")
def resolve(name: str):
    if name not in _instances:
        raise HTTPException(404, f"Serviço '{name}' não registrado")
    return _view(_instances[name])


@app.get("/health", include_in_schema=False)
def health():
    return {"service": "registry", "status": "UP"}


@app.get("/", response_class=HTMLResponse, include_in_schema=False)
def dashboard():
    rows = "".join(
        f"<tr><td><b>{i['name'].upper()}</b><br><small>{i['title']}</small></td><td>
{i['feature'] or '—'}</td>"
        f"<td class=p>{i['port']}</td><td><code>{i['url']}</code></td>"
        f"<td><span class='s {i['status'].lower()}'>{i['status']}</span></td><td>
{i['seconds_since_heartbeat']} s</td>"
        f"<td>{i['registered_at']}</td></tr>" for i in services())
    return f"""<!doctype html><html lang=pt-br><meta charset=utf-8><meta http-equiv=refresh
content=10>
<title>Service Registry</title><style>body{{font-family:system-ui,sans-
serif;margin:32px;background:#f5f7f9;color:#1f2937}}
```

```
h1{{margin:0}}.sub{{color:#6b7280;margin:4px 0 20px}}table{{border-
collapse:collapse;width:100%;background:#fff;box-shadow:0 1px 3px #0002}}
th{{background:#5a8f29;color:#fff;text-align:left}}th,td{{padding:9px 12px;border-
bottom:1px solid #e5e7eb}}.p{{font:700 17px ui-monospace,monospace}}
.s{{padding:2px 10px;border-radius:99px;font-
weight:600}}.up{{background:#dcfce7;color:#166534}}.down{{background:#fee2e2;color:#991b1b}
}</style>
<h1>Service Registry</h1><p class=sub>Instâncias registradas (papel do Eureka) · porta 8761
· lease de {int(LEASE)} s</p>
<table><tr><th>Aplicação</th><th>Feature</th><th>Porta</th><th>URL</th><th>Status</th>
<th>Último heartbeat</th><th>Registrado às</th></tr>{rows}</table></html>"""
```

Fonte: código do projeto (2026).

Código 16 – Arquivo `common/discovery.py`

```
"""Cliente do Service Registry (descoberta de serviços, papel do Eureka).

* Ao iniciar, cada microservice se **registra** (nome, URL, feature, porta).
* A cada 10 s envia um **heartbeat**; sem heartbeat por 30 s ele fica DOWN.
* Gateway e serviços **descobrem** endereços consultando o registry, com cache
  local de 10 s. Se o registry cair, usam a configuração do produto.
"""
from __future__ import annotations

import logging
import threading
import time

import httpx

from .config import registry_url

log = logging.getLogger("discovery")
_cache: dict[str, tuple[float, str | None]] = {}
TTL = 10.0


def lookup(name: str) -> str | None:
    now = time.monotonic()
    hit = _cache.get(name)
    if hit and now - hit[0] < TTL:
        return hit[1]
    url = None
    try:
        r = httpx.get(f"{registry_url()}/registry/services/{name}", timeout=1.5)
        if r.status_code == 200 and r.json().get("status") == "UP":
            url = r.json()["url"]
    except httpx.HTTPError:
        pass
    _cache[name] = (now, url)
    return url


def register(name: str, url: str, port: int, feature: str | None, title: str) -> None:
    """Registra o serviço e mantém heartbeat em thread de fundo."""
    body = {"name": name, "url": url, "port": port, "feature": feature, "title": title}

    def loop():
        registered = False
        while True:
            try:
                if not registered:
                    httpx.post(f"{registry_url()}/registry/register", json=body,
timeout=2).raise_for_status()
                    registered = True
                    log.info("Registrado no Service Registry: %s -> %s", name, url)
                else:
                    r = httpx.put(f"{registry_url()}/registry/heartbeat/{name}", timeout=2)
```

```
                if r.status_code == 404:
                    registered = False
            except httpx.HTTPError:
                registered = False
            time.sleep(10 if registered else 2)

    threading.Thread(target=loop, daemon=True, name=f"heartbeat-{name}").start()
```

Fonte: código do projeto (2026).

6.3.3 Users Service — porta 8001 (obrigatório)

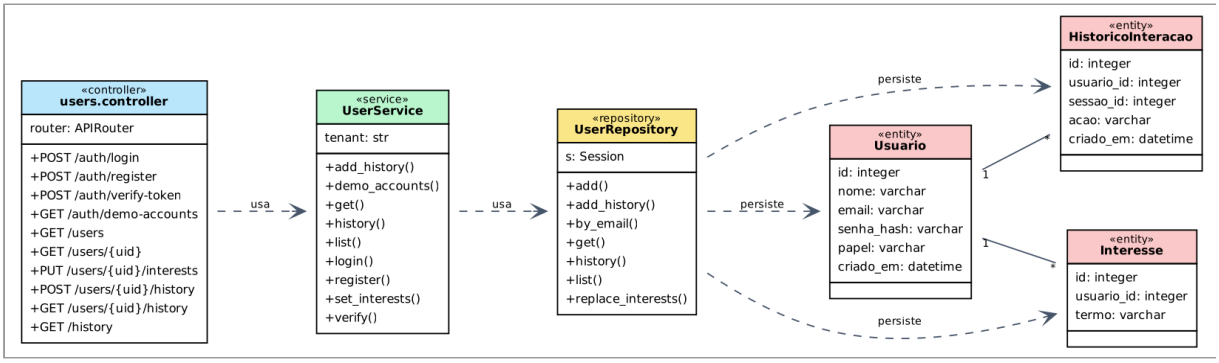
Feature **Gestão de Usuários**: cadastro, login com senha (emissão de JWT), validação de token para o gateway, interesses declarados e histórico de interações (que alimenta a recomendação). Banco `lps_<t>_users`: `usuario`, `interesse`, `historico_interacao`.

Quadro 16 – Endpoints do Users Service, acessados pelo API Gateway (http://127.0.0.1:8080)

Método	URL	Acesso	Descrição
POST	/api/users/auth/login	pública	Login: e-mail + senha → token JWT
POST	/api/users/auth/register	pública	Cadastro de usuário
POST	/api/users/auth/verify-token	pública	Valida um JWT (usado pelo API Gateway)
GET	/api/users/auth/demo-accounts	pública	Contas de demonstração (ambiente acadêmico)
GET	/api/users/users	JWT	Lista usuários
GET	/api/users/users/{uid}	JWT	Detalha usuário (com interesses)
PUT	/api/users/users/{uid}/interests	JWT	Atualiza interesses declarados
POST	/api/users/users/{uid}/history	JWT	Registra interação (view/favorite/agenda)
GET	/api/users/users/{uid}/history	JWT	Histórico de um usuário
GET	/api/users/history	JWT	Histórico global (estratégia Popularidade)

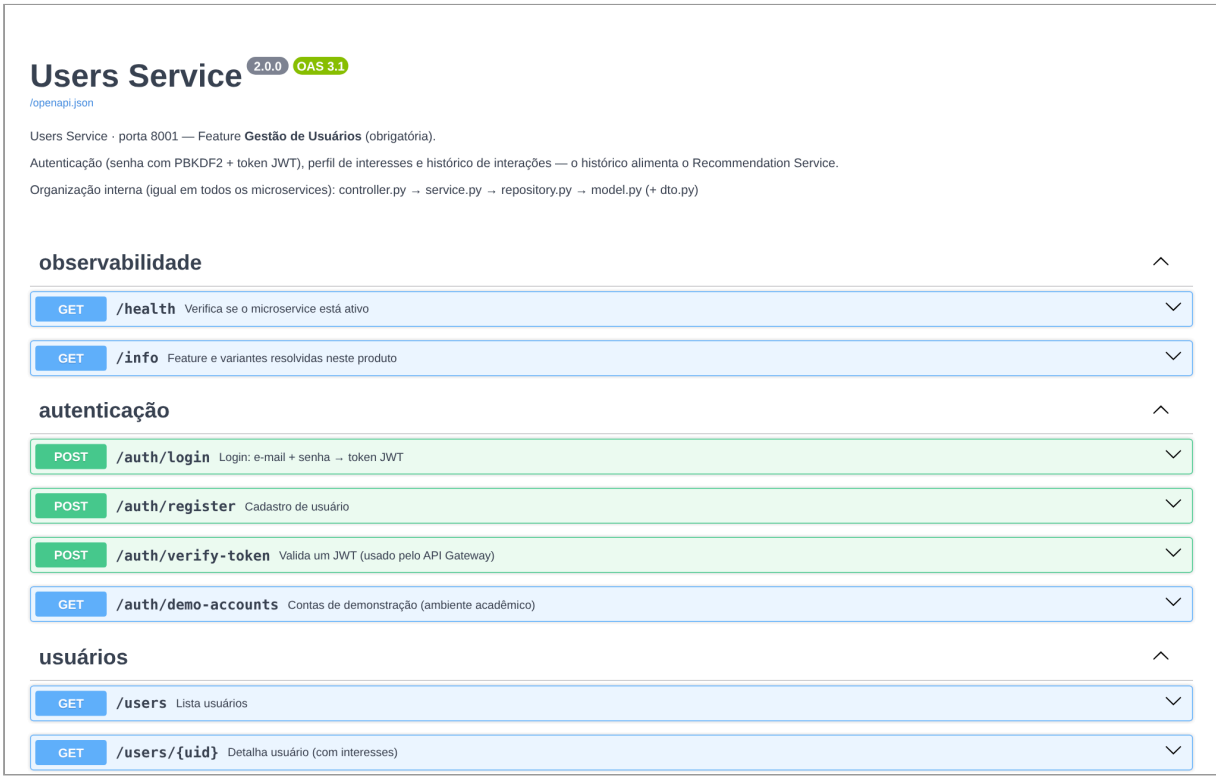
Fonte: elaborado pelas autoras (2026).

Figura 16 – Diagrama de classes — Users Service



Fonte: elaborado pelas autoras (2026).

Figura 17 – Swagger do Users Service



Fonte: elaborado pelas autoras (2026).

Código 17 – Arquivo `services/users/controller.py`

```
"""Controller — API REST do Users Service (rotas públicas e protegidas)."""
from fastapi import APIRouter, Depends, HTTPException

from common.service import Caller, current_user, tenant_header

from .dto import HistoryIn, HistoryOut, InterestsIn, LoginIn, RegisterIn, TokenOut,
UserOut, VerifyIn
from .service import DomainError, UserService

router = APIRouter()


def svc(tenant: str = Depends(tenant_header)) -> UserService:
    return UserService(tenant)


def guard(fn, *args):
    try:
        return fn(*args)
    except DomainError as e:
        raise HTTPException(e.status, str(e))


@router.post("/auth/login", response_model=TokenOut, tags=["autenticação"], summary="Login:
e-mail + senha → token JWT")
def login(body: LoginIn, s: UserService = Depends(svc)):
    return guard(s.login, body.email, body.password)


@router.post("/auth/register", response_model=UserOut, status_code=201, tags=
["autenticação"], summary="Cadastro de usuário")
def register(body: RegisterIn, s: UserService = Depends(svc)):
    return guard(s.register, body)
```

```
@router.post("/auth/verify-token", tags=["autenticação"], summary="Valida um JWT (usado
pelo API Gateway)")
def verify(body: VerifyIn, s: UserService = Depends(svc)):
    return guard(s.verify, body.token)


@router.get("/auth/demo-accounts", tags=["autenticação"], summary="Contas de demonstração
(ambiente acadêmico)")
def demo(s: UserService = Depends(svc)):
    return s.demo_accounts()


@router.get("/users", response_model=list[UserOut], tags=["usuários"], summary="Lista
usuários")
def list_users(s: UserService = Depends(svc)):
    return s.list()


@router.get("/users/{uid}", response_model=UserOut, tags=["usuários"], summary="Detalha
usuário (com interesses)")
def get_user(uid: int, s: UserService = Depends(svc)):
    return guard(s.get, uid)


@router.put("/users/{uid}/interests", response_model=UserOut, tags=["perfil"],
summary="Atualiza interesses declarados")
def set_interests(uid: int, body: InterestsIn, s: UserService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_self_or(uid, "organizador")
    return guard(s.set_interests, uid, body.interests)


@router.post("/users/{uid}/history", response_model=HistoryOut, status_code=201, tags=
["perfil"],
             summary="Registra interação (view/favorite/agenda)")
def add_history(uid: int, body: HistoryIn, s: UserService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_self_or(uid, "organizador")
    return guard(s.add_history, uid, body.session_id, body.action)


@router.get("/users/{uid}/history", response_model=list[HistoryOut], tags=["perfil"],
summary="Histórico de um usuário")
def user_history(uid: int, s: UserService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_self_or(uid, "organizador")
    return s.history(uid)


@router.get("/history", response_model=list[HistoryOut], tags=["perfil"],
summary="Histórico global (estratégia Popularidade)")
def all_history(s: UserService = Depends(svc)):
    return s.history()
```

Fonte: código do projeto (2026).

Código 18 – Arquivo `services/users/model.py`

```
"""Model — entidades do Users Service (banco ``lps_<tenant>_users``).

Modelo relacional normalizado (3FN):
  usuario(id PK, nome, email UNIQUE, senha_hash, papel, criado_em)
  interesse(id PK, usuario_id FK→usuario, termo)            UNIQUE(usuario_id, termo)
  historico_interacao(id PK, usuario_id FK→usuario, sessao_id, acao, criado_em)

``sessao_id`` é referência lógica ao Events Service (database per service:
não há FK física entre bancos de microservices diferentes).
"""
```

```
from __future__ import annotations

from datetime import datetime

from sqlalchemy import DateTime, ForeignKey, Integer, String, UniqueConstraint, func
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship


class Base(DeclarativeBase):
    pass


class Usuario(Base):
    __tablename__ = "usuario"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    nome: Mapped[str] = mapped_column(String(120))
    email: Mapped[str] = mapped_column(String(160), unique=True, index=True)
    senha_hash: Mapped[str] = mapped_column(String(255))
    papel: Mapped[str] = mapped_column(String(20), default="participante")
    criado_em: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())
    interesses: Mapped[list["Interesse"]] = relationship(back_populates="usuario",
cascade="all, delete-orphan",
                                                         order_by="Interesse.id")
    historico: Mapped[list["HistoricoInteracao"]] = relationship(back_populates="usuario",
cascade="all, delete-orphan")


class Interesse(Base):
    __tablename__ = "interesse"
    __table_args__ = (UniqueConstraint("usuario_id", "termo",
name="uq_interesse_usuario_termo"),)
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    usuario_id: Mapped[int] = mapped_column(ForeignKey("usuario.id", ondelete="CASCADE"),
index=True)
    termo: Mapped[str] = mapped_column(String(80))
    usuario: Mapped[Usuario] = relationship(back_populates="interesses")


class HistoricoInteracao(Base):
    __tablename__ = "historico_interacao"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    usuario_id: Mapped[int] = mapped_column(ForeignKey("usuario.id", ondelete="CASCADE"),
index=True)
    sessao_id: Mapped[int] = mapped_column(Integer, index=True)
    acao: Mapped[str] = mapped_column(String(10))          # view | favorite | agenda
    criado_em: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())
    usuario: Mapped[Usuario] = relationship(back_populates="historico")
```

Fonte: código do projeto (2026).

6.3.4 Events Service — porta 8002 (obrigatório)

Feature **Gestão do Evento**: dados do evento e programação (sessões com trilha, tags, palestrante, dia, horário e sala). A rota `/tracks` e o filtro por trilha só existem quando a feature **Trilhas** está no produto. Somente o papel `organizador` cadastra sessões. Banco `lps_<t>_events`: `trilha`, `tag`, `sessao`, `sessao_tag`.

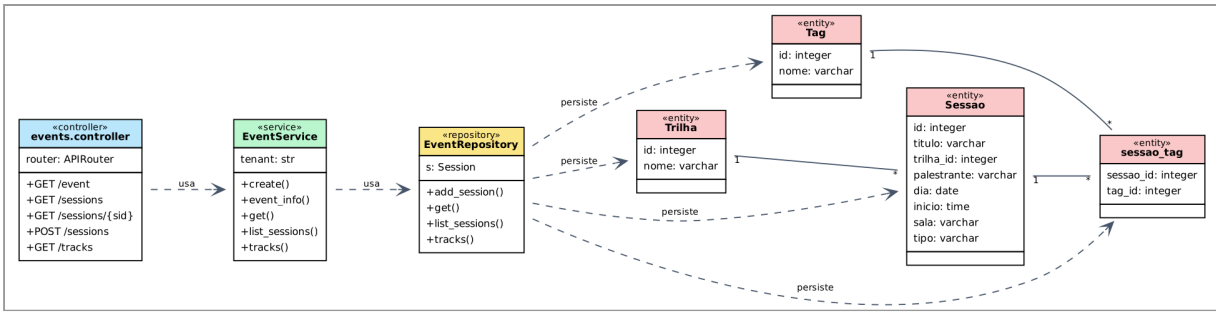
Quadro 17 – Endpoints do Events Service, acessados pelo API Gateway (http://127.0.0.1:8080)

Método	URL	Acesso	Descrição
GET	/api/events/event	pública	Dados gerais do evento (tenant)
GET	/api/events/sessions	pública	Lista/busca sessões

Método	URL	Acesso	Descrição
GET	/api/events/sessions/{sid}	pública	Detalha uma sessão
POST	/api/events/sessions	JWT	Cadastra sessão (organizador)
GET	/api/events/tracks	pública	Trilhas temáticas (feature opcional Trilhas)

Fonte: elaborado pelas autoras (2026).

Figura 18 – Diagrama de classes — Events Service



Fonte: elaborado pelas autoras (2026).

Código 19 – Arquivo `services/events/controller.py`

```
"""Controller — API REST do Events Service."""
from fastapi import APIRouter, Depends, HTTPException

from common.config import has_feature
from common.service import Caller, current_user, tenant_header

from .dto import EventOut, SessionIn, SessionOut
from .service import EventService

router = APIRouter()


def svc(tenant: str = Depends(tenant_header)) -> EventService:
    return EventService(tenant)


@router.get("/event", response_model=EventOut, tags=["evento"], summary="Dados gerais do
evento (tenant)")
def event(s: EventService = Depends(svc)):
    return s.event_info()


@router.get("/sessions", response_model=list[SessionOut], tags=["programação"],
summary="Lista/busca sessões")
def sessions(q: str | None = None, track: str | None = None, day: str | None = None, s:
EventService = Depends(svc)):
    return s.list_sessions(q, track, day)


@router.get("/sessions/{sid}", response_model=SessionOut, tags=["programação"],
summary="Detalha uma sessão")
def session(sid: int, s: EventService = Depends(svc)):
    x = s.get(sid)
    if not x:
        raise HTTPException(404, "Sessão não encontrada")
    return x


@router.post("/sessions", response_model=SessionOut, status_code=201, tags=["programação"],
summary="Cadastra sessão (organizador)")
def create(body: SessionIn, s: EventService = Depends(svc), who: Caller =
```

```
Depends(current_user)):
    who.ensure_role("organizador")
    return s.create(body)


@router.get("/tracks", response_model=list[str], tags=["trilhas"], summary="Trilhas
temáticas (feature opcional Trilhas)")
def tracks(s: EventService = Depends(svc)):
    if not has_feature("Trilhas"):
        raise HTTPException(404, "Feature 'Trilhas' não incluída neste produto")
    return s.tracks()
```

Fonte: código do projeto (2026).

Código 20 – Arquivo `services/events/model.py`

```
"""Model — entidades do Events Service (banco ``lps_<tenant>_events``).

Modelo relacional normalizado (3FN). As tags e trilhas, que antes eram listas
dentro da sessão, viraram tabelas próprias (1FN: atributos atômicos) e a
relação N:N sessão–tag usa tabela associativa:
  trilha(id PK, nome UNIQUE)
  sessao(id PK, titulo, trilha_id FK→trilha NULL, palestrante, dia, inicio, sala, tipo)
  tag(id PK, nome UNIQUE)
  sessao_tag(sessao_id FK→sessao, tag_id FK→tag)  PK composta
"""
from __future__ import annotations

from datetime import date, time

from sqlalchemy import Column, Date, ForeignKey, Integer, String, Table, Time
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship


class Base(DeclarativeBase):
    pass


sessao_tag = Table(
    "sessao_tag", Base.metadata,
    Column("sessao_id", ForeignKey("sessao.id", ondelete="CASCADE"), primary_key=True),
    Column("tag_id", ForeignKey("tag.id", ondelete="CASCADE"), primary_key=True),
)


class Trilha(Base):
    __tablename__ = "trilha"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    nome: Mapped[str] = mapped_column(String(80), unique=True)


class Tag(Base):
    __tablename__ = "tag"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    nome: Mapped[str] = mapped_column(String(80), unique=True)


class Sessao(Base):
    __tablename__ = "sessao"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    titulo: Mapped[str] = mapped_column(String(200))
    trilha_id: Mapped[int | None] = mapped_column(ForeignKey("trilha.id", ondelete="SET
NULL"), nullable=True, index=True)
    palestrante: Mapped[str] = mapped_column(String(120), default="")
    dia: Mapped[date] = mapped_column(Date, index=True)
    inicio: Mapped[time] = mapped_column(Time)
    sala: Mapped[str] = mapped_column(String(60), default="")
    tipo: Mapped[str] = mapped_column(String(30), default="Palestra")
```

```
    trilha: Mapped[Trilha | None] = relationship()
    tags: Mapped[list[Tag]] = relationship(secondary=sessao_tag, order_by=Tag.id)
```

Fonte: código do projeto (2026).

6.3.5 Registration Service — porta 8003 (obrigatório, VP Pagamento)

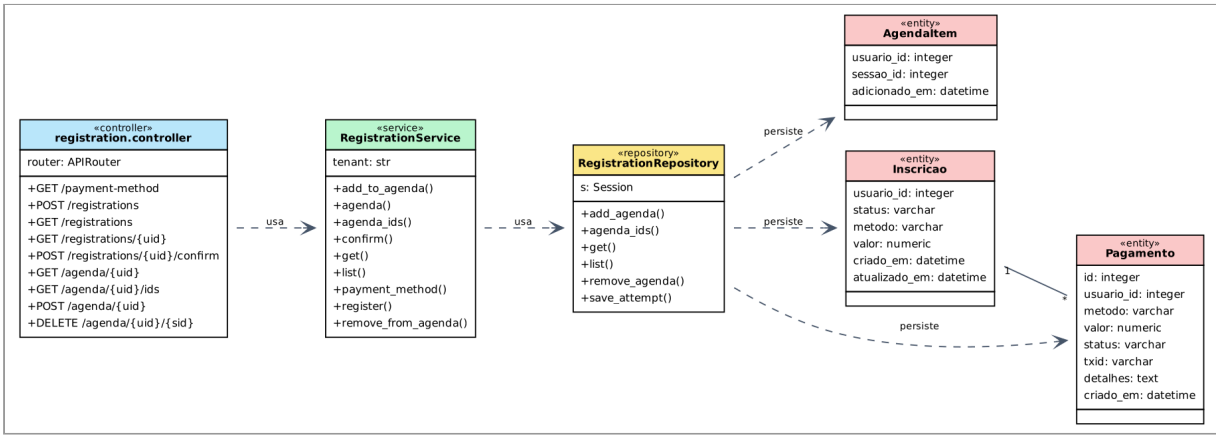
Feature **Inscrição**: inscrição no evento usando o componente `PaymentMethod` resolvido para o produto (Gratuito, PIX ou Cartão — todos simulados), confirmação do PIX e agenda pessoal. Ao adicionar uma sessão à agenda, registra a ação no histórico do Users Service e publica o evento `registration.confirmed`. Banco `lps_<t>_registration`: `inscricao`, `pagamento`, `agenda_item`.

Quadro 18 – Endpoints do Registration Service, acessados pelo API Gateway (http://127.0.0.1:8080)

Método	URL	Acesso	Descrição
GET	/api/registration/payment-method	pública	Variante de pagamento ativa
POST	/api/registration/registrations	JWT	Realiza inscrição e cobra via componente de pagamento
GET	/api/registration/registrations	JWT	Lista inscrições (organizador)
GET	/api/registration/registrations/{uid}	JWT	Situação da inscrição
POST	/api/registration/registrations/{uid}/confirm	JWT	Confirma pagamento pendente (PIX simulado)
GET	/api/registration/agenda/{uid}	JWT	Agenda pessoal (sessões detalhadas)
GET	/api/registration/agenda/{uid}/ids	JWT	IDs das sessões na agenda
POST	/api/registration/agenda/{uid}	JWT	Adiciona sessão à agenda (gera histórico)
DELETE	/api/registration/agenda/{uid}/{sid}	JWT	Remove sessão da agenda

Fonte: elaborado pelas autoras (2026).

Figura 19 – Diagrama de classes — Registration Service



Fonte: elaborado pelas autoras (2026).

Código 21 – Arquivo `services/registration/controller.py`

```
"""Controller — API REST do Registration Service."""
from fastapi import APIRouter, Depends, HTTPException

from common.service import Caller, current_user, tenant_header

from .dto import AgendaIn, PaymentMethodOut, RegistrationIn, RegistrationOut
from .service import RegistrationService

router = APIRouter()


def svc(tenant: str = Depends(tenant_header)) -> RegistrationService:
    return RegistrationService(tenant)


@router.get("/payment-method", response_model=PaymentMethodOut, tags=["pagamento"],
summary="Variante de pagamento ativa")
def payment_method():
    return RegistrationService.payment_method()


@router.post("/registrations", response_model=RegistrationOut, status_code=201, tags=
["inscrição"],
             summary="Realiza inscrição e cobra via componente de pagamento")
def register(body: RegistrationIn, s: RegistrationService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_self_or(body.user_id, "organizador")
    return s.register(body.user_id, body.model_dump())


@router.get("/registrations", response_model=list[RegistrationOut], tags=["inscrição"],
summary="Lista inscrições (organizador)")
def list_all(s: RegistrationService = Depends(svc), who: Caller = Depends(current_user)):
    who.ensure_role("organizador")
    return s.list()


@router.get("/registrations/{uid}", response_model=RegistrationOut, tags=["inscrição"],
summary="Situação da inscrição")
def get(uid: int, s: RegistrationService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_self_or(uid, "organizador")
    r = s.get(uid)
    if not r:
        raise HTTPException(404, "Usuário não inscrito")
    return r
```

```
@router.post("/registrations/{uid}/confirm", response_model=RegistrationOut, tags=
["pagamento"],
             summary="Confirma pagamento pendente (PIX simulado)")
def confirm(uid: int, s: RegistrationService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_self_or(uid, "organizador")
    r = s.confirm(uid)
    if not r:
        raise HTTPException(404, "Usuário não inscrito")
    return r


@router.get("/agenda/{uid}", tags=["agenda"], summary="Agenda pessoal (sessões
detalhadas)")
def agenda(uid: int, s: RegistrationService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_self_or(uid, "organizador")
    return s.agenda(uid)


@router.get("/agenda/{uid}/ids", response_model=list[int], tags=["agenda"], summary="IDs
das sessões na agenda")
def agenda_ids(uid: int, s: RegistrationService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_self_or(uid, "organizador")
    return s.agenda_ids(uid)


@router.post("/agenda/{uid}", tags=["agenda"], summary="Adiciona sessão à agenda (gera
histórico)")
def add(uid: int, body: AgendaIn, s: RegistrationService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_self_or(uid)
    return s.add_to_agenda(uid, body.session_id)


@router.delete("/agenda/{uid}/{sid}", tags=["agenda"], summary="Remove sessão da agenda")
def remove(uid: int, sid: int, s: RegistrationService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_self_or(uid)
    return s.remove_from_agenda(uid, sid)
```

Fonte: código do projeto (2026).

Código 22 – Arquivo `services/registration/model.py`

```
"""Model — entidades do Registration Service (banco ``lps_<tenant>_registration``).

  inscricao(usuario_id PK, status, metodo, valor, criado_em, atualizado_em)
  pagamento(id PK, usuario_id FK→inscricao, metodo, valor, status, txid, detalhes,
criado_em)
  agenda_item(usuario_id, sessao_id, adicionado_em)              PK composta (usuario_id,
sessao_id)

Uma inscrição pode ter várias tentativas de pagamento (1:N) — por isso
``pagamento`` é tabela separada (evita grupo repetido, 1FN/2FN).
"""
from __future__ import annotations

from datetime import datetime
from decimal import Decimal

from sqlalchemy import DateTime, ForeignKey, Integer, Numeric, String, Text, func
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship


class Base(DeclarativeBase):
    pass
```

```
class Inscricao(Base):
    __tablename__ = "inscricao"
    usuario_id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=False)
    status: Mapped[str] = mapped_column(String(10))            # pending | approved |
refused
    metodo: Mapped[str] = mapped_column(String(20))            # variante do componente de
pagamento
    valor: Mapped[Decimal] = mapped_column(Numeric(10, 2))
    criado_em: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())
    atualizado_em: Mapped[datetime] = mapped_column(DateTime, server_default=func.now(),
onupdate=func.now())
    pagamentos: Mapped[list["Pagamento"]] = relationship(cascade="all, delete-orphan",
order_by="Pagamento.id")


class Pagamento(Base):
    __tablename__ = "pagamento"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    usuario_id: Mapped[int] = mapped_column(ForeignKey("inscricao.usuario_id",
ondelete="CASCADE"), index=True)
    metodo: Mapped[str] = mapped_column(String(20))
    valor: Mapped[Decimal] = mapped_column(Numeric(10, 2))
    status: Mapped[str] = mapped_column(String(10))
    txid: Mapped[str | None] = mapped_column(String(40), nullable=True)
    detalhes: Mapped[str] = mapped_column(Text, default="{}")   # resposta do componente
(JSON)
    criado_em: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())


class AgendaItem(Base):
    __tablename__ = "agenda_item"
    usuario_id: Mapped[int] = mapped_column(Integer, primary_key=True)
    sessao_id: Mapped[int] = mapped_column(Integer, primary_key=True)
    adicionado_em: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())
```

Fonte: código do projeto (2026).

6.3.6 Recommendation Service — porta 8004 (obrigatório, VP Estratégia)

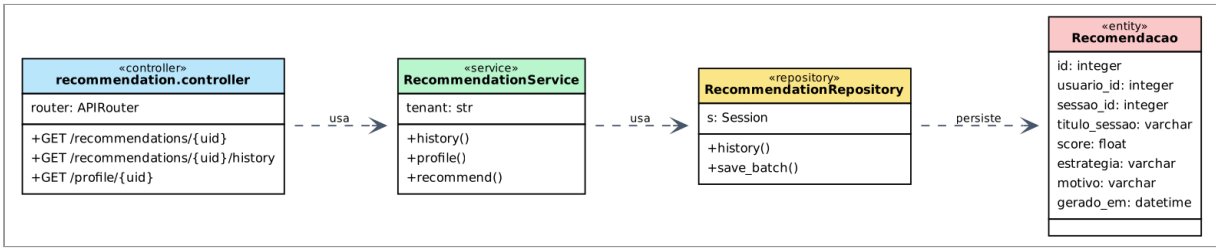
Feature **Recomendação** (descrita na seção 6.1). Banco `lps_<t>_recommendation`: `recomendacao` (histórico das recomendações geradas, com estratégia, score e motivo).

Quadro 19 – Endpoints do Recommendation Service, acessados pelo API Gateway (http://127.0.0.1:8080)

Método	URL	Acesso	Descrição
GET	/api/recommendation/recommendations/{uid}	JWT	Top-k sessões recomendadas pelo perfil (interesses + histórico)
GET	/api/recommendation/recommendations/{uid}/history	JWT	Recomendações já geradas para o usuário (persistidas)
GET	/api/recommendation/profile/{uid}	JWT	Perfil consolidado usado pela recomendação

Fonte: elaborado pelas autoras (2026).

Figura 20 – Diagrama de classes — Recommendation Service



Fonte: elaborado pelas autoras (2026).

Código 23 – Arquivo `services/recommendation/controller.py`

```
"""Controller — API REST do Recommendation Service."""
from fastapi import APIRouter, Depends, HTTPException

from common.service import Caller, current_user, tenant_header

from .dto import RecommendationLog, RecommendationOut
from .service import RecommendationService

router = APIRouter()


def svc(tenant: str = Depends(tenant_header)) -> RecommendationService:
    return RecommendationService(tenant)


@router.get("/recommendations/{uid}", response_model=RecommendationOut, tags=
["recomendação"],
            summary="Top-k sessões recomendadas pelo perfil (interesses + histórico)")
def recommend(uid: int, k: int = 5, s: RecommendationService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_self_or(uid, "organizador")
    r = s.recommend(uid, k)
    if r is None:
        raise HTTPException(404, "Usuário não encontrado")
    return r


@router.get("/recommendations/{uid}/history", response_model=list[RecommendationLog], tags=
["recomendação"],
            summary="Recomendações já geradas para o usuário (persistidas)")
def history(uid: int, s: RecommendationService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_self_or(uid, "organizador")
    return s.history(uid)


@router.get("/profile/{uid}", tags=["recomendação"], summary="Perfil consolidado usado pela
recomendação")
def profile(uid: int, s: RecommendationService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_self_or(uid, "organizador")
    p = s.profile(uid)
    if p is None:
        raise HTTPException(404, "Usuário não encontrado")
    return p
```

Fonte: código do projeto (2026).

Código 24 – Arquivo `services/recommendation/model.py`

```
"""Model — entidades do Recommendation Service (banco ``lps_<tenant>_recommendation``).

Guarda cada recomendação gerada, para auditoria e para o histórico do usuário:
```

```
  recomendacao(id PK, usuario_id, sessao_id, titulo_sessao, score, estrategia, motivo,
gerado_em)
"""
from __future__ import annotations

from datetime import datetime

from sqlalchemy import DateTime, Float, Integer, String, func
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column


class Base(DeclarativeBase):
    pass


class Recomendacao(Base):
    __tablename__ = "recomendacao"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    usuario_id: Mapped[int] = mapped_column(Integer, index=True)
    sessao_id: Mapped[int] = mapped_column(Integer)
    titulo_sessao: Mapped[str] = mapped_column(String(200))
    score: Mapped[float] = mapped_column(Float)
    estrategia: Mapped[str] = mapped_column(String(30))
    motivo: Mapped[str] = mapped_column(String(300))
    gerado_em: Mapped[datetime] = mapped_column(DateTime, server_default=func.now(),
index=True)
```

Fonte: código do projeto (2026).

6.3.7 Submissions Service — porta 8005 (opcional, VP Formatos)

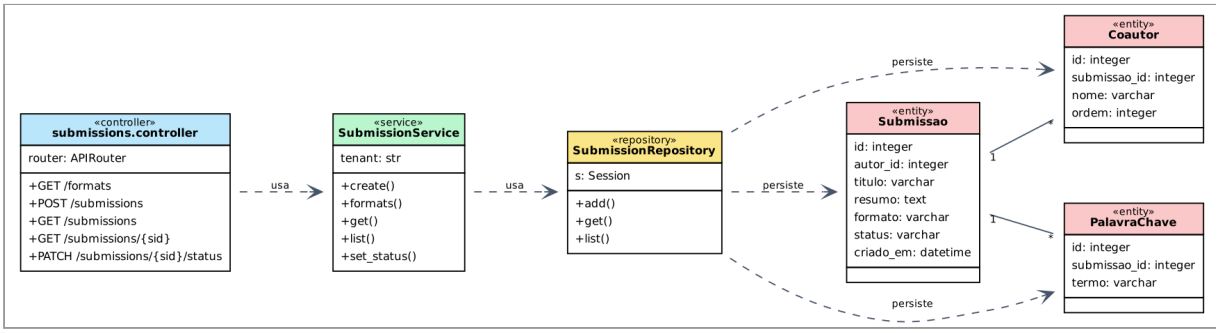
Feature **Submissão de Trabalhos**: submissão nos formatos aceitos pelo produto (grupo or: Artigo completo, Resumo expandido, Pôster), com coautores e palavras-chave; o status é atualizado pelo Reviews Service. Publica `submission.created`. Banco `lps_<t>_submissions`: `submissao`, `coautor`, `palavra_chave`.

Quadro 20 – Endpoints do Submissions Service, acessados pelo API Gateway (http://127.0.0.1:8080)

Método	URL	Acesso	Descrição
GET	/api/submissions/formats	pública	Formatos aceitos neste produto
POST	/api/submissions/submissions	JWT	Submete um trabalho
GET	/api/submissions/submissions	JWT	Lista submissões (filtro por autor)
GET	/api/submissions/submissions/{sid}	JWT	Detalha submissão
PATCH	/api/submissions/submissions/{sid}/status	JWT	Atualiza status (chamado pelo Reviews Service)

Fonte: elaborado pelas autoras (2026).

Figura 21 – Diagrama de classes — Submissions Service



Fonte: elaborado pelas autoras (2026).

Código 25 – Arquivo `services/submissions/controller.py`

```
"""Controller — API REST do Submissions Service."""
from fastapi import APIRouter, Depends, HTTPException

from common.service import Caller, current_user, tenant_header

from .dto import FormatOut, StatusIn, SubmissionIn, SubmissionOut
from .service import SubmissionService

router = APIRouter()


def svc(tenant: str = Depends(tenant_header)) -> SubmissionService:
    return SubmissionService(tenant)


@router.get("/formats", response_model=list[FormatOut], tags=["submissão"],
summary="Formatos aceitos neste produto")
def formats():
    return SubmissionService.formats()


@router.post("/submissions", response_model=SubmissionOut, status_code=201, tags=
["submissão"], summary="Submete um trabalho")
def create(body: SubmissionIn, s: SubmissionService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_self_or(body.author_id)
    try:
        return s.create(body)
    except ValueError as e:
        raise HTTPException(422, str(e))


@router.get("/submissions", response_model=list[SubmissionOut], tags=["submissão"],
summary="Lista submissões (filtro por autor)")
def list_(author_id: int | None = None, s: SubmissionService = Depends(svc), who: Caller =
Depends(current_user)):
    if author_id is None:
        who.ensure_role("revisor", "organizador")
    else:
        who.ensure_self_or(author_id, "revisor", "organizador")
    return s.list(author_id)


@router.get("/submissions/{sid}", response_model=SubmissionOut, tags=["submissão"],
summary="Detalha submissão")
def get(sid: int, s: SubmissionService = Depends(svc)):
    x = s.get(sid)
    if not x:
        raise HTTPException(404, "Submissão não encontrada")
    return x
```

```
@router.patch("/submissions/{sid}/status", response_model=SubmissionOut, tags=
["submissão"],
              summary="Atualiza status (chamado pelo Reviews Service)")
def status(sid: int, body: StatusIn, s: SubmissionService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_role("organizador")
    x = s.set_status(sid, body.status)
    if not x:
        raise HTTPException(404, "Submissão não encontrada")
    return x
```

Fonte: código do projeto (2026).

Código 26 – Arquivo `services/submissions/model.py`

```
"""Model — entidades do Submissions Service (banco ``lps_<tenant>_submissions``).

  submissao(id PK, autor_id, titulo, resumo, formato, status, criado_em)
  coautor(id PK, submissao_id FK→submissao, nome, ordem)
  palavra_chave(id PK, submissao_id FK→submissao, termo)

Autores e palavras-chave são multivalorados, então ficam em tabelas próprias (1FN).
"""
from __future__ import annotations

from datetime import datetime

from sqlalchemy import DateTime, ForeignKey, Integer, String, Text, func
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship


class Base(DeclarativeBase):
    pass


class Submissao(Base):
    __tablename__ = "submissao"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    autor_id: Mapped[int] = mapped_column(Integer, index=True)
    titulo: Mapped[str] = mapped_column(String(250))
    resumo: Mapped[str] = mapped_column(Text, default="")
    formato: Mapped[str] = mapped_column(String(20))
    status: Mapped[str] = mapped_column(String(15), default="submitted")
    criado_em: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())
    coautores: Mapped[list["Coautor"]] = relationship(cascade="all, delete-orphan",
order_by="Coautor.ordem")
    palavras: Mapped[list["PalavraChave"]] = relationship(cascade="all, delete-orphan",
order_by="PalavraChave.id")


class Coautor(Base):
    __tablename__ = "coautor"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    submissao_id: Mapped[int] = mapped_column(ForeignKey("submissao.id",
ondelete="CASCADE"), index=True)
    nome: Mapped[str] = mapped_column(String(120))
    ordem: Mapped[int] = mapped_column(Integer)


class PalavraChave(Base):
    __tablename__ = "palavra_chave"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    submissao_id: Mapped[int] = mapped_column(ForeignKey("submissao.id",
ondelete="CASCADE"), index=True)
    termo: Mapped[str] = mapped_column(String(60))
```

Fonte: código do projeto (2026).

6.3.8 Reviews Service — porta 8006 (opcional, VP Política de Revisão)

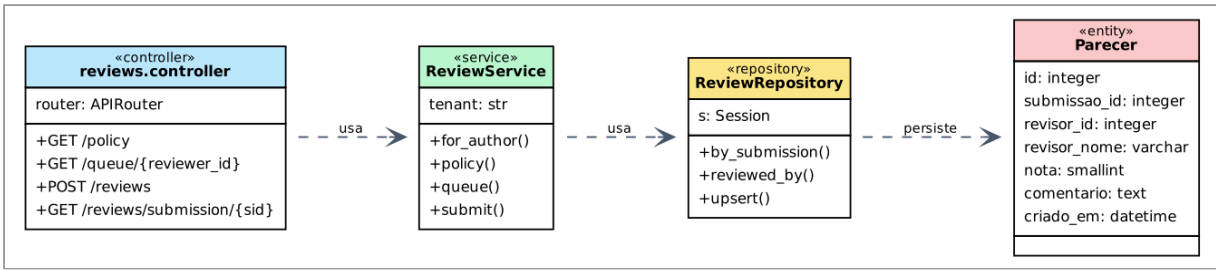
Feature **Revisão por Pares** (*requires* Submissão). O componente `ReviewPolicy` decide o que o revisor vê do autor e o que o autor vê do revisor (simples-cego, duplo-cego ou aberta). A cada parecer é recalculada a média das notas (escala −3 a 3): média ≥ 1 aceita, média < 0 rejeita e, entre esses valores, o trabalho continua em revisão; o status é atualizado no Submissions Service. Banco `lps_<t>_reviews`: `parecer` (UQ submissão + revisor).

Quadro 21 – Endpoints do Reviews Service, acessados pelo API Gateway (http://127.0.0.1:8080)

Método	URL	Acesso	Descrição
GET	/api/reviews/policy	pública	Política de revisão ativa
GET	/api/reviews/queue/{reviewer_id}	JWT	Fila do revisor (com anonimização da política)
POST	/api/reviews/reviews	JWT	Registra parecer e decide aceite pela média
GET	/api/reviews/reviews/submission/{sid}	JWT	Pareceres vistos pelo autor (com anonimização)

Fonte: elaborado pelas autoras (2026).

Figura 22 – Diagrama de classes — Reviews Service



Fonte: elaborado pelas autoras (2026).

Código 27 – Arquivo `services/reviews/controller.py`

```
"""Controller — API REST do Reviews Service."""
from fastapi import APIRouter, Depends

from common.service import Caller, current_user, tenant_header

from .dto import DecisionOut, PolicyOut, ReviewIn, ReviewOut
from .service import ReviewService

router = APIRouter()


def svc(tenant: str = Depends(tenant_header)) -> ReviewService:
    return ReviewService(tenant)


@router.get("/policy", response_model=PolicyOut, tags=["revisão"], summary="Política de
revisão ativa")
def policy():
    return ReviewService.policy()
```

```
@router.get("/queue/{reviewer_id}", tags=["revisão"], summary="Fila do revisor (com
anonimização da política)")
def queue(reviewer_id: int, s: ReviewService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_role("revisor", "organizador")
    who.ensure_self_or(reviewer_id, "organizador")
    return s.queue(reviewer_id)


@router.post("/reviews", response_model=DecisionOut, status_code=201, tags=["revisão"],
             summary="Registra parecer e decide aceite pela média")
def review(body: ReviewIn, s: ReviewService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_role("revisor", "organizador")
    who.ensure_self_or(body.reviewer_id, "organizador")
    return s.submit(body)


@router.get("/reviews/submission/{sid}", response_model=list[ReviewOut], tags=["revisão"],
            summary="Pareceres vistos pelo autor (com anonimização)")
def for_author(sid: int, s: ReviewService = Depends(svc)):
    return s.for_author(sid)
```

Fonte: código do projeto (2026).

Código 28 – Arquivo `components/review_policy.py`

```
"""Ponto de variação **Política de Revisão** (xor): SimplesCego | DuploCego | Aberta.

A política decide o que cada papel enxerga: o revisor vê os autores? O autor vê
o nome do revisor?
"""
from __future__ import annotations

from abc import ABC, abstractmethod

from .base import VariationPoint


class ReviewPolicy(ABC):
    label = "?"
    reviewer_sees_authors: bool
    author_sees_reviewer: bool

    def submission_for_reviewer(self, sub: dict) -> dict:
        if self.reviewer_sees_authors:
            return sub
        return {**sub, "authors": "[anônimo]", "author_id": None}

    def review_for_author(self, review: dict) -> dict:
        if self.author_sees_reviewer:
            return review
        return {**review, "reviewer_id": None, "reviewer_name": "Revisor anônimo"}

    @abstractmethod
    def describe(self) -> str: ...


review_vp: VariationPoint[ReviewPolicy] = VariationPoint("review_policy", ReviewPolicy)


@review_vp.variant("SimplesCego")
class SingleBlind(ReviewPolicy):
    label = "Simples-cego"
    reviewer_sees_authors = True
    author_sees_reviewer = False

    def describe(self):
```

```
        return "O revisor conhece os autores; os autores não conhecem o revisor."


@review_vp.variant("DuploCego")
class DoubleBlind(ReviewPolicy):
    label = "Duplo-cego"
    reviewer_sees_authors = False
    author_sees_reviewer = False

    def describe(self):
        return "Autores e revisores permanecem anônimos entre si."


@review_vp.variant("Aberta")
class OpenReview(ReviewPolicy):
    label = "Revisão aberta"
    reviewer_sees_authors = True
    author_sees_reviewer = True

    def describe(self):
        return "Identidades de autores e revisores são públicas."
```

Fonte: código do projeto (2026).

6.3.9 Certificates Service — porta 8007 (opcional, VP Tipos de Certificado)

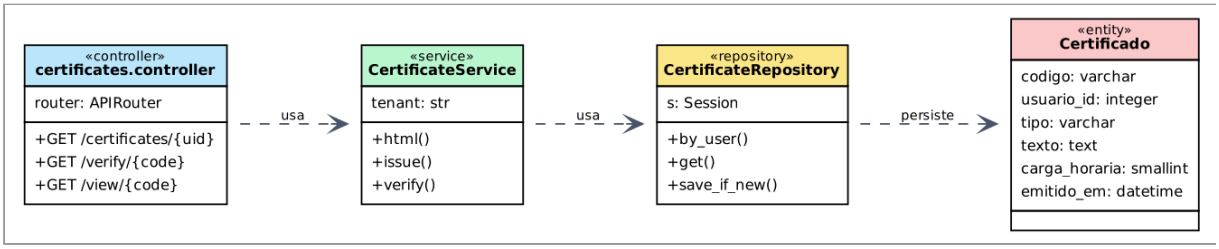
Feature **Emissão de Certificados**: o componente `CertificateIssuer` (grupo or) emite Participação (inscrição confirmada) e/ou Apresentação (trabalho aceito), com código de verificação público e versão HTML. Banco `lps_<t>_certificates`: `certificado`.

Quadro 22 – Endpoints do Certificates Service, acessados pelo API Gateway (http://127.0.0.1:8080)

Método	URL	Acesso	Descrição
GET	/api/certificates/certificates/{uid}	JWT	Emite (idempotente) e lista certificados do usuário
GET	/api/certificates/verify/{code}	pública	Verifica autenticidade (público)
GET	/api/certificates/view/{code}	pública	Certificado em HTML (público)

Fonte: elaborado pelas autoras (2026).

Figura 23 – Diagrama de classes — Certificates Service



Fonte: elaborado pelas autoras (2026).

Código 29 – Arquivo `services/certificates/controller.py`

```
"""Controller — API REST do Certificates Service."""
from fastapi import APIRouter, Depends, HTTPException
from fastapi.responses import HTMLResponse
```

```
from common.service import Caller, current_user, tenant_header

from .dto import CertificateOut
from .service import CertificateService

router = APIRouter()


def svc(tenant: str = Depends(tenant_header)) -> CertificateService:
    return CertificateService(tenant)


@router.get("/certificates/{uid}", response_model=list[CertificateOut], tags=
["certificados"],
            summary="Emite (idempotente) e lista certificados do usuário")
def issue(uid: int, s: CertificateService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_self_or(uid, "organizador")
    r = s.issue(uid)
    if r is None:
        raise HTTPException(404, "Usuário não encontrado")
    return r


@router.get("/verify/{code}", response_model=CertificateOut, tags=["certificados"],
summary="Verifica autenticidade (público)")
def verify(code: str, s: CertificateService = Depends(svc)):
    c = s.verify(code)
    if not c:
        raise HTTPException(404, "Certificado inválido")
    return c


@router.get("/view/{code}", response_class=HTMLResponse, tags=["certificados"],
summary="Certificado em HTML (público)")
def view(code: str, s: CertificateService = Depends(svc)):
    h = s.html(code)
    if not h:
        raise HTTPException(404, "Certificado inválido")
    return h
```

Fonte: código do projeto (2026).

Código 30 – Arquivo `services/certificates/model.py`

```
"""Model — entidades do Certificates Service (banco ``lps_<tenant>_certificates``).

  certificado(codigo PK, usuario_id, tipo, texto, carga_horaria, emitido_em)
O código (hash SHA-256 truncado) é a chave natural e serve para verificação pública.
"""
from __future__ import annotations

from datetime import datetime

from sqlalchemy import DateTime, Integer, SmallInteger, String, Text, func
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column


class Base(DeclarativeBase):
    pass


class Certificado(Base):
    __tablename__ = "certificado"
    codigo: Mapped[str] = mapped_column(String(16), primary_key=True)
    usuario_id: Mapped[int] = mapped_column(Integer, index=True)
    tipo: Mapped[str] = mapped_column(String(20))
    texto: Mapped[str] = mapped_column(Text)
```

```
    carga_horaria: Mapped[int] = mapped_column(SmallInteger)
    emitido_em: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())
```

Fonte: código do projeto (2026).

6.3.10 Notifications Service — porta 8008 (opcional, VP Canais)

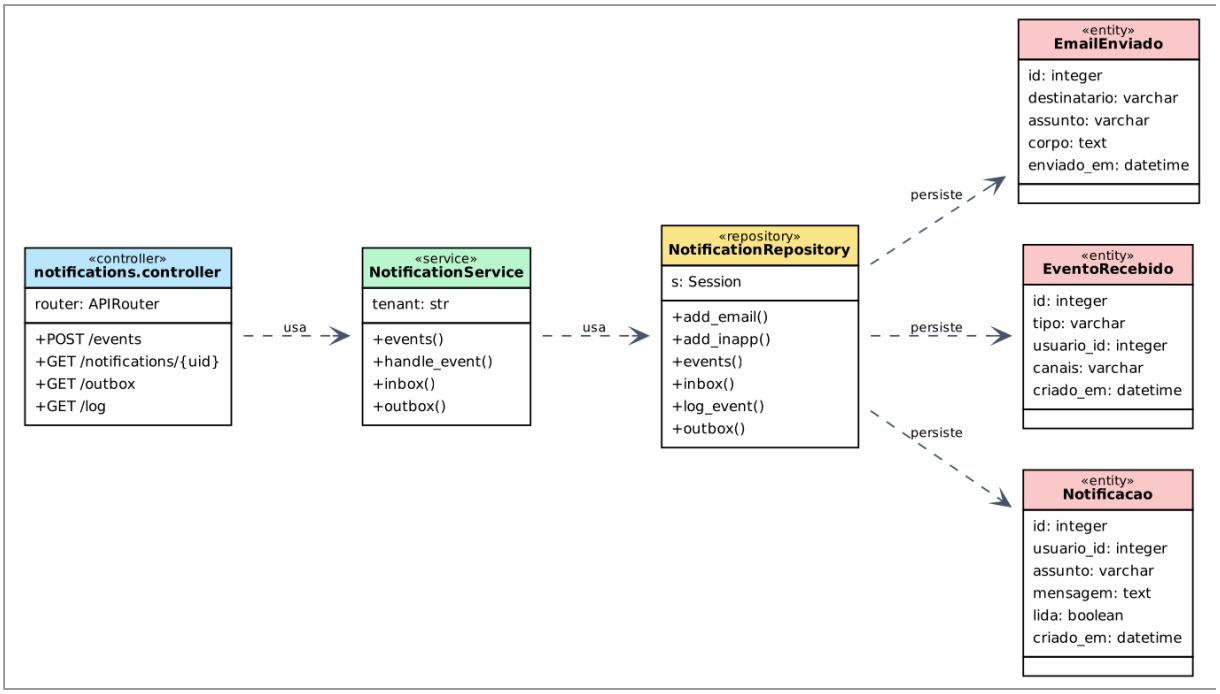
Feature **Notificações**: recebe os eventos de domínio publicados pelos outros serviços (coreografia) e os entrega pelos canais selecionados (`EmailChannel` simulado e/ou `InAppChannel`). Banco `lps_<t>_notifications`: `evento_recebido`, `notificacao`, `email_enviado`.

Quadro 23 – Endpoints do Notifications Service, acessados pelo API Gateway (http://127.0.0.1:8080)

Método	URL	Acesso	Descrição
`POST`	`/api/notifications/events`	JWT	Recebe evento de domínio e notifica pelos canais ativos (interno)
`GET`	`/api/notifications/notifications/{uid}`	JWT	Caixa de entrada in-app
`GET`	`/api/notifications/outbox`	JWT	E-mails simulados enviados (organizador)
`GET`	`/api/notifications/log`	JWT	Últimos eventos processados (organizador)

Fonte: elaborado pelas autoras (2026).

Figura 24 – Diagrama de classes — Notifications Service



Fonte: elaborado pelas autoras (2026).

Código 31 – Arquivo `services/notifications/controller.py`

```
"""Controller — API REST do Notifications Service."""
from fastapi import APIRouter, Depends
```

```
from common.service import Caller, current_user, tenant_header

from .dto import EmailOut, EventIn, NotificationOut
from .service import NotificationService

router = APIRouter()


def svc(tenant: str = Depends(tenant_header)) -> NotificationService:
    return NotificationService(tenant)


@router.post("/events", tags=["eventos"], summary="Recebe evento de domínio e notifica
pelos canais ativos (interno)")
def event(body: EventIn, s: NotificationService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_role("organizador")          # só chamadas internas (ou organizador)
publicam eventos
    return s.handle_event(body)


@router.get("/notifications/{uid}", response_model=list[NotificationOut], tags=
["notificações"], summary="Caixa de entrada in-app")
def inbox(uid: int, s: NotificationService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_self_or(uid, "organizador")
    return s.inbox(uid)


@router.get("/outbox", response_model=list[EmailOut], tags=["notificações"], summary="E-
mails simulados enviados (organizador)")
def outbox(s: NotificationService = Depends(svc), who: Caller = Depends(current_user)):
    who.ensure_role("organizador")
    return s.outbox()


@router.get("/log", tags=["notificações"], summary="Últimos eventos processados
(organizador)")
def log(s: NotificationService = Depends(svc), who: Caller = Depends(current_user)):
    who.ensure_role("organizador")
    return s.events()
```

Fonte: código do projeto (2026).

Código 32 – Arquivo components/notification.py

```
"""Ponto de variação **Canal de Notificação** (or): Email | InApp.

Grupo *or*: o produto pode combinar vários canais; a mensagem é entregue a
todos os componentes selecionados.
"""
from __future__ import annotations

from abc import ABC, abstractmethod
from typing import Callable

from .base import VariationPoint


class NotificationChannel(ABC):
    label = "?"

    def __init__(self, store: Callable | None = None, mail_store: Callable | None = None):
        self.store = store              # persiste notificação in-app (tabela notificacao)
        self.mail_store = mail_store    # persiste e-mail simulado (tabela email_enviado)

    @abstractmethod
    def send(self, user: dict, subject: str, message: str) -> dict: ...
```

```
channel_vp: VariationPoint[NotificationChannel] = VariationPoint("notification_channels",
NotificationChannel)


@channel_vp.variant("Email")
class EmailChannel(NotificationChannel):
    label = "E-mail (simulado)"

    def send(self, user, subject, message):
        # Envio SIMULADO: o e-mail é gravado na caixa de saída (banco), nunca enviado de
fato.
        self.mail_store(user.get("email") or f"usuario-{user.get('id')}@desconhecido",
subject, message)
        return {"channel": "Email", "to": user.get("email"), "delivered": True}


@channel_vp.variant("InApp")
class InAppChannel(NotificationChannel):
    label = "No aplicativo"

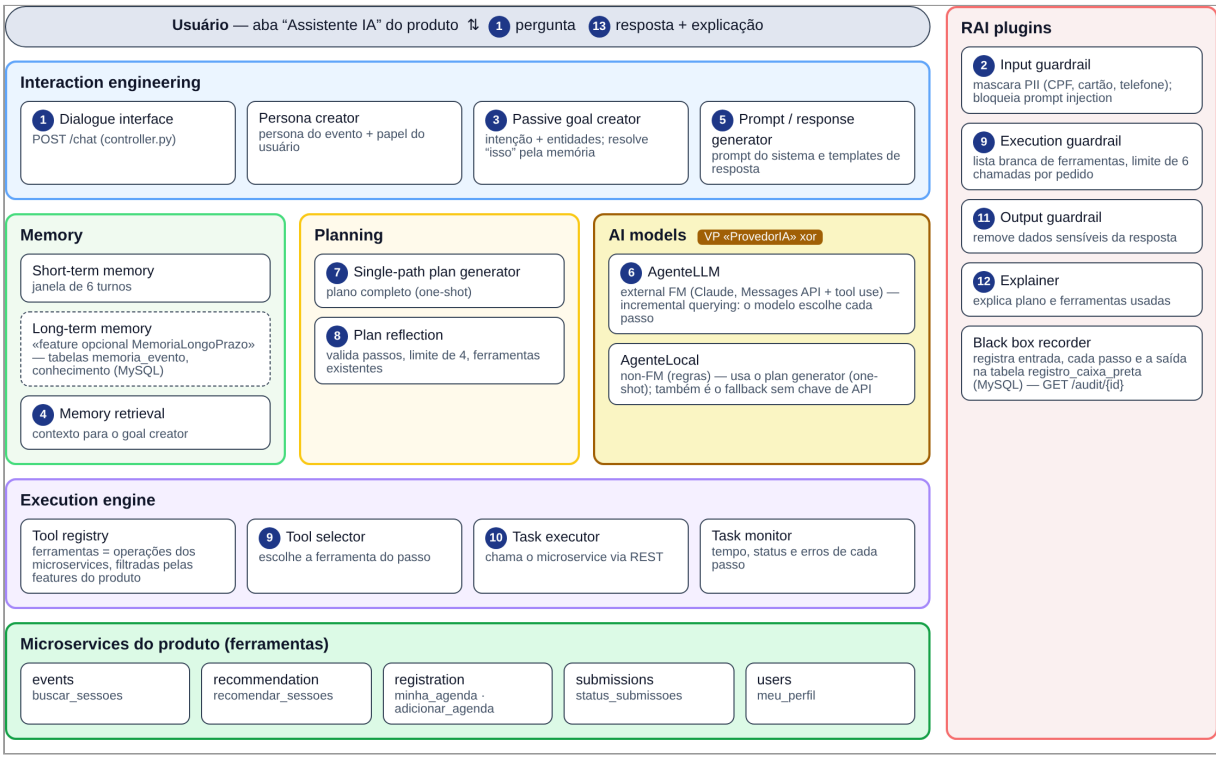
    def send(self, user, subject, message):
        self.store(user["id"], subject, message)
        return {"channel": "InApp", "delivered": True}
```

Fonte: código do projeto (2026).

6.3.11 AI Agent Service — porta 8009 (opcional, VP Provedor e Memória)

Feature **Assistente IA**. O microservice foi organizado segundo a **arquitetura de referência para agentes baseados em *foundation models*** de Lu et al. (2024), que divide o agente em *interaction engineering*, *memory*, *planning*, *execution engine*, *RAI plugins* (guardrails, *black box recorder*, *explainer*) e *AI models*. Os números indicam a ordem do fluxo de uma pergunta:

Figura 25 – Arquitetura do agente mapeada na arquitetura de referência de Lu et al. (2024)



Fonte: elaborado pelas autoras (2026).

Quadro 24 – Mapeamento do agente na arquitetura de referência de Lu et al. (2024)

Componente da arquitetura de referência	**Implementação (arquivo / classe)**	**Observação**
Dialogue interface	`controller.py` — `POST /chat`	entrada e saída do agente
Persona creator	`interaction.py` — `PersonaCreator`	persona do assistente do evento ativo
Passive goal creator	`interaction.py` — `PassiveGoalCreator`	intenção + entidades; resolve “isso” com a memória
Prompt / response generator	`interaction.py` — `PromptResponseGenerator`	prompt do sistema e templates de resposta
Short-term memory	`memory.py` — `ShortTermMemory`	janela dos últimos 6 turnos
Long-term memory	`memory.py` — `LongTermMemory` + tabelas `memoria_evento`, `conhecimento`	**feature opcional** MemoriaLongoPrazo
Memory retrieval	`LongTermMemory.retrieve()`	contexto para o goal creator
Single-path plan generator	`planning.py` — `SinglePathPlanGenerator`	plano completo de uma vez (*one-shot*)
Plan reflection	`planning.py` — `PlanReflection`	remove passos inválidos, limita a 4 passos
Tool registry / Tool selector	`execution.py` — `ToolRegistry`, `ToolSelector`	ferramentas = operações dos microservices **do produto**

Componente da arquitetura de referência	Implementação (arquivo / classe)	Observação
Task executor / Task monitor	`execution.py` — `TaskExecutor`, `TaskMonitor`	chamada REST + tempo e status de cada passo
Input / Execution / Output guardrails	`rai.py`	PII, *prompt injection*, lista branca, limite de chamadas
Black box recorder	`rai.py` — `BlackBoxRecorder` + tabela `registro_caixa_preta`	auditoria em `GET /audit/{request_id}`
Explainer	`rai.py` — `Explainer`	explica plano, ferramentas e memória usadas
AI models (external FM / non-FM)	`components/agent.py` — `LLMAgent`, `LocalRuleAgent`	**ponto de variação xor** `ai_provider`

Fonte: elaborado pelas autoras (2026).

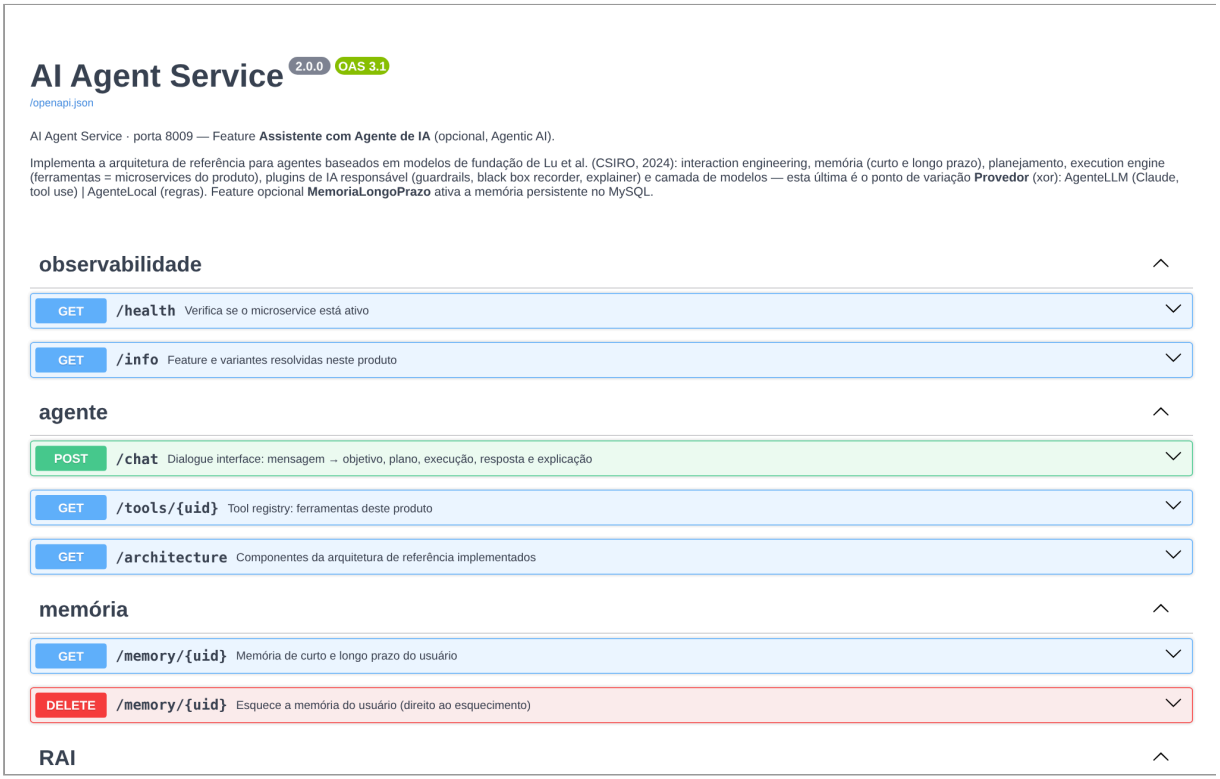
A variabilidade da LPS aparece em três lugares: o **provedor** (`AgenteLLM` usa o Claude com *tool use* e consulta incremental; `AgenteLocal` usa regras e plano *one-shot*; sem chave de API, o LLM cai para o local); a **memória de longo prazo** (feature opcional); e as **ferramentas**, que dependem das features do produto (sem Submissão, a ferramenta `status_submissoes` não existe). Banco `lps_<t>_ai_agent`: `memoria_evento`, `conhecimento`, `registro_caixa_preta`.

Quadro 25 – Endpoints do AI Agent Service, acessados pelo API Gateway (http://127.0.0.1:8080)

Método	URL	Acesso	Descrição
`POST`	`/api/ai_agent/chat`	JWT	Dialogue interface: mensagem → objetivo, plano, execução, resposta e explicação
`GET`	`/api/ai_agent/tools/{uid}`	JWT	Tool registry: ferramentas deste produto
`GET`	`/api/ai_agent/architecture`	pública	Componentes da arquitetura de referência implementados
`GET`	`/api/ai_agent/memory/{uid}`	JWT	Memória de curto e longo prazo do usuário
`DELETE`	`/api/ai_agent/memory/{uid}`	JWT	Esquece a memória do usuário (direito ao esquecimento)
`GET`	`/api/ai_agent/audit/{request_id}`	JWT	Black box recorder: registro de cada componente para uma requisição

Fonte: elaborado pelas autoras (2026).

Figura 26 – Swagger do AI Agent Service



Fonte: elaborado pelas autoras (2026).

Código 33 – Arquivo `services/ai_agent/controller.py`

```
"""Controller — interface de diálogo e auditoria do AI Agent Service."""
from fastapi import APIRouter, Depends, HTTPException

from common.service import Caller, current_user, tenant_header

from .dto import AuditOut, ChatIn, ChatOut, ToolOut
from .service import ARCHITECTURE, PROVIDER, AgentService

router = APIRouter()


def svc(tenant: str = Depends(tenant_header)) -> AgentService:
    return AgentService(tenant)


@router.post("/chat", response_model=ChatOut, tags=["agente"],
             summary="Dialogue interface: mensagem → objetivo, plano, execução, resposta e
explicação")
def chat(body: ChatIn, s: AgentService = Depends(svc), who: Caller =
Depends(current_user)):
    who.ensure_self_or(body.user_id)
    try:
        return s.chat(body.user_id, body.message)
    except Exception as exc:
        raise HTTPException(502, f"Falha no agente: {exc}")


@router.get("/tools/{uid}", response_model=list[ToolOut], tags=["agente"], summary="Tool
registry: ferramentas deste produto")
def tools(uid: int, s: AgentService = Depends(svc)):
    return s.tools(uid)


@router.get("/architecture", tags=["agente"], summary="Componentes da arquitetura de
```

```
referência implementados")
def architecture():
    return {"reference": "Lu et al., Towards Responsible Generative AI: A Reference
Architecture for Designing "
                          "Foundation Model based Agents (2024)", "provider":
PROVIDER.label,
            "querying": PROVIDER.querying, "components": ARCHITECTURE}


@router.get("/memory/{uid}", tags=["memória"], summary="Memória de curto e longo prazo do
usuário")
def memory(uid: int, s: AgentService = Depends(svc), who: Caller = Depends(current_user)):
    who.ensure_self_or(uid, "organizador")
    return s.memory(uid)


@router.delete("/memory/{uid}", tags=["memória"], summary="Esquece a memória do usuário
(direito ao esquecimento)")
def forget(uid: int, s: AgentService = Depends(svc), who: Caller = Depends(current_user)):
    who.ensure_self_or(uid, "organizador")
    return s.forget(uid)


@router.get("/audit/{request_id}", response_model=list[AuditOut], tags=["RAI"],
            summary="Black box recorder: registro de cada componente para uma requisição")
def audit(request_id: str, s: AgentService = Depends(svc)):
    return s.audit(request_id)
```

Fonte: código do projeto (2026).

Código 34 – Arquivo `services/ai_agent/service.py`

```
"""Service — orquestra o agente segundo a arquitetura de referência de Lu et al. (2024).

Fluxo de uma mensagem na interface de diálogo::

  input guardrail → persona creator → memory retrieval (curto + longo prazo)
  → passive goal creator → prompt generator → provedor (variante LLM ou Local):
        planning (single-path plan generator + plan reflection)
        execution engine (tool selector → execution guardrail → task executor + monitor)
  → response generator → output guardrail → explainer
  → atualização da memória → black box recorder (persistido no MySQL)
"""
from __future__ import annotations

from common.client import call
from common.config import component, has_feature
from components.agent import AgentContext, agent_vp

from .execution import TaskExecutor, TaskMonitor, ToolRegistry, ToolSelector
from .interaction import PassiveGoalCreator, PersonaCreator, PromptResponseGenerator
from .memory import LongTermMemory, ShortTermMemory
from .planning import PlanReflection, SinglePathPlanGenerator
from .rai import BlackBoxRecorder, ExecutionGuardrail, Explainer, InputGuardrail,
OutputGuardrail

PROVIDER = agent_vp.create(component("ai_provider", "AgenteLocal"))

ARCHITECTURE = {
    "interaction engineering": ["dialogue interface (POST /chat)", "persona creator",
"passive goal creator",
                                "prompt/response generator (templates)"],
    "memory": ["short-term memory (janela da conversa)", "long-term memory (MySQL) —
feature MemoriaLongoPrazo",
               "memory retrieval"],
    "planning": ["single-path plan generator", "plan reflection (self-reflection)",
                 "one-shot (AgenteLocal) / incremental model querying (AgenteLLM)"],
    "execution engine": ["tool registry (operações dos microservices do produto)", "tool
selector", "task executor",
```

```
                        "task monitor"],
    "RAI plugins": ["input guardrail (PII, prompt injection)", "execution guardrail (lista
branca, limite)",
                    "output guardrail", "black box recorder (MySQL)", "explainer"],
    "AI models": ["AgenteLLM: external FM (Claude) — variante", "AgenteLocal: non-FM model
(regras) — variante"],
}


def _facts(goal, tasks: list[dict]) -> dict[str, str]:
    """O que vale guardar como conhecimento de longo prazo após a interação."""
    f: dict[str, str] = {}
    if goal.intent == "buscar" and goal.entities.get("query"):
        f["ultimo_tema"] = goal.entities["query"]
    for t in tasks:
        if t["tool"] == "recomendar_sessoes" and t["output"]:
            f["ultima_recomendacao"] = t["output"][0]["title"]
        if t["tool"] == "adicionar_agenda" and isinstance(t["output"], dict) and
t["output"].get("ok"):
            f["ultima_sessao_adicionada"] = t["output"]["session"]["title"]
    return f


class AgentService:
    def __init__(self, tenant: str):
        self.tenant = tenant

    def tools(self, uid: int) -> list[dict]:
        return [{"name": t.name, "description": t.description, "feature": t.feature,
"service": t.service}
                for t in ToolRegistry(self.tenant, uid).build().values()]

    def chat(self, uid: int, message: str) -> dict:
        recorder = BlackBoxRecorder(self.tenant, uid)
        notes: list[str] = []

        # 1. RAI — guardrail de entrada
        text, n, allowed = InputGuardrail().apply(message)
        notes += n
        recorder.record("input guardrail", message, {"text": text, "notes": n, "allowed":
allowed})
        if not allowed:
            answer = "Não posso seguir instruções que alterem minhas regras. Posso ajudar
com a programação do evento."
            recorder.record("dialogue interface", text, answer)
            recorder.flush()
            return {"request_id": recorder.request_id, "answer": answer, "provider":
PROVIDER.label, "goal": {},
                    "plan": [], "trace": [], "explanation": [f"Guardrail: {x}" for x in
notes], "guardrails": notes,
                    "memory": {}}

        # 2. Interaction engineering + memória
        user = call("users", "GET", f"/users/{uid}", self.tenant) or {"id": uid, "name":
"?", "interests": []}
        event = call("events", "GET", "/event", self.tenant, default={"name": self.tenant})
        registry = ToolRegistry(self.tenant, uid).build()
        persona = PersonaCreator().create(event, registry)
        short, long_ = ShortTermMemory(self.tenant, uid), LongTermMemory(self.tenant, uid)
        retrieved = long_.retrieve()
        memory = {"recent": retrieved["recent"] or short.recent(), "knowledge":
retrieved["knowledge"],
                  "long_term_enabled": has_feature("MemoriaLongoPrazo"),
"long_term_events": retrieved["events"]}
        recorder.record("memory retrieval", {"user": uid}, memory)
        goal = PassiveGoalCreator().create(text, memory)
        recorder.record("passive goal creator", text, goal.__dict__)
        gen = PromptResponseGenerator()
        prompt = gen.prompt(persona, user, memory, goal)
        recorder.record("prompt generator", goal.__dict__, prompt)
```

```
        # 3. Planning + execution engine (via provedor = ponto de variação)
        monitor = TaskMonitor()
        executor = TaskExecutor(registry, ToolSelector(), monitor,
ExecutionGuardrail(set(registry)), recorder)
        reflection = PlanReflection()

        def plan_generator(g, tools, mem):
            plan = SinglePathPlanGenerator().generate(g, tools, mem)
            kept, refl = reflection.reflect(plan, tools)
            notes.extend(refl)
            recorder.record("single-path plan generator + plan reflection", g.__dict__,
                            {"plan": [p.__dict__ for p in kept], "reflection": refl})
            return kept

        ctx = AgentContext(goal=goal, persona=persona, prompt=prompt, memory=memory,
tools=registry,
                           execute=executor, plan_generator=plan_generator,
respond=gen.respond)
        try:
            result = PROVIDER.solve(ctx)
        except PermissionError as exc:
            result = {"answer": f"Ação bloqueada pelo guardrail: {exc}", "plan": [],
"model": PROVIDER.label}

        # 4. RAI — guardrail de saída + explainer
        answer, out_notes = OutputGuardrail().apply(result["answer"] or "")
        notes += out_notes
        explanation = Explainer().explain(goal, result["plan"], monitor.tasks, memory,
result["model"], notes)
        recorder.record("output guardrail", result["answer"], {"answer": answer, "notes":
out_notes})
        recorder.record("explainer", {"plan": result["plan"]}, explanation)

        # 5. Atualiza memória e grava a caixa-preta
        short.add("usuario", text)
        short.add("agente", answer)
        facts = _facts(goal, monitor.tasks)
        long_.store(text, answer, facts)
        recorder.record("memory update", {"facts": facts}, {"long_term":
has_feature("MemoriaLongoPrazo")})
        recorder.flush()

        return {"request_id": recorder.request_id, "answer": answer, "provider":
result["model"], "goal": goal.__dict__,
                "plan": result["plan"], "trace": monitor.tasks, "explanation": explanation,
"guardrails": notes,
                "memory": {"knowledge": memory["knowledge"], "long_term_enabled":
memory["long_term_enabled"],
                           "recent_messages": len(memory["recent"]), "facts_saved": facts}}

    def memory(self, uid: int) -> dict:
        return {"short_term": ShortTermMemory(self.tenant, uid).recent(),
**LongTermMemory(self.tenant, uid).retrieve(),
                "long_term_enabled": has_feature("MemoriaLongoPrazo")}

    def forget(self, uid: int) -> dict:
        LongTermMemory(self.tenant, uid).forget()
        return {"ok": True}

    def audit(self, request_id: str) -> list[dict]:
        return BlackBoxRecorder.read(self.tenant, request_id)
```

Fonte: código do projeto (2026).

Código 35 – Arquivo `services/ai_agent/rai.py`

```
"""Responsible AI plugins (Lu et al., 2024, seção 3.5).
```

```
* **Guardrails** de entrada (mascara dados pessoais e bloqueia tentativas de
  *prompt injection*), de execução (lista branca de ferramentas, limite de passos)
  e de saída (remove dados pessoais, limita tamanho).
* **Black box recorder** — registra entrada e saída de cada componente, com
  data e hora, no banco do serviço (``registro_caixa_preta``), para auditoria.
* **Explainer** — explica ao usuário o objetivo entendido, o plano, as
  ferramentas (microservices) usadas e o que veio da memória.
"""
from __future__ import annotations

import json
import re
import uuid

from common.database import session_scope

from .model import Base
from .repository import BlackBoxRepository

PII = [
    (re.compile(r"\b\d{3}\.?\d{3}\.?\d{3}-?\d{2}\b"), "[CPF removido]"),
    (re.compile(r"\b(?:\d[ -]?){13,19}\b"), "[número de cartão removido]"),
    (re.compile(r"\(?\b\d{2}\)?\s?9?\d{4}-?\d{4}\b"), "[telefone removido]"),
]
INJECTION = re.compile(r"(ignore|desconsidere|esqueça).{0,25}
(instru|regras|anteriores)|system prompt|prompt do sistema|"
                       r"modo desenvolvedor|jailbreak", re.I)
MAX_INPUT, MAX_OUTPUT = 1000, 2500


class InputGuardrail:
    def apply(self, text: str) -> tuple[str, list[str], bool]:
        notes = []
        if INJECTION.search(text):
            return text, ["entrada bloqueada: tentativa de alterar as instruções do
agente"], False
        clean = text[:MAX_INPUT]
        if len(text) > MAX_INPUT:
            notes.append(f"entrada truncada para {MAX_INPUT} caracteres")
        for rx, repl in PII:
            if rx.search(clean):
                clean = rx.sub(repl, clean)
                notes.append(f"dado pessoal mascarado na entrada: {repl}")
        return clean, notes, True


class ExecutionGuardrail:
    MAX_CALLS = 6

    def __init__(self, allowed: set[str]):
        self.allowed = allowed

    def check(self, tool, args: dict, n_calls: int) -> tuple[bool, str]:
        if tool.name not in self.allowed:
            return False, f"ferramenta '{tool.name}' fora da lista branca"
        if n_calls > self.MAX_CALLS:
            return False, "limite de chamadas de ferramentas excedido"
        return True, ""


class OutputGuardrail:
    def apply(self, text: str) -> tuple[str, list[str]]:
        notes = []
        for rx, repl in PII:
            if rx.search(text):
                text = rx.sub(repl, text)
                notes.append("dado pessoal removido da resposta")
        if len(text) > MAX_OUTPUT:
            text = text[:MAX_OUTPUT] + "…"
            notes.append("resposta truncada")
```

```
        return text, notes


class BlackBoxRecorder:
    def __init__(self, tenant: str, user_id: int):
        self.tenant, self.user_id = tenant, user_id
        self.request_id = uuid.uuid4().hex
        self.records: list[dict] = []

    def record(self, component: str, inp, out) -> None:
        dump = lambda x: x if isinstance(x, str) else json.dumps(x, ensure_ascii=False,
default=str)
        self.records.append({"order": len(self.records) + 1, "component": component,
"input": dump(inp), "output": dump(out)})

    def flush(self) -> None:
        with session_scope("ai_agent", self.tenant, Base) as s:
            BlackBoxRepository(s).save(self.request_id, self.user_id, self.records)

    @staticmethod
    def read(tenant: str, request_id: str) -> list[dict]:
        with session_scope("ai_agent", tenant, Base) as s:
            return [{"order": r.ordem, "component": r.componente, "input": r.entrada,
"output": r.saida,
                     "at": r.criado_em.isoformat(sep=" ", timespec="seconds") if
r.criado_em else ""}
                    for r in BlackBoxRepository(s).by_request(request_id)]


class Explainer:
    def explain(self, goal, plan: list[dict], tasks: list[dict], memory: dict, model: str,
notes: list[str]) -> list[str]:
        out = [f"Objetivo entendido: {goal.intent}" + (f" {goal.entities}" if goal.entities
else "") + "."]
        if plan:
            out.append("Plano: " + " → ".join(f"{i + 1}) {p['tool']}" for i, p in
enumerate(plan)) + ".")
        for t in tasks:
            out.append(f"Passo {t['step']}: {t['tool']} no {t['service']}-service —
{t['status']}.")
        if memory.get("knowledge"):
            out.append("Memória de longo prazo consultada: " + ",
".join(memory["knowledge"]) + ".")
        out.append(f"Modelo: {model}.")
        out += [f"Guardrail: {n}" for n in notes]
        return out
```

Fonte: código do projeto (2026).

Código 36 – Arquivo `components/agent.py`

```
"""Ponto de variação **Provedor do Agente de IA** (xor): AgenteLLM | AgenteLocal.

Na arquitetura de referência de agentes de Lu et al. (CSIRO/Data61, 2024), este
componente corresponde à camada **AI models** e à forma de **planejamento**:

* ``AgenteLLM``   — *external FM* (Claude, Messages API com tool use) com
  *incremental model querying*: o modelo é consultado a cada passo e decide a
  próxima ferramenta a chamar.
* ``AgenteLocal`` — *non-FM model* (regras): usa o *single-path plan generator*
  determinístico do serviço, que gera o plano inteiro de uma vez
  (*one-shot*) e o executa passo a passo.

Os dois recebem o mesmo :class:`AgentContext`, ou seja, as mesmas ferramentas
(operações dos microservices do produto), a mesma memória e os mesmos
guardrails. Isso torna o provedor intercambiável, como em toda variante de LPS.
"""
from __future__ import annotations
```

```
import json
import os
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from typing import Any, Callable

import httpx

from .base import VariationPoint


@dataclass
class Tool:
    name: str
    description: str
    parameters: dict                # JSON Schema
    fn: Callable[..., object]
    feature: str                    # feature de que a ferramenta depende
    service: str                    # microservice que a ferramenta chama


@dataclass
class Goal:
    """Saída do *passive goal creator*: intenção + parâmetros extraídos do pedido."""
    text: str
    intent: str
    entities: dict = field(default_factory=dict)


@dataclass
class Step:
    tool: str
    args: dict
    why: str


@dataclass
class AgentContext:
    """Tudo o que o provedor pode usar — montado pelo serviço do agente."""
    goal: Goal
    persona: dict
    prompt: str                                 # gerado pelo prompt/response generator
    memory: dict                                # visão da memória (curta + longa)
    tools: dict[str, Tool]                      # registro de ferramentas já filtrado pelo
produto
    execute: Callable[[str, dict, str], Any]    # executor (guardrail + monitor + caixa-
preta)
    plan_generator: Callable[[Goal, dict, dict], list[Step]]
    respond: Callable[[Goal, list[dict], dict], str]   # gerador de resposta (templates)


class AgentProvider(ABC):
    label = "?"
    querying = "?"

    @abstractmethod
    def solve(self, ctx: AgentContext) -> dict:
        """Retorna {answer, plan: [{tool, args, why}], model}."""


agent_vp: VariationPoint[AgentProvider] = VariationPoint("ai_provider", AgentProvider)


@agent_vp.variant("AgenteLocal")
class LocalRuleAgent(AgentProvider):
    """Non-FM model: plano de caminho único gerado por regras e executado em sequência."""
    label = "Agente local (regras)"
    querying = "one-shot (plano gerado de uma vez)"

    def solve(self, ctx: AgentContext) -> dict:
```

```
        plan = ctx.plan_generator(ctx.goal, ctx.tools, ctx.memory)
        results = []
        for i, step in enumerate(plan):
            args = _resolve(step.args, results)
            out = ctx.execute(step.tool, args, step.why)
            results.append({"step": i + 1, "tool": step.tool, "args": args, "output": out})
        answer = ctx.respond(ctx.goal, results, ctx.memory)
        return {"answer": answer, "plan": [s.__dict__ for s in plan], "model": self.label}


@agent_vp.variant("AgenteLLM")
class LLMAgent(AgentProvider):
    """External FM (Claude): consulta incremental com tool use. Sem chave, cai para o
agente local."""
    label = "Agente LLM (Claude)"
    querying = "incremental (o modelo decide cada passo)"
    API = "https://api.anthropic.com/v1/messages"
    MAX_STEPS = 6

    def solve(self, ctx: AgentContext) -> dict:
        key = os.environ.get("ANTHROPIC_API_KEY")
        if not key:
            res = LocalRuleAgent().solve(ctx)
            res["model"] = self.label + " — sem ANTHROPIC_API_KEY, fallback para o agente
local"
            return res
        model = os.environ.get("ANTHROPIC_MODEL", "claude-sonnet-4-5")
        tool_specs = [{"name": t.name, "description": t.description, "input_schema":
t.parameters} for t in ctx.tools.values()]
        messages = [{"role": "user", "content": ctx.goal.text}]
        headers = {"x-api-key": key, "anthropic-version": "2023-06-01", "content-type":
"application/json"}
        plan: list[dict] = []
        for _ in range(self.MAX_STEPS):
            r = httpx.post(self.API, headers=headers, timeout=60, json={
                "model": model, "max_tokens": 1024, "system": ctx.prompt, "tools":
tool_specs, "messages": messages})
            r.raise_for_status()
            data = r.json()
            messages.append({"role": "assistant", "content": data["content"]})
            calls = [b for b in data["content"] if b["type"] == "tool_use"]
            if data.get("stop_reason") != "tool_use" or not calls:
                text = "".join(b.get("text", "") for b in data["content"] if b["type"] ==
"text")
                return {"answer": text, "plan": plan, "model": f"{self.label} ({model})"}
            results = []
            for c in calls:
                args = c.get("input", {})
                plan.append({"tool": c["name"], "args": args, "why": "escolhido pelo
modelo"})
                try:
                    out = ctx.execute(c["name"], args, "escolhido pelo modelo")
                except Exception as exc:                 # erro (ou bloqueio de guardrail)
volta ao modelo
                    out = {"error": str(exc)}
                results.append({"type": "tool_result", "tool_use_id": c["id"],
                                "content": json.dumps(out, ensure_ascii=False, default=str)
[:6000]})
            messages.append({"role": "user", "content": results})
        return {"answer": "Limite de passos do agente atingido.", "plan": plan, "model":
self.label}


def _resolve(args: dict, results: list[dict]) -> dict:
    """Substitui referências a resultados anteriores, ex.: ``"$1.items[0].id"``."""
    out = {}
    for k, v in args.items():
        if isinstance(v, str) and v.startswith("$"):
            ref, _, path = v[1:].partition(".")
            val: Any = results[int(ref) - 1]["output"] if results else None
```

```
            for part in filter(None, path.replace("]", "").replace("[", ".").split(".")):
                val = val[int(part)] if part.isdigit() else (val or {}).get(part)
                if val is None:
                    break
            out[k] = val
        else:
            out[k] = v
    return out
```

Fonte: código do projeto (2026).

O código dos demais componentes do agente (`interaction.py`, `memory.py`, `planning.py`, `execution.py`) e os `dto.py`, `repository.py` e `service.py` de todos os serviços estão no GitHub.

6.4 Tela mostrando as portas de todos os microservices em execução

Página `http://127.0.0.1:8080/ports` do API Gateway. Os endereços e o último heartbeat vêm do Service Registry (produto SBES 2026: registry, gateway e 9 microservices):

Figura 27 – Portas de todos os microservices em execução (SBES 2026)



Fonte: elaborado pelas autoras (2026).

Código 37 – terminal — python -m spl.cli run sbes-2026

```
Produto: SBES 2026 — Simpósio Brasileiro de Engenharia de Software  (tenant: sbes-2026)
MICROSERVICE    PORTA   STATUS
```

```
registry        8761    UP
users           8001    UP
events          8002    UP
registration    8003    UP
recommendation  8004    UP
submissions     8005    UP
reviews         8006    UP
certificates    8007    UP
notifications   8008    UP
ai_agent        8009    UP
gateway         8080    UP

Banco de dados: 127.0.0.1:3306/{db}
Interface:  http://127.0.0.1:8080/
Registry:   http://127.0.0.1:8761/
Portas:     http://127.0.0.1:8080/ports
Swagger:    http://127.0.0.1:<porta>/docs

Ctrl+C para encerrar.
```

Fonte: elaborado pelas autoras (2026).

Para comparação, o produto Semana UFAL tem apenas os microservices das features selecionadas:

Figura 28 – Portas do produto Semana UFAL



Fonte: elaborado pelas autoras (2026).

6.5 Passo a passo para construir uma Aplicação (Produto) a partir da LPS

1. **Preparar o ambiente:** Python 3.10+ e `pip install -r requirements.txt`; MySQL (ou MariaDB) com o usuário `lps`/`lps123` com permissão para criar

bancos `lps_%` (o comando está no README). Alternativa sem servidor: `DATABASE_URL=sqlite:///data/{db}.db`.

2. **Conhecer o modelo de features:** `python -m spl.cli tree` lista features, grupos, restrições e o microservice de cada feature.
3. **Abrir a Tela de Configuração:** `python -m spl.cli configurator` → `http://127.0.0.1:7000`.
4. **Selecionar as features:** marcar as opcionais desejadas (Submissão, Revisão, Certificados, Notificações, Assistente IA, Memória de longo prazo, Trilhas), escolher uma variante em cada grupo xor (Pagamento, Estratégia de recomendação, Política de revisão, Provedor do agente) e uma ou mais em cada grupo or (Formatos, Tipos de certificado, Canais).
5. **Validar:** a tela mostra “Configuração válida” ou a lista de violações (ex.: Revisão requer Submissão).
6. **Nomear e gerar:** informar nome e cor do tema e clicar em **Gerar produto**. A derivação (`spl/derivation.py`) resolve feature → microservice (+ banco) e ponto de variação → variante, e grava `products/<id>.json` e `products/<id>.compose.yml`. Pela CLI: `python -m spl.cli derive "<Nome>" <features...>`.
7. **Executar: Executar produto** ou `python -m spl.cli run <id>`. O launcher sobe o Service Registry, cada microservice (que cria o seu banco MySQL e se registra) e o gateway, e imprime a tabela de portas. Com Docker: `docker compose -f products/<id>.compose.yml up --build` (sobe também o MySQL).
8. **Verificar:** abrir `http://127.0.0.1:8080/ports` e o painel `http://127.0.0.1:8761/` e conferir se todos estão UP.
9. **Usar o produto:** abrir `http://127.0.0.1:8080/` e entrar com uma conta de demonstração (senha `senha123`). A interface mostra só as abas das features selecionadas e, em cada painel, o microservice e a porta que o atendem.
10. **(Opcional) Agente LLM:** exportar `ANTHROPIC_API_KEY` antes do passo 7 se o produto usar a variante `AgenteLLM`.

PARTE 2 — IMPLEMENTAÇÃO

7 TECNOLOGIAS UTILIZADAS NA IMPLEMENTAÇÃO

Quadro 26 – Tecnologias utilizadas

Camada / ferramenta	**Tecnologia**
Linguagem	Python 3.10+ (testado em 3.13)
Framework dos microservices, gateway e registry	FastAPI + Uvicorn (ASGI); documentação OpenAPI/Swagger automática
Validação (DTOs)	Pydantic v2
Banco de dados	**MySQL 8** (Docker) / MariaDB 10.11; um banco por microservice e por tenant
Acesso a dados	SQLAlchemy 2 (ORM, padrão Repository) + PyMySQL
Segurança	JWT (PyJWT, HS256) e senhas com PBKDF2-SHA256
Descoberta de serviço	Service Registry próprio (papel do Eureka), com heartbeat
Comunicação entre serviços	REST/JSON com httpx + Circuit Breaker próprio; eventos de domínio
Frontend	HTML5 + CSS + JavaScript, servido pelo gateway
Agente de IA	Anthropic Messages API (tool use) ou agente local por regras
Contêineres	Docker (`Dockerfile`) + Docker Compose gerado por produto (com `mysql:8.0`)
Testes	pytest (13 testes); telas capturadas com Playwright
Diagramas	Graphviz, gerados a partir do modelo de features e dos modelos ORM
Versionamento	Git / GitHub

Fonte: elaborado pelas autoras (2026).

Linguagem e banco de dados de cada microservice:

Quadro 27 – Linguagem e banco de dados de cada microservice

Microservice	**Porta**	**Linguagem / framework**	**Banco de dados (MySQL)**	**Tabelas**
Service Registry	8761	Python / FastAPI	— (registro em memória)	—
API Gateway	8080	Python / FastAPI	—	—
Users Service	8001	Python / FastAPI	`lps_<t>_users`	usuario, interesse, historico_interacao
Events Service	8002	Python / FastAPI	`lps_<t>_events`	trilha, tag, sessao, sessao_tag
Registration Service	8003	Python / FastAPI	`lps_<t>_registration`	inscricao, pagamento, agenda_item
Recommendation Service	8004	Python / FastAPI	`lps_<t>_recommendation`	recomendacao

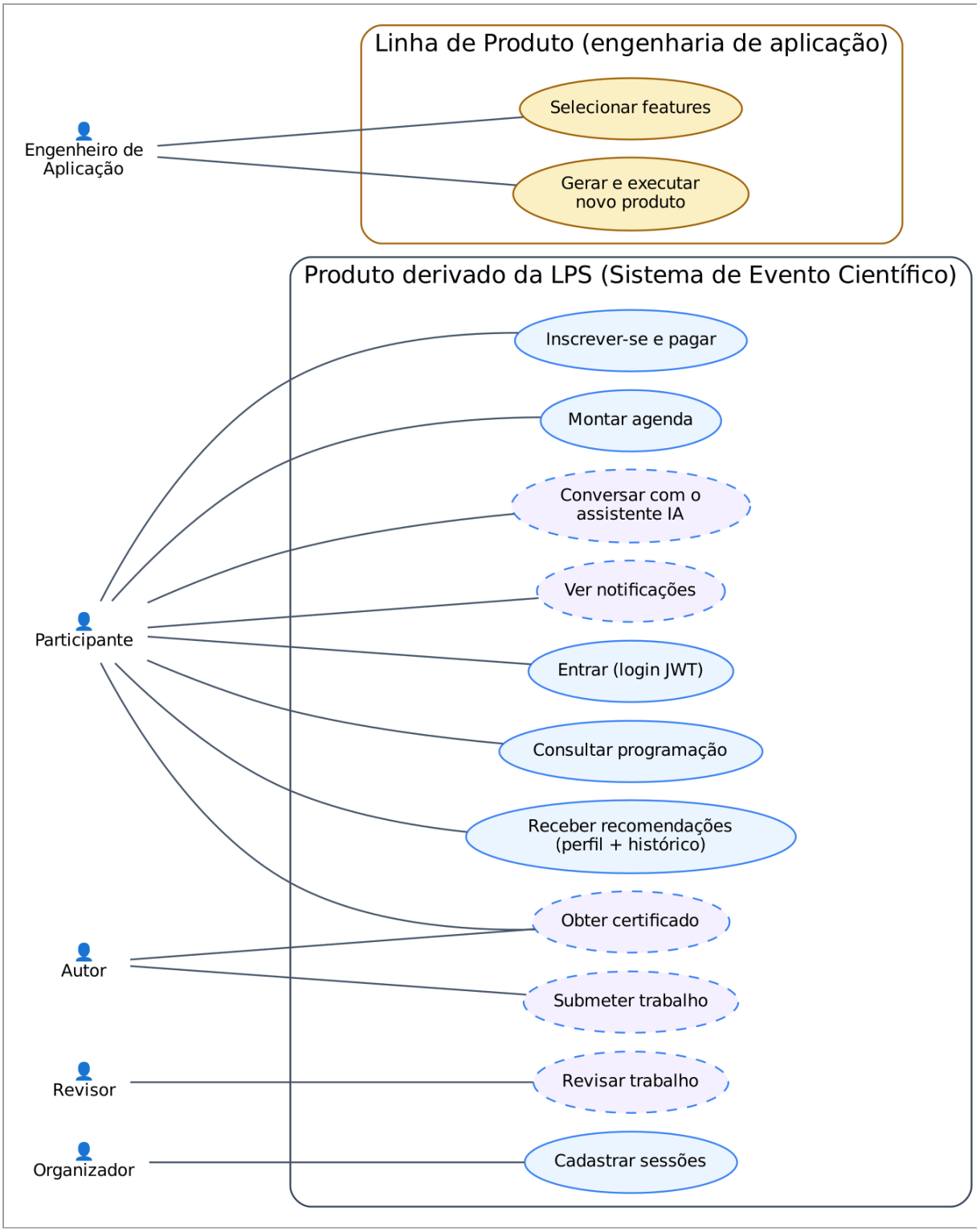
Microservice	Porta	Linguagem / framework	Banco de dados (MySQL)	Tabelas
Submissions Service	8005	Python / FastAPI	`lps_<t>_submissions`	submissao, coautor, palavra_chave
Reviews Service	8006	Python / FastAPI	`lps_<t>_reviews`	parecer
Certificates Service	8007	Python / FastAPI	`lps_<t>_certificates`	certificado
Notifications Service	8008	Python / FastAPI	`lps_<t>_notifications`	evento_recebido, notificacao, email_enviado
AI Agent Service	8009	Python / FastAPI	`lps_<t>_ai_agent`	memoria_evento, conhecimento, registro_caixa_preta
Tela de Configuração	7000	Python / FastAPI + HTML/JS	arquivos JSON (`feature_model.json`, `products/`)	—

Fonte: elaborado pelas autoras (2026).

8 TELAS DO FUNCIONAMENTO DO PRODUTO CONSTRUÍDO

Diagrama de casos de uso. Atores da engenharia de aplicação e do produto derivado (casos tracejados dependem de features opcionais):

Figura 29 – Diagrama de casos de uso

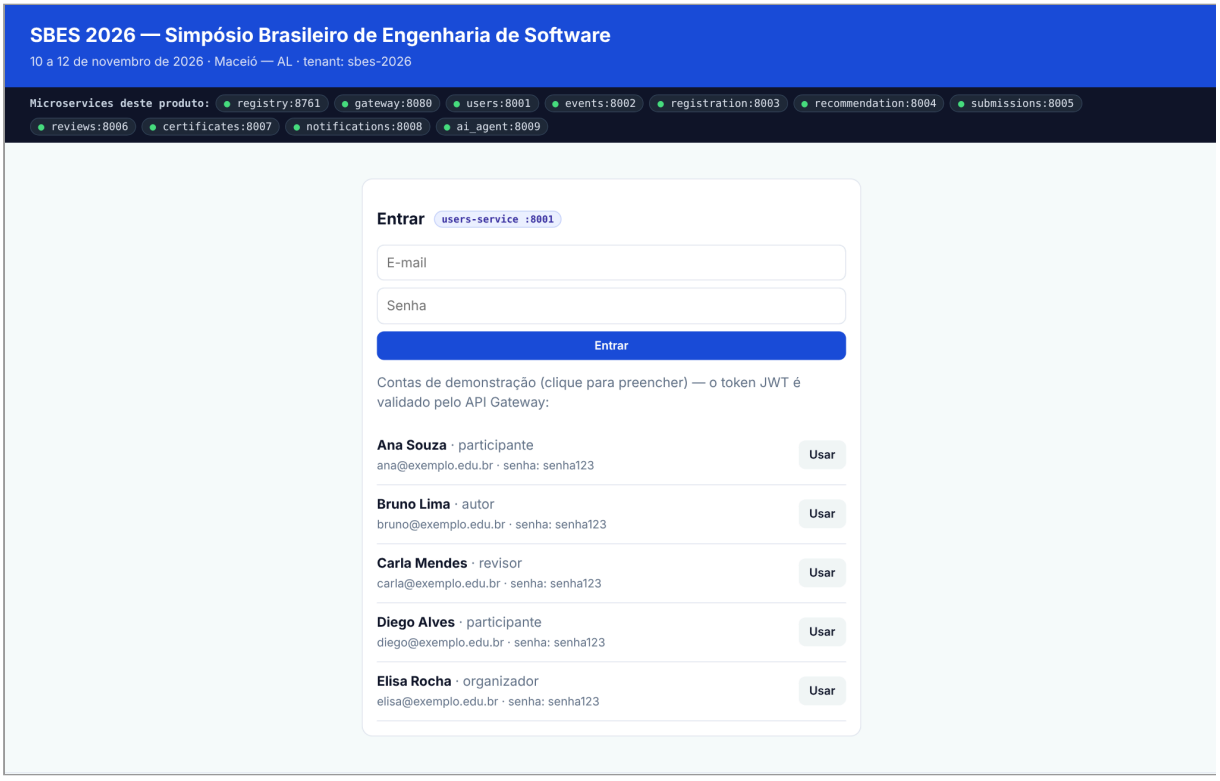


Fonte: elaborado pelas autoras (2026).

Produto demonstrado: **SBES 2026**, com 9 microservices. Na interface principal, a **faixa escura abaixo do cabeçalho mostra os microservices usados pelo produto** (com status), e cada painel exibe a etiqueta do microservice e da porta que o atende (ex.: `recommendation-service :8004`).

8.1 Login. Login com e-mail e senha (Users Service :8001). O token JWT retornado é enviado em todas as chamadas seguintes; sem ele, o gateway responde 401. As contas de demonstração podem ser preenchidas com o botão “Usar”.

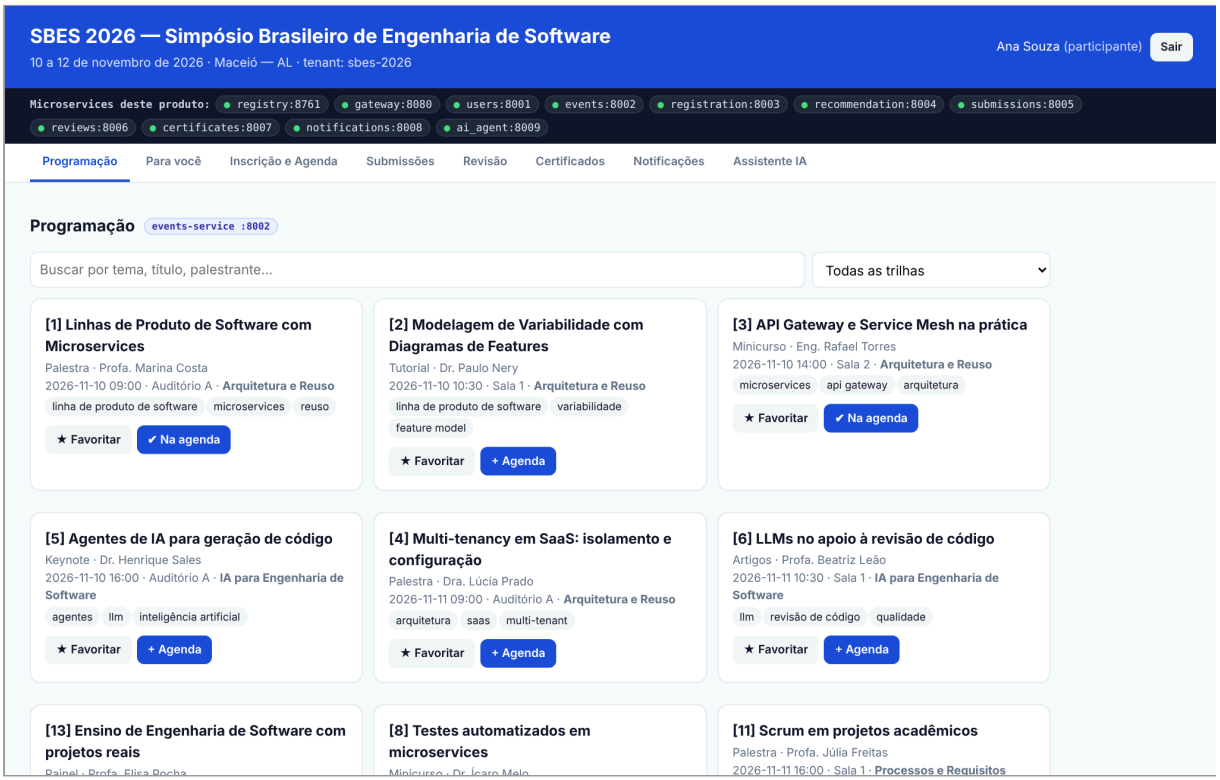
Figura 30 – Login — Users Service



Fonte: elaborado pelas autoras (2026).

8.2 Programação. Lista e busca das sessões, com filtro por trilha porque a feature Trilhas foi selecionada (Events Service :8002). Favoritar uma sessão registra o histórico no Users Service.

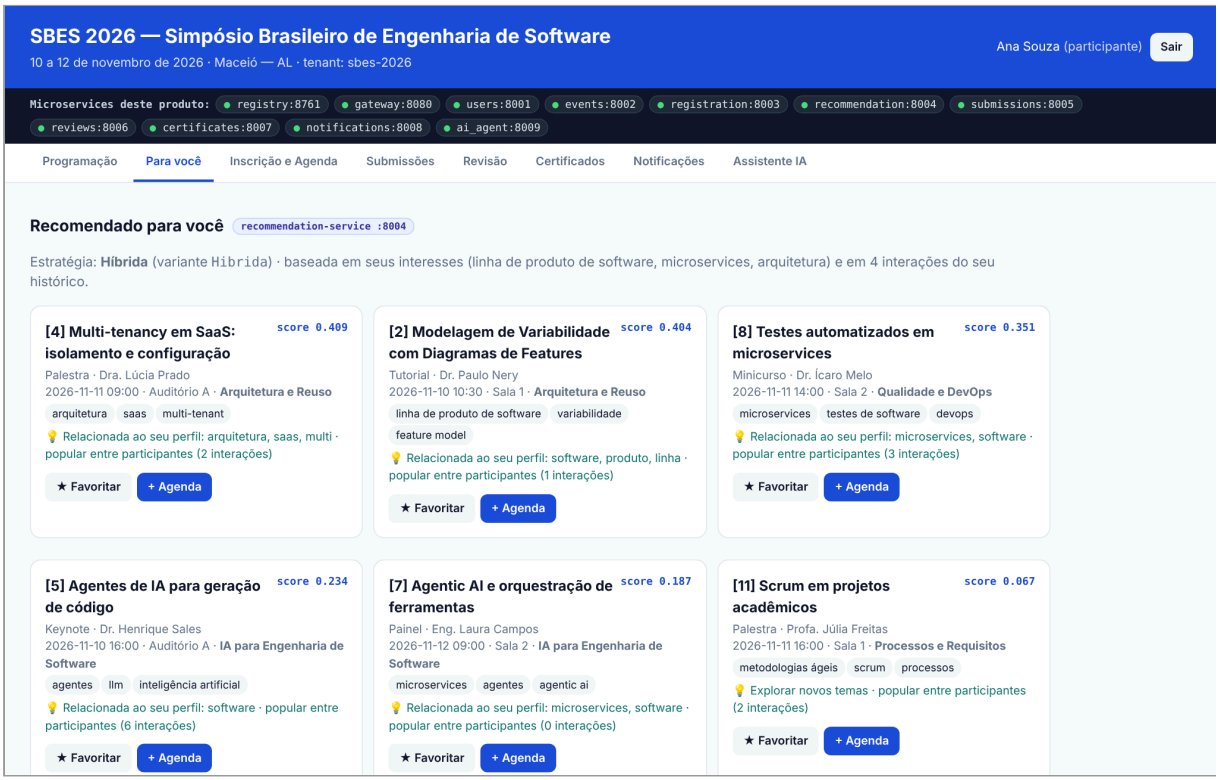
Figura 31 – Programação — Events Service



Fonte: elaborado pelas autoras (2026).

8.3 Recomendações (“Para você”). O Recommendation Service (:8004) usa a estratégia Híbrida sobre os interesses da usuária e o histórico dela; cada cartão mostra score e justificativa.

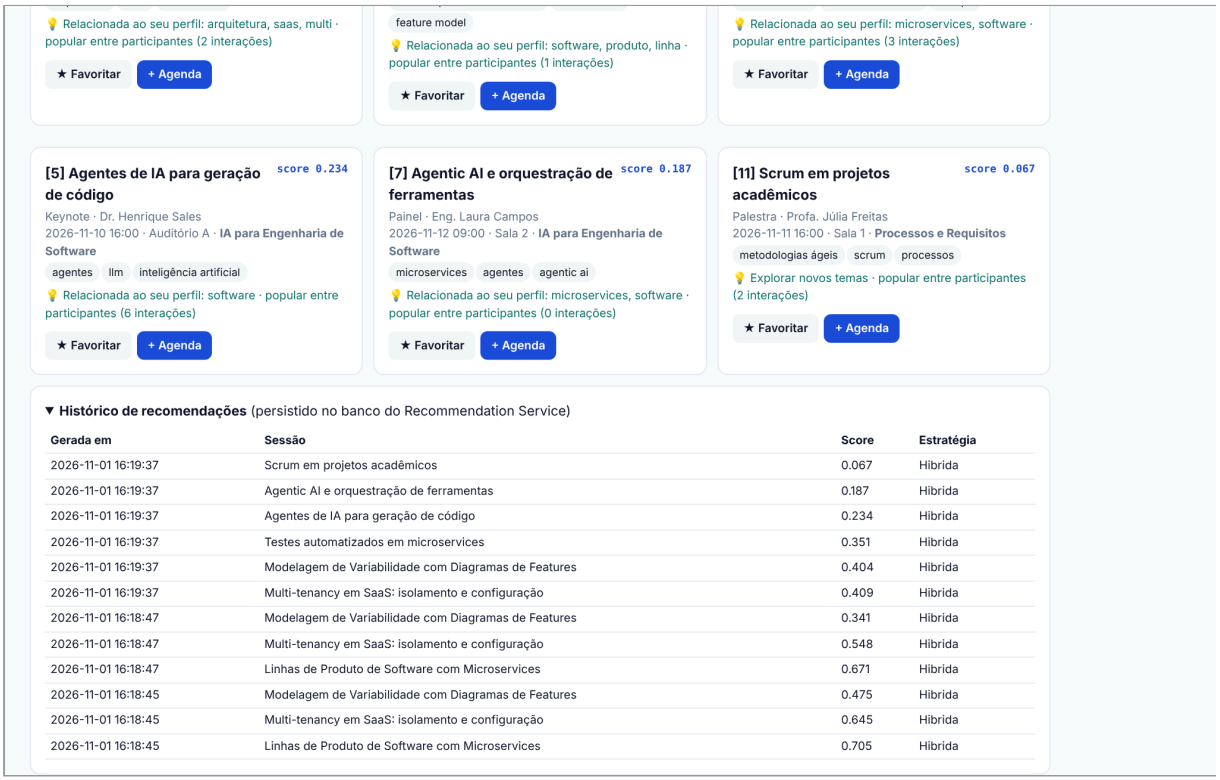
Figura 32 – Recomendações por perfil — Recommendation Service



Fonte: elaborado pelas autoras (2026).

Abaixo das recomendações, o histórico das recomendações geradas, lido da tabela `recomendacao` do MySQL:

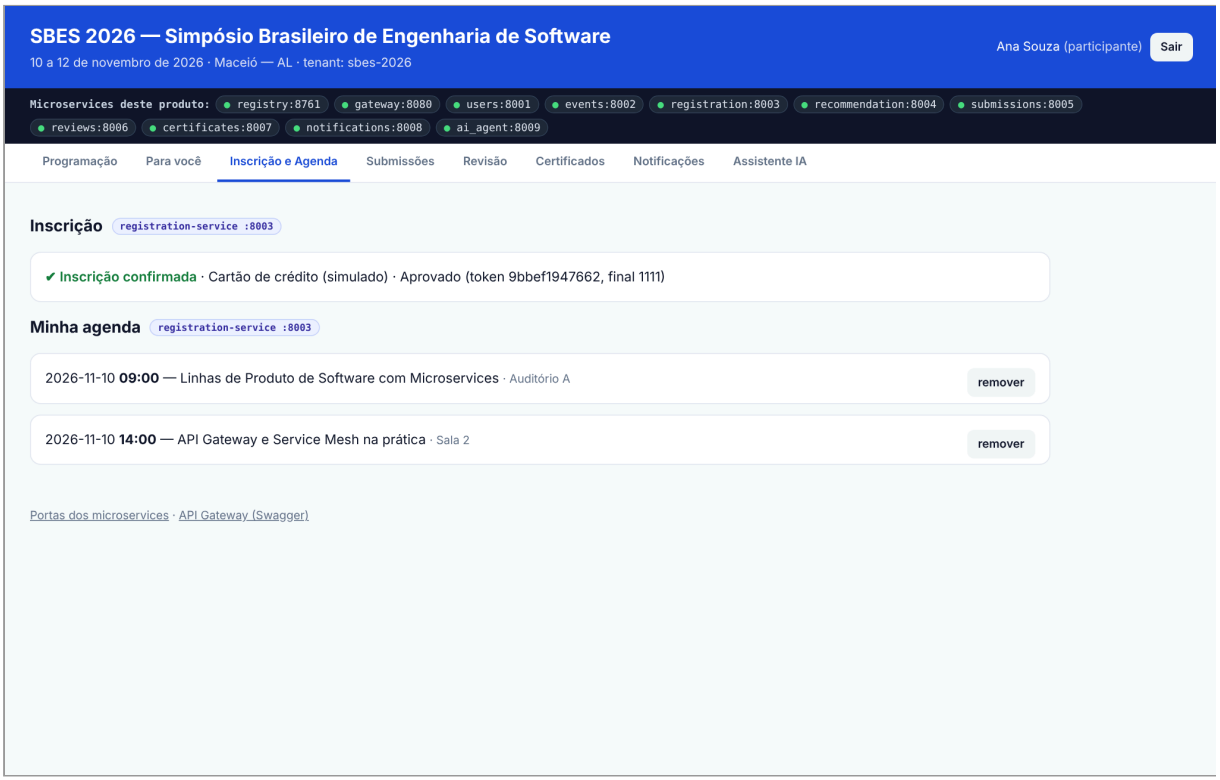
Figura 33 – Histórico de recomendações gravado no banco



Fonte: elaborado pelas autoras (2026).

8.4 Inscrição e Agenda. O Registration Service (:8003) usa a variante Cartão de crédito (simulado): o cartão de teste é aprovado e a inscrição é confirmada. A sessão recomendada foi adicionada à agenda, o que também gera histórico e notificação.

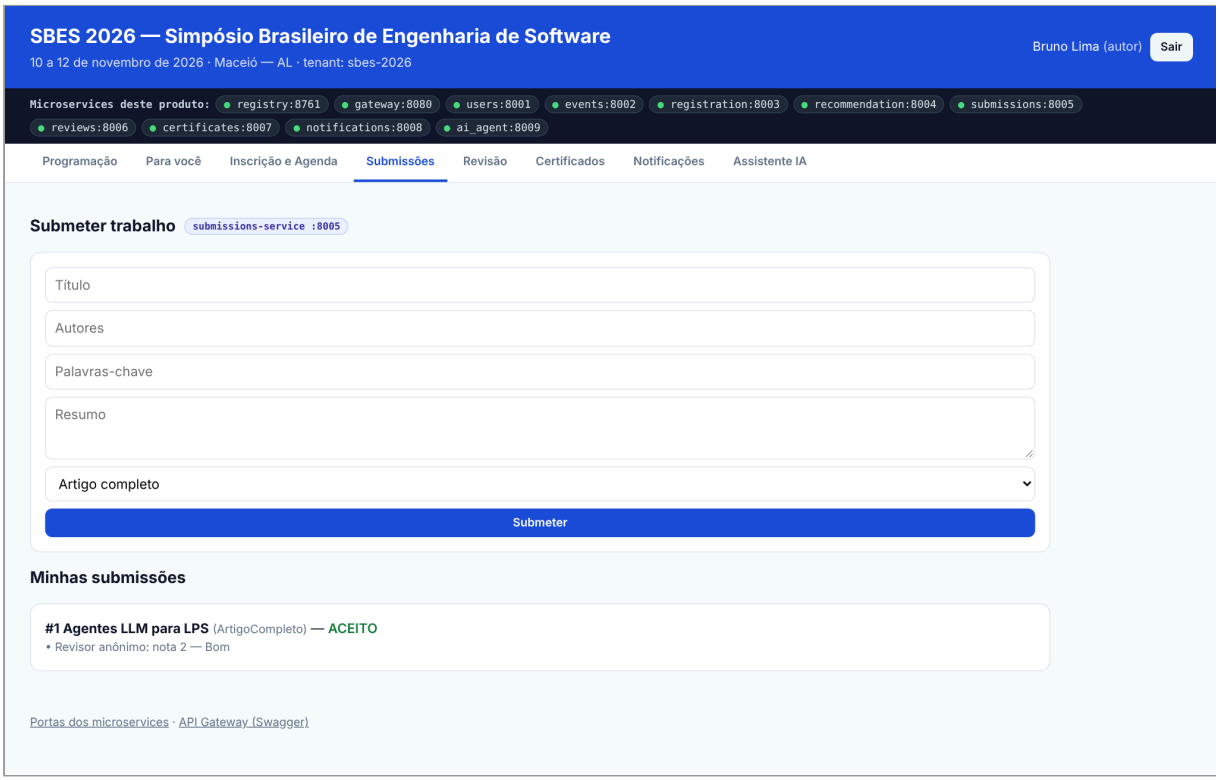
Figura 34 – Inscrição e agenda — Registration Service



Fonte: elaborado pelas autoras (2026).

8.5 Submissões. O autor submete um trabalho nos formatos aceitos (Artigo completo, Resumo expandido) pelo Submissions Service (:8005) e vê o status e o parecer anônimo, por causa da política duplo-cego.

Figura 35 – Submissões — Submissions + Reviews Services



Fonte: elaborado pelas autoras (2026).

8.6 Revisão. A revisora vê a fila com os autores anonimizados (Duplo-cego) e envia o parecer; o Reviews Service (:8006) decide o aceite pela média e atualiza o Submissions Service.

Figura 36 – Revisão duplo-cego — Reviews Service



Fonte: elaborado pelas autoras (2026).

8.7 Certificados. O Certificates Service (:8007) emite Participação e Apresentação com código de verificação; o certificado também pode ser aberto em HTML.

Figura 37 – Certificados — Certificates Service



Fonte: elaborado pelas autoras (2026).

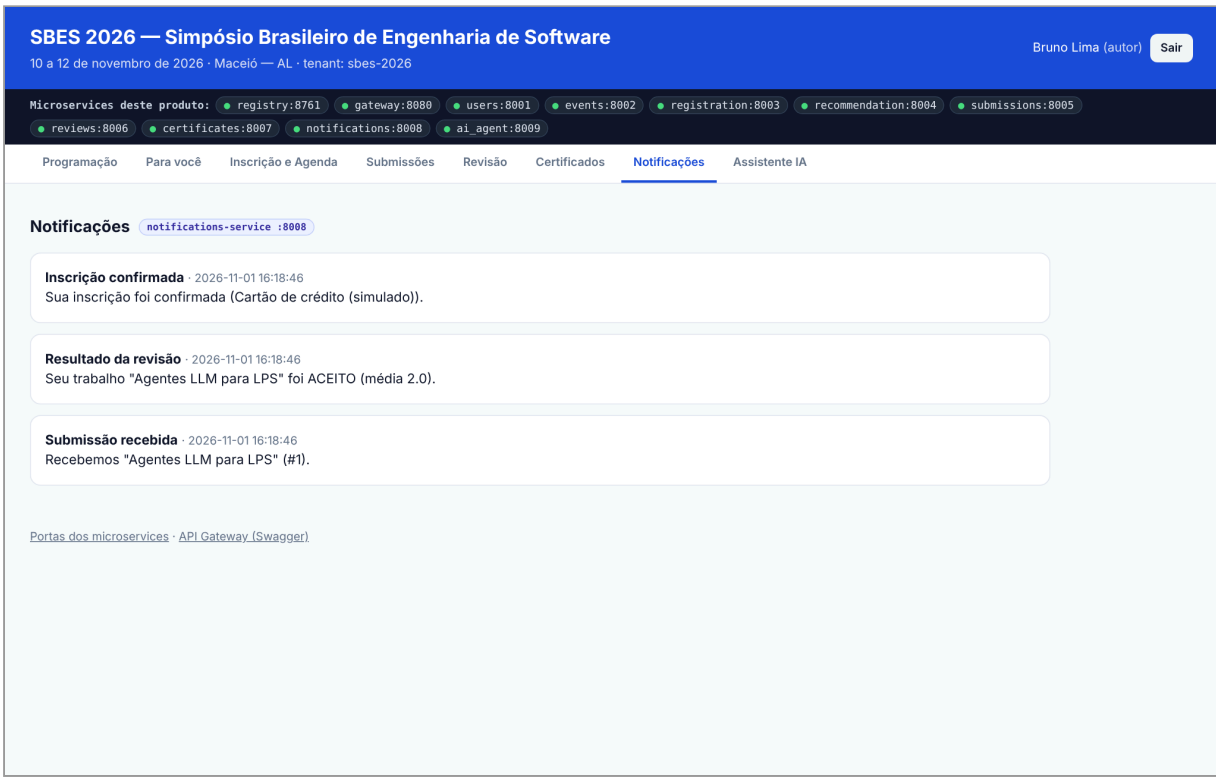
Figura 38 – Certificado em HTML



Fonte: elaborado pelas autoras (2026).

8.8 Notificações. Eventos de domínio de Registration, Submissions e Reviews chegam ao Notifications Service (:8008) e são entregues pelos canais In-app e E-mail (simulado).

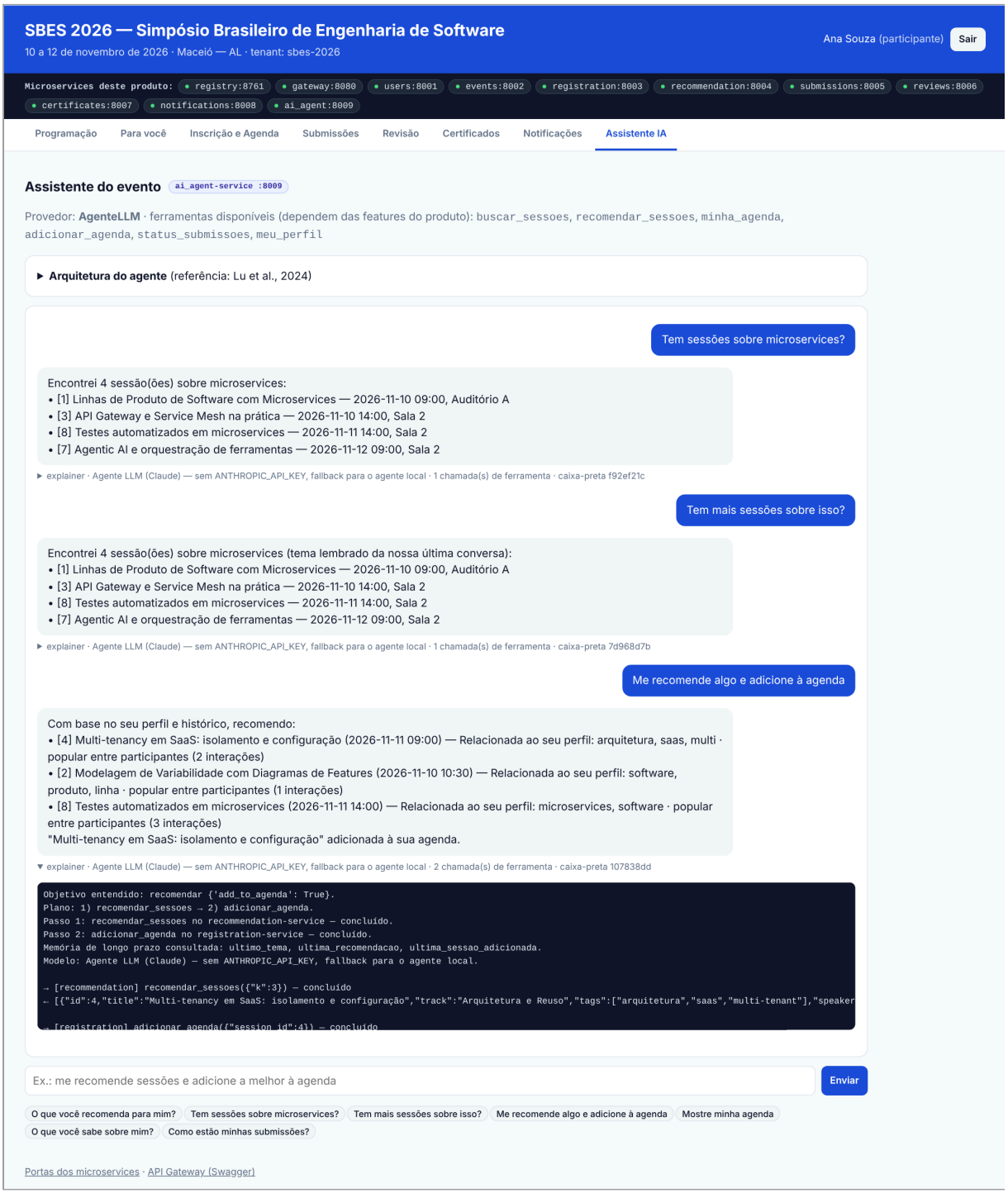
Figura 39 – Notificações — Notifications Service



Fonte: elaborado pelas autoras (2026).

8.9 Assistente com Agente de IA. O AI Agent Service (:8009) responde “Tem sessões sobre microservices?”, depois resolve “Tem mais sessões sobre isso?” pela **memória de longo prazo** (tema lembrado da conversa anterior) e, para “Me recomende algo e adicione à agenda”, gera o plano `recomendar_sessoes → adicionar_agenda`, executa as duas ferramentas e mostra a explicação (*explainer*) com o id da caixa-preta. Como não havia chave de API na captura, a variante LLM caiu para o agente local.

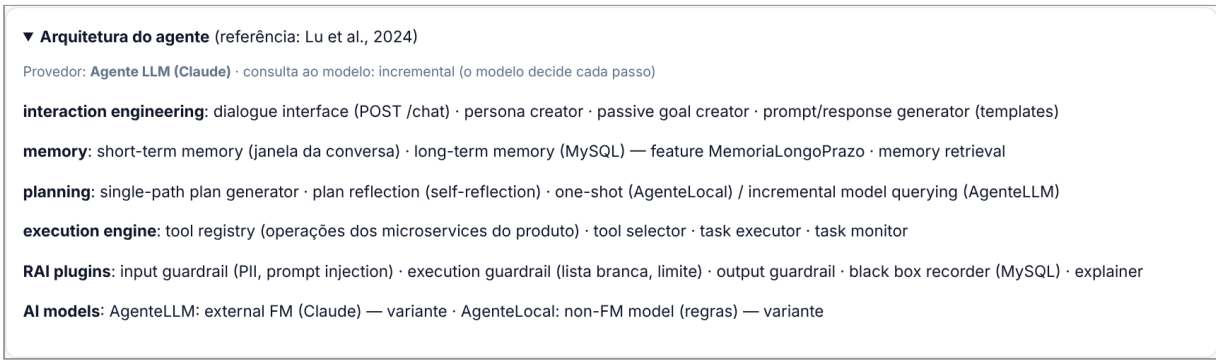
Figura 40 – Assistente IA — AI Agent Service com explainer



Fonte: elaborado pelas autoras (2026).

O painel “Arquitetura do agente” mostra, na própria interface, os componentes da arquitetura de referência usados neste produto:

Figura 41 – Arquitetura do agente exibida na interface



Fonte: elaborado pelas autoras (2026).

9 CÓDIGO NO GITHUB

Todo o código está no repositório indicado na capa: **https://github.com/sthefanyblima/lps-eventos-cientificos**. O `README.md` traz as instruções de execução (instalação, MySQL, derivação, Tela de Configuração, execução, Docker e testes). Estrutura do repositório:

```
feature_model.json     modelo de features (engenharia de domínio)
spl/                   feature model, derivação, CLI, launcher, Tela de Configuração
components/            componentes de software (pontos de variação + variantes)
common/                núcleo comum: service, config, database, security, discovery,
                       client + circuit breaker, eventos
services/registry/     Service Registry (Eureka)
services/<svc>/        controller.py · dto.py · service.py · repository.py · model.py ·
main.py
gateway/               API Gateway (Zuul)
frontend/              interface web do produto
products/              produtos derivados (.json + .compose.yml) e mysql-init.sql
docs/schema_mysql.sql  DDL de todos os bancos
seeds/                 dados de demonstração
tests/                 testes (pytest)
Dockerfile · requirements.txt · README.md
```

Execução rápida:

```
pip install -r requirements.txt
# MySQL: CREATE USER 'lps'@'%' IDENTIFIED BY 'lps123'; GRANT ALL ON `lps\_%`.* TO
'lps'@'%';
python -m pytest -q                    # 13 passed
python -m spl.cli configurator         # http://127.0.0.1:7000
python -m spl.cli run sbes-2026        # http://127.0.0.1:8080  ·  /ports  ·  registry
:8761
# ou, com Docker:  docker compose -f products/sbes-2026.compose.yml up --build
```

Código 38 – terminal — python -m pytest

```
.............                                                            [100%]
13 passed in 0.54s
```

Fonte: elaborado pelas autoras (2026).

10 REFERÊNCIAS BIBLIOGRÁFICAS

ANTHROPIC. **Tool use with Claude**. [S. l.]: Anthropic, 2025. Documentação da Messages API. Disponível em: https://docs.anthropic.com. Acesso em: 1 nov. 2026.

CLEMENTS, Paul; NORTHROP, Linda. **Software product lines**: practices and patterns. Boston: Addison-Wesley, 2001.

CODD, Edgar F. A relational model of data for large shared data banks. **Communications of the ACM**, New York, v. 13, n. 6, p. 377-387, 1970.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2018.

FASTAPI. **FastAPI documentation**. [S. l.], 2026. Disponível em: https://fastapi.tiangolo.com. Acesso em: 1 nov. 2026.

FOWLER, Martin. **Patterns of enterprise application architecture**. Boston: Addison-Wesley, 2002.

FOWLER, Martin; LEWIS, James. **Microservices**: a definition of this new architectural term. [S. l.], 2014. Disponível em: https://martinfowler.com/articles/microservices.html. Acesso em: 1 nov. 2026.

GAMMA, Erich *et al.* **Design patterns**: elements of reusable object-oriented software. Boston: Addison-Wesley, 1994.

JONES, Michael; BRADLEY, John; SAKIMURA, Nat. **JSON Web Token (JWT)**. RFC 7519. [S. l.]: IETF, 2015. Disponível em: https://www.rfc-editor.org/rfc/rfc7519. Acesso em: 1 nov. 2026.

KANG, Kyo C. *et al.* **Feature-Oriented Domain Analysis (FODA) feasibility study**. Pittsburgh: Software Engineering Institute, Carnegie Mellon University, 1990. (Technical Report CMU/SEI-90-TR-21).

LU, Qinghua *et al.* **Towards responsible generative AI**: a reference architecture for designing foundation model based agents. arXiv:2311.13148v3, 2024. Disponível em: https://arxiv.org/abs/2311.13148. Acesso em: 1 nov. 2026.

NEWMAN, Sam. **Building microservices**: designing fine-grained systems. Sebastopol: O'Reilly, 2015.

NYGARD, Michael T. **Release it!**: design and deploy production-ready software. Raleigh: Pragmatic Bookshelf, 2007.

ORACLE. **MySQL 8.0 reference manual**. [S. l.], 2026. Disponível em: https://dev.mysql.com/doc/refman/8.0/en/. Acesso em: 1 nov. 2026.

POHL, Klaus; BÖCKLE, Günter; VAN DER LINDEN, Frank. **Software product line engineering**: foundations, principles and techniques. Berlin: Springer, 2005.

RICHARDSON, Chris. **Microservices patterns**: with examples in Java. Shelter Island: Manning, 2018.

SCHWABER, Ken; SUTHERLAND, Jeff. **The Scrum Guide**. [S. l.], 2020. Disponível em: https://scrumguides.org. Acesso em: 1 nov. 2026.

SQLALCHEMY. **SQLAlchemy 2.0 documentation**. [S. l.], 2026. Disponível em: https://docs.sqlalchemy.org. Acesso em: 1 nov. 2026.

SZYPERSKI, Clemens. **Component software**: beyond object-oriented programming. 2. ed. London: Addison-Wesley, 2002.

TIZZEI, Leonardo P. *et al.* Using microservices and software product line engineering to support reuse of evolving multi-tenant SaaS. *In*: INTERNATIONAL SYSTEMS AND SOFTWARE PRODUCT LINE CONFERENCE, 21., 2017, Sevilla. **Proceedings** […]. New York: ACM, 2017. p. 205-214. DOI: 10.1145/3106195.3106224.

UNIVERSIDADE FEDERAL DE ALAGOAS. **Elementos do relatório do projeto**: disciplina Tópicos em Engenharia de Software. Maceió: UFAL, 2026. Material da disciplina, incluindo a figura *Microservices e Componentes*, o relatório-modelo *SmartClass* e os vídeos de monitoria sobre banco relacional, modelo ER, normalização e MySQL.

VAN GURP, Jilles; BOSCH, Jan; SVAHNBERG, Mikael. On the notion of variability in software product lines. *In*: WORKING IEEE/IFIP CONFERENCE ON SOFTWARE ARCHITECTURE, 2001, Amsterdam. **Proceedings** […]. Los Alamitos: IEEE, 2001. p. 45-54.

WIGGINS, Adam. **The twelve-factor app**. [S. l.], 2017. Disponível em: https://12factor.net. Acesso em: 1 nov. 2026.